



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Operating systems hardening and Buffer Overflow Attacks

materials are partly from security courses at other institutions

OS Hardening



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY

How people see my application



How I see my application



What is OS hardening?

- Hardening process that includes planning, installation, configuration, update, and maintenance of the operating system and the key applications in use.
- OS architecture:

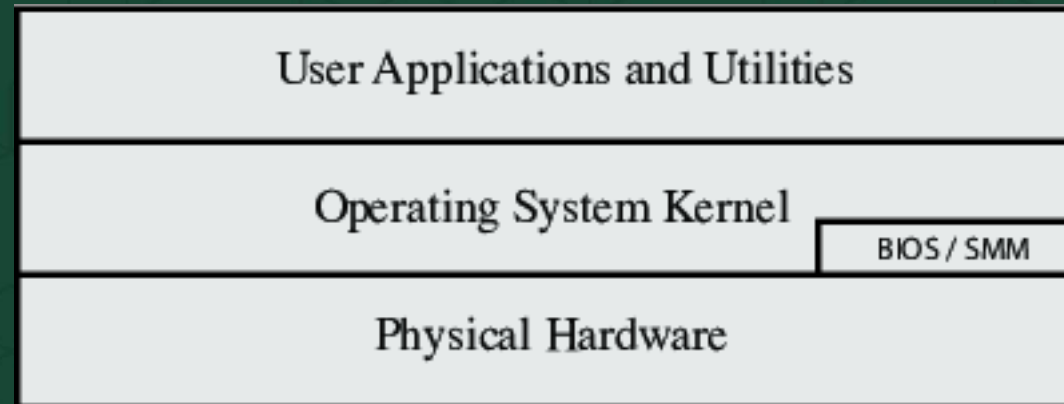
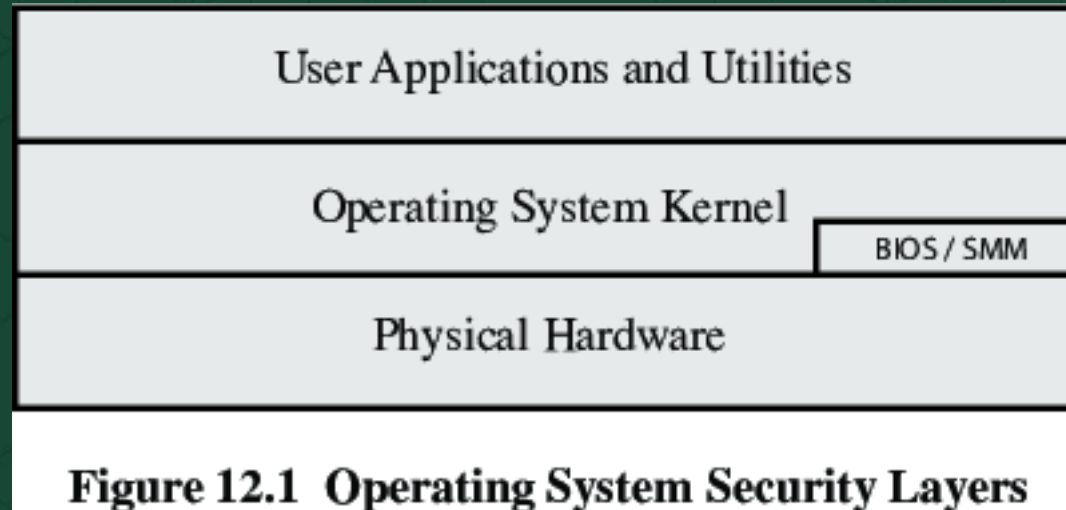


Figure 12.1 Operating System Security Layers

What is OS hardening?



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —



- Each of these layers of code needs appropriate hardening measures in place to provide appropriate security services. And each layer is vulnerable to attack from **below**, should the lower layers not also be secured appropriately. Is the same true from above?

Things to consider during enterprise hardening



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- The **purpose of the system**, the type of information stored, the applications and services provided, and their security requirements.
- The **categories of users** of the system, the privileges they have, and the types of information they can access.
- How the users are **authenticated**.
- How **access** to the information stored on the system is managed.
- What access the system has to **information stored on other hosts**, such as file or database servers, and how this is managed.
- **Who will administer** the system, and how they will manage the system (via local or remote access).
- Any additional security measures required on the system, including the use of host **firewalls**, anti-virus, or other malware protection mechanisms, and logging.

System hardening



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- **Hardware** is the most important layer (in some ultimate context, maybe not a practical one), but a compromise at any layer is sufficient to break any of the CIA triad.
- Lower levels can control or weaken higher layers, but the inverse is not always true.
- Applications and software are easier to verify, design, and program, and are more likely to be open-source.
- Hardware is much less accessible, and manufacturing details are kept secret.
- Most hardware is not open-source, e.g.,
 - Intel chips, firmware blobs, closed BIOS.
 - For an alternative, see IBM's power8, openmoko, and new RISC chips, etc.

Operating system hardening



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- Overview:
- **Install** and **patch** the operating system.
- Harden and configure the operating system to adequately address the identified security needs of the system by:
 - **Removing unnecessary services**, applications, and protocols.
 - Configuring **users, groups, and permissions**.
 - Configuring resource controls.
- Install and configure additional security controls, such as host-based **firewalls** or intrusion detection systems (IDS), if needed.
- **Test** the security of the basic operating system to ensure that the steps taken adequately address its security needs!

Secure installation procedures



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- System security **begins with the installation** of the operating system.
- **Check hash** of install media to insure validity
- **Encrypt full hard disk** during installation
- Install only the **minimum needed software**
- Install in an **isolated environment** (OS is vulnerable until patched)
- Setup BIOS boot and configuration password
- **Avoid installing non-open binary** drivers, especially with kernel access
- **Patch** operating system to most current, and enable updates

Minimize base install

- Minimizing attack surface
- Ideally, install only the **minimum installed software** needed
- However, if a standard install is required, then **remove extras**
- Further, **disable any unneeded protocols and services!**



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Users, Groups, Authentication



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- Determine the needed permissions for each user of the system
- Map user permissions to groups of users
- Minimize the time that administrators will work under the state of elevated privilege
- **Remove any default or guest accounts, change any default passwords!!!**

Buffer Overflow Attack



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- One of the most common software and OS bugs is a buffer overflow
- The developer fails to include code that checks whether an input fits into its buffer array
- An input to the running process exceeds the length of the buffer
- The input string overwrites a portion of the memory of the process
- Causes the application to behave improperly and unexpectedly

History



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- Buffer overflow attack history

1988	The Morris Internet Worm uses a buffer overflow exploit in "fingerd" as one of its attack mechanisms.
1995	A buffer overflow in NCSA httpd 1.3 was discovered and published on the Bugtraq mailing list by Thomas Lopatic.
1996	Aleph One published "Smashing the Stack for Fun and Profit" in <i>Phrack</i> magazine, giving a step by step introduction to exploiting stack-based buffer overflow vulnerabilities.
2001	The Code Red worm exploits a buffer overflow in Microsoft IIS 5.0.
2003	The Slammer worm exploits a buffer overflow in Microsoft SQL Server 2000.
2004	The Sasser worm exploits a buffer overflow in Microsoft Windows 2000/XP Local Security Authority Subsystem Service (LSASS).

Bufferoverflow Note

- You need to have some knowledge of
 - some programming skills
 - assembly and C
 - Linux operating system
 - CPU (x86) architecture



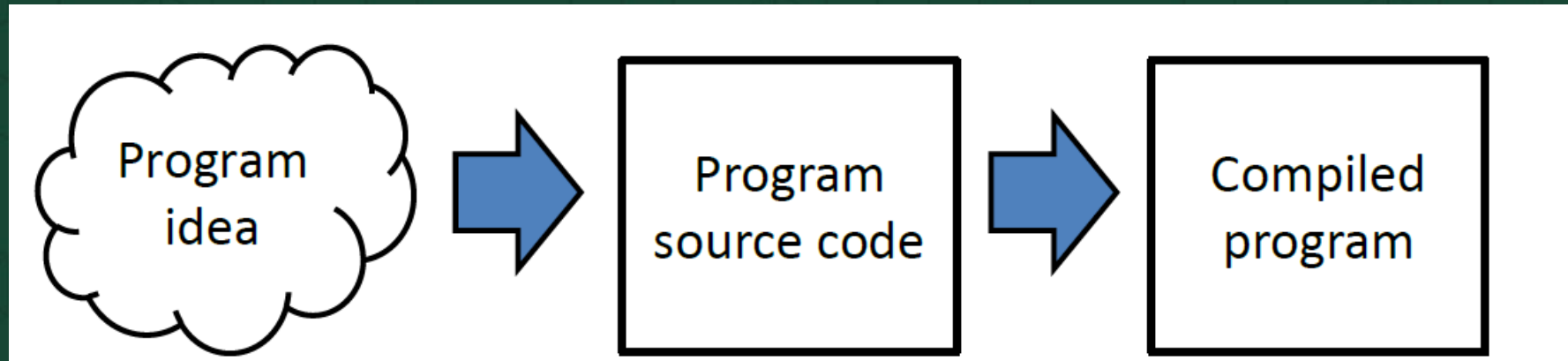
MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

From Concept to Execution

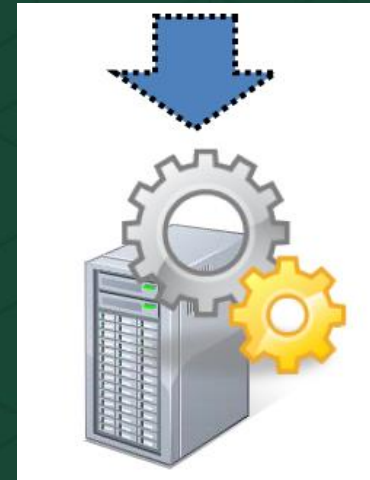


MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY

ROLLA, MISSOURI



What can go wrong?



Objective

- Understand how code is translated and executes at the CPU level.
- Why?
 - Go wrong => possible attack
 - Go wrong => possible fix



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Outline

- **Compilation steps**
- **x86 execution model**
- **Registers**
- **Endianness**
- **Stack**
- **Heap**
- **Basic instructions**



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

The C Language



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- designed to be compiled using a relatively straightforward compiler
 - can be relatively easy translated to assembly
 - low-level access to memory (pointers)
 - extremely efficient
 - Applications
 - operating system, network application, time-critical/resource-constrained programs, system utilities, other computer languages, ...
- Popularity
- Foundation to other computer languages

What Will Executing This Program Do?

```
/* 42.c */  
#include <stdio.h>  
void answer(char *name, int x) {  
    printf("%s, the answer is: %d\n", name, x);  
}  
void main(int argc, char *argv[]) {  
    int x;  
    x = 40 + 2;  
    answer(argv[1], x);  
}
```



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

```
/* 42.c */  
#include <stdio.h>  
void answer(char *name, int x) {  
    printf("%s, the answer is: %d\n", name, x);  
}  
void main(int argc, char *argv[]) {  
    int x;  
    x = 40 + 2;  
    answer(argv[1], x);  
}
```

Compilation

Input: "David"

0101011101
0101010101
0010001111
111

Output: "David, the answer is 42"

Two key questions

- 1. How is C code translated to executable code?**
- 2. What is the machine model for executing code?**



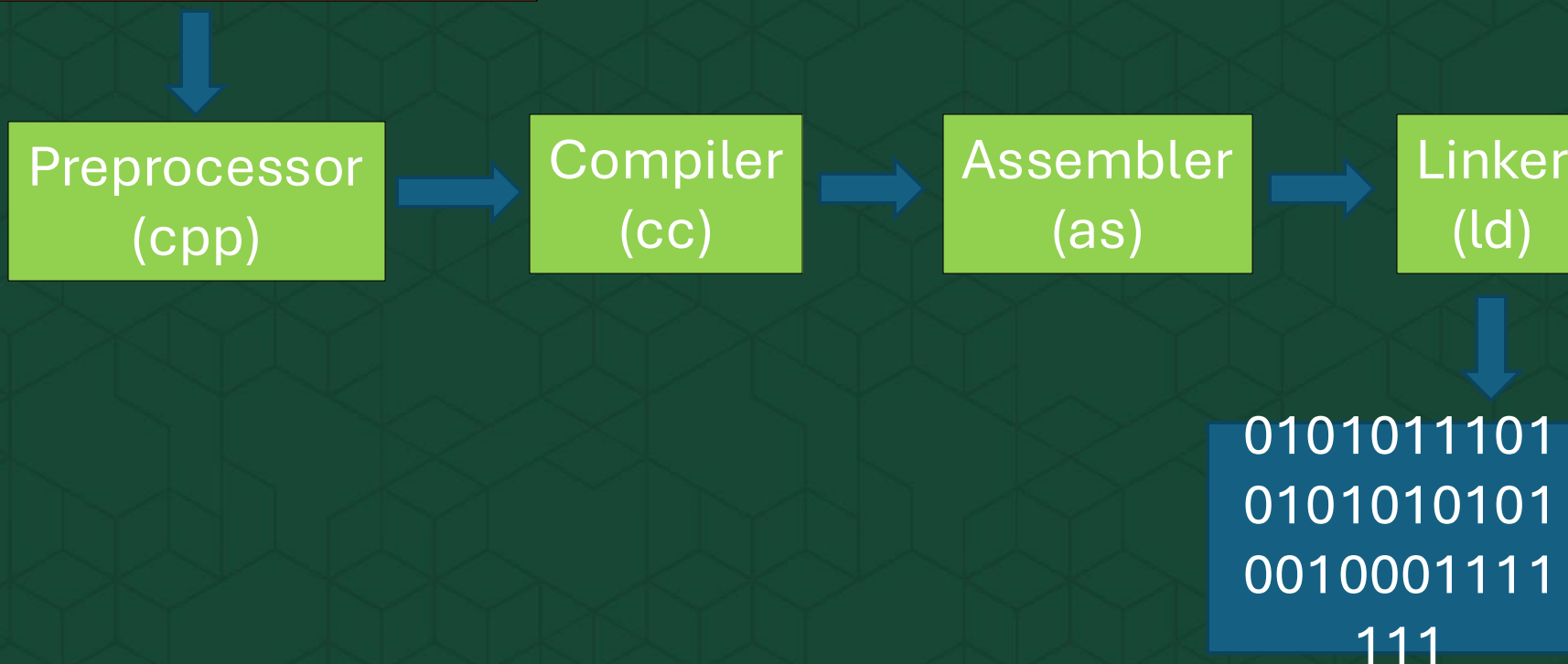
MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Source: 42.c → Executable: 42 in x86

```
/* 42.c */  
#include <stdio.h>  
void answer(char *name, int x) {  
    printf("%s, the answer is: %d\n", name, x);  
}  
void main(int argc, char *argv[]) {  
    int x;  
    x = 40 + 2;  
    answer(argv[1], x);  
}
```



Preprocessor

- Linux command: `cpp`
- `#include` expansion
- `#define` substitution



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

```
/* 42.c */  
#include <stdio.h>  
void answer(char *name, int x) {  
    printf("%s, the answer is: %d\n", name, x);  
}  
...
```

Compiler

- Linux command: gcc -S

```
/* 42.c */  
#include <stdio.h>  
void answer(char *name, int x) {  
    printf("%s, the answer is: %d\n", name, x);  
}  
...
```



```
...  
pushl %ebp  
movl %esp, %ebp  
subl $8, %esp  
...
```

Assembler

- Linux command: `as`

```
...  
pushl %ebp  
movl %esp, %ebp  
subl $8, %esp  
...
```



object/machine code
(`.o` file)



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Linker

- Linux command: ld

object/machine code
(42.o file)



Links with other files and
libraries to produce an exe

executable (42)



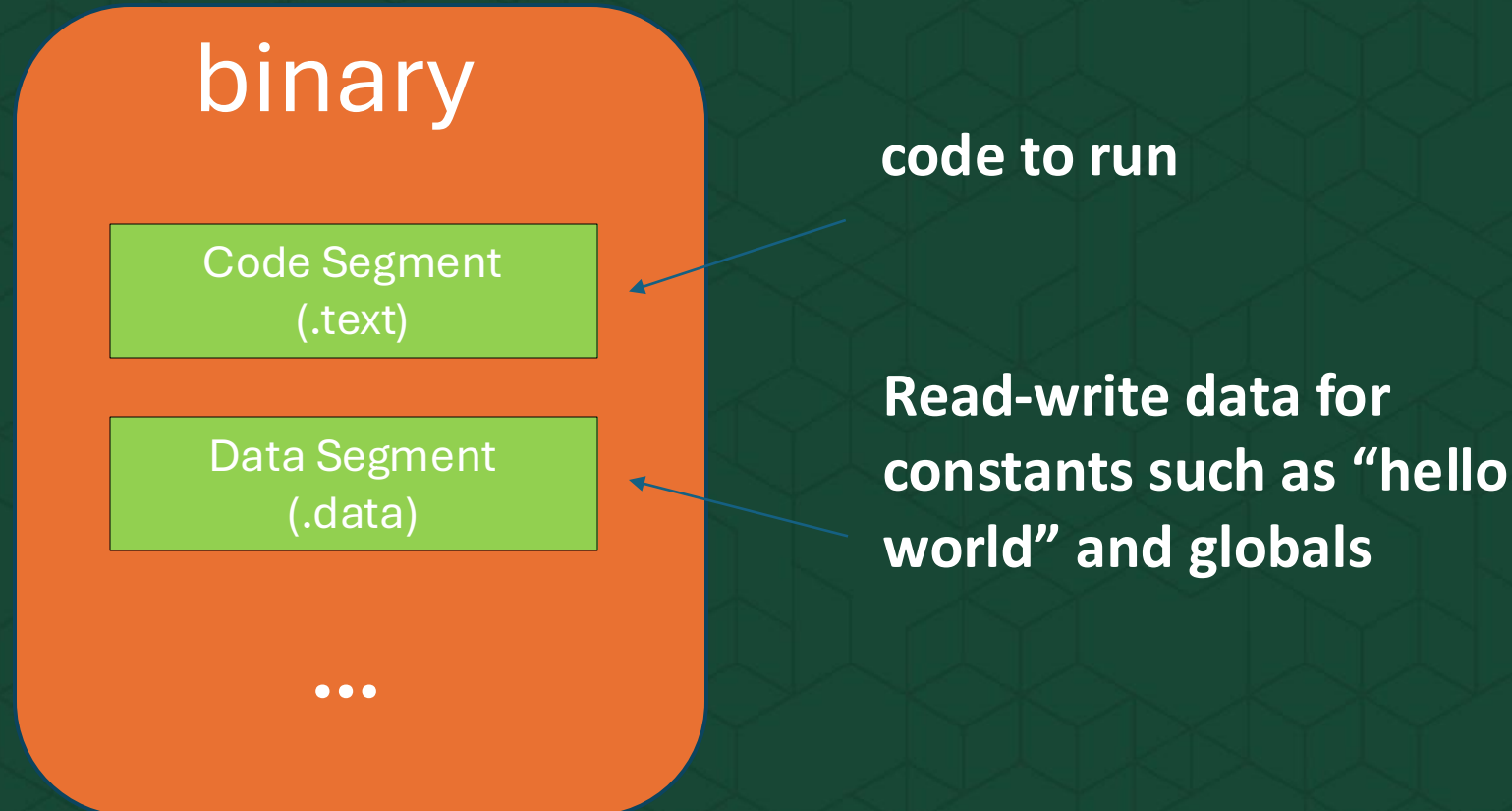
MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Binary/Executable



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

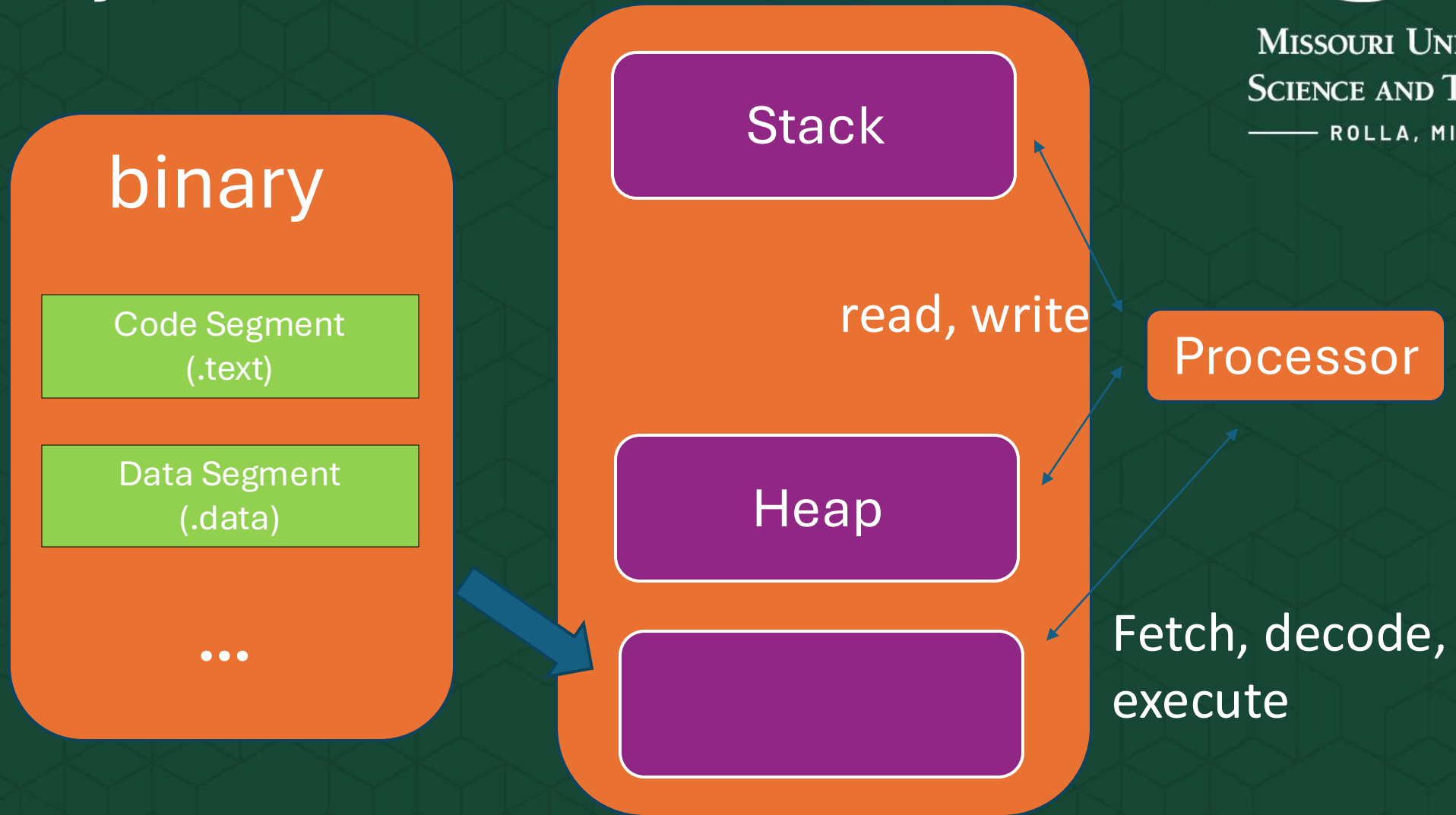
- The program **binary** (aka **executable**) consists of several segments



Binary Code Execution



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —





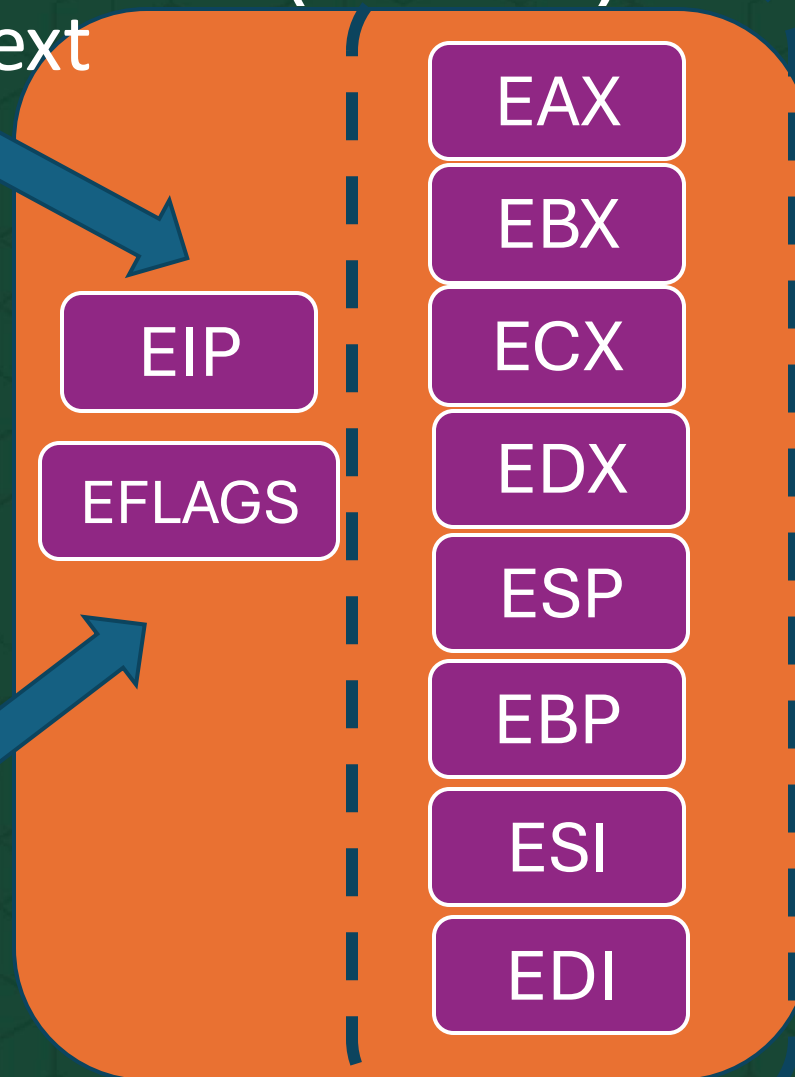
MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

x86 Architecture

The x86 Processor (32-bit)

Address of next
instruction

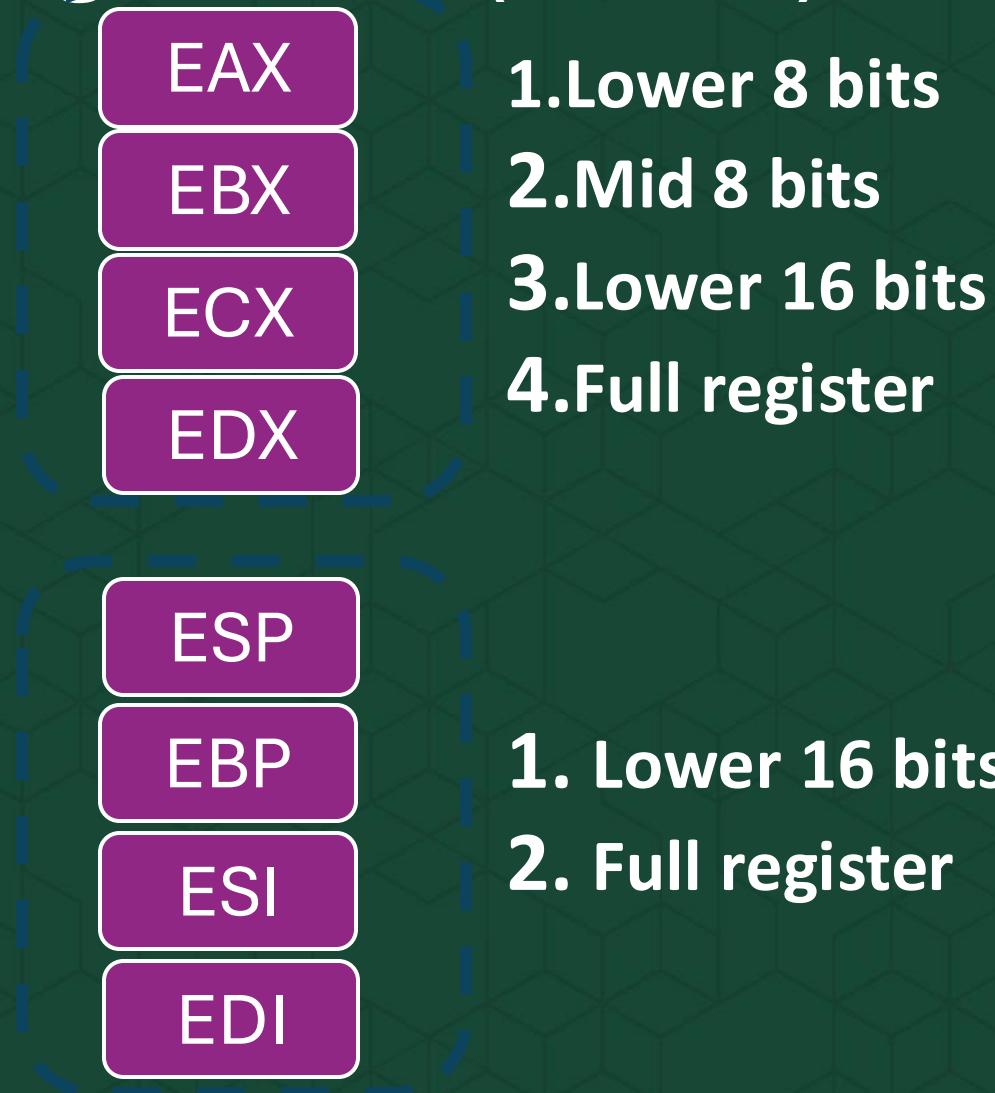
Condition
code



General-purpose
registers

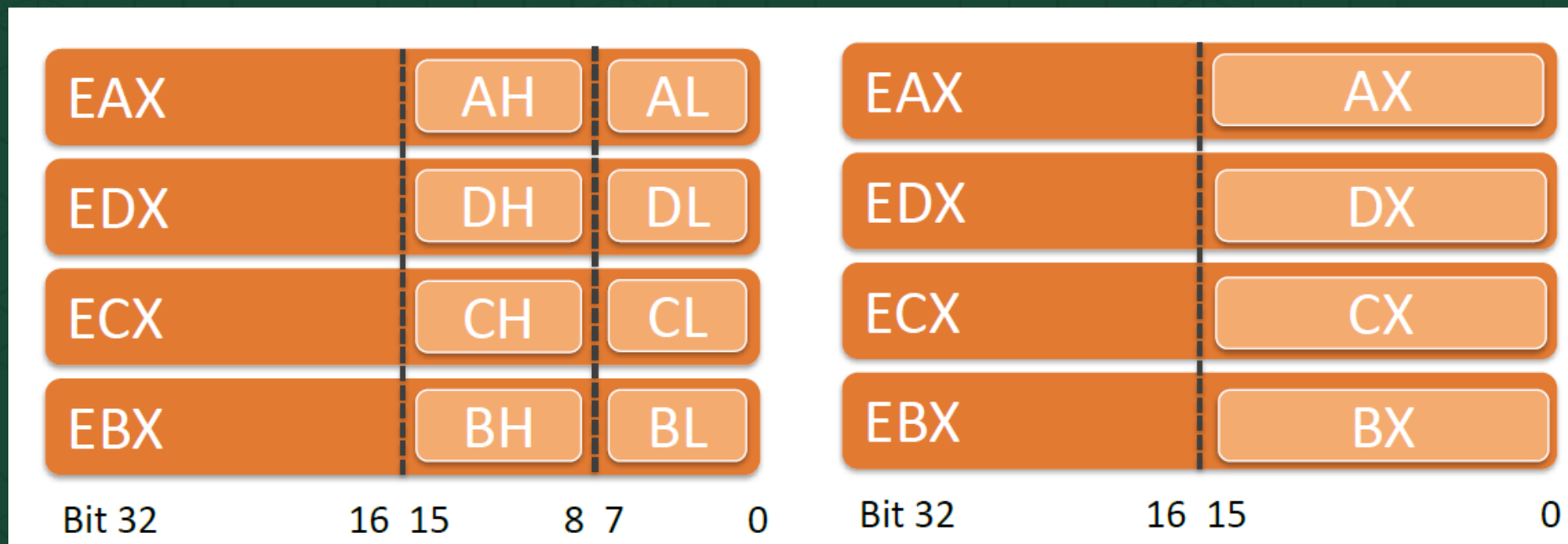
Register: a small
amount of storage
in CPU that can do
computation

Addressing Modes (32-bit)



EAX, EBX, ECX, EDX (32 bits)

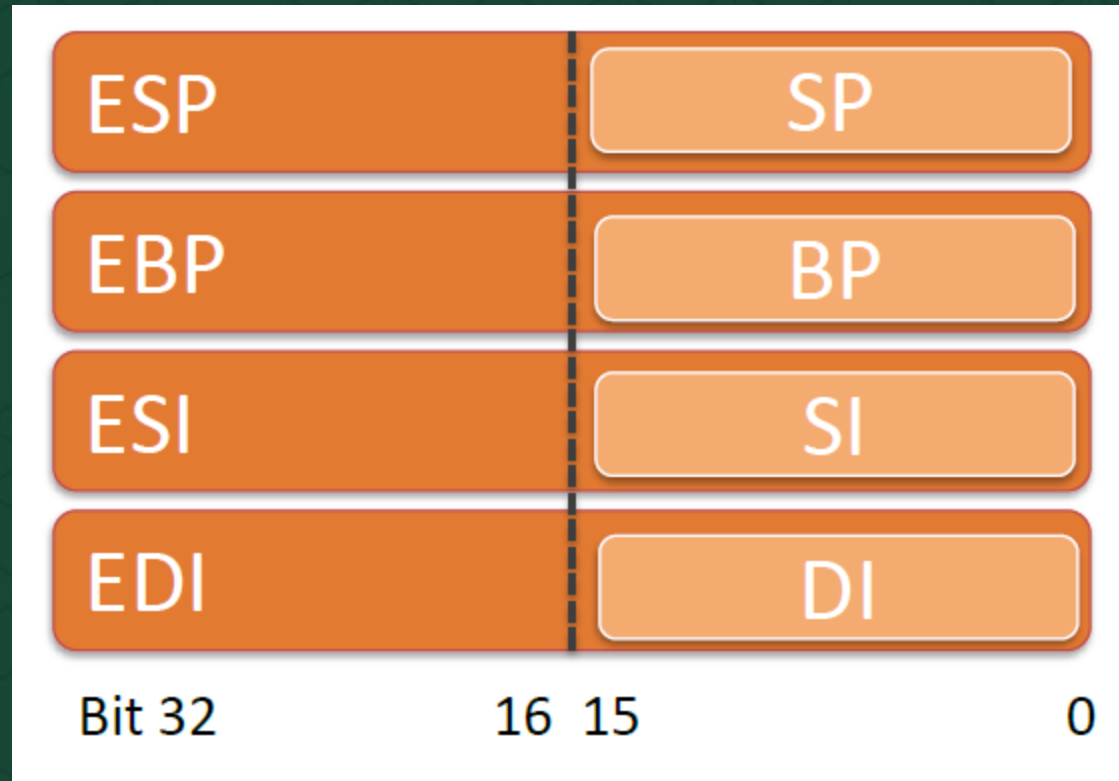
- 32 bit registers (3 letters)
- Lower 16 bits (bits 0-15) (2 letters, ending with X)
- Lower bits (bits 0-7) (two letters, ending with L)
- Mid-bits (bits 8-15) (two letters, ending with H)



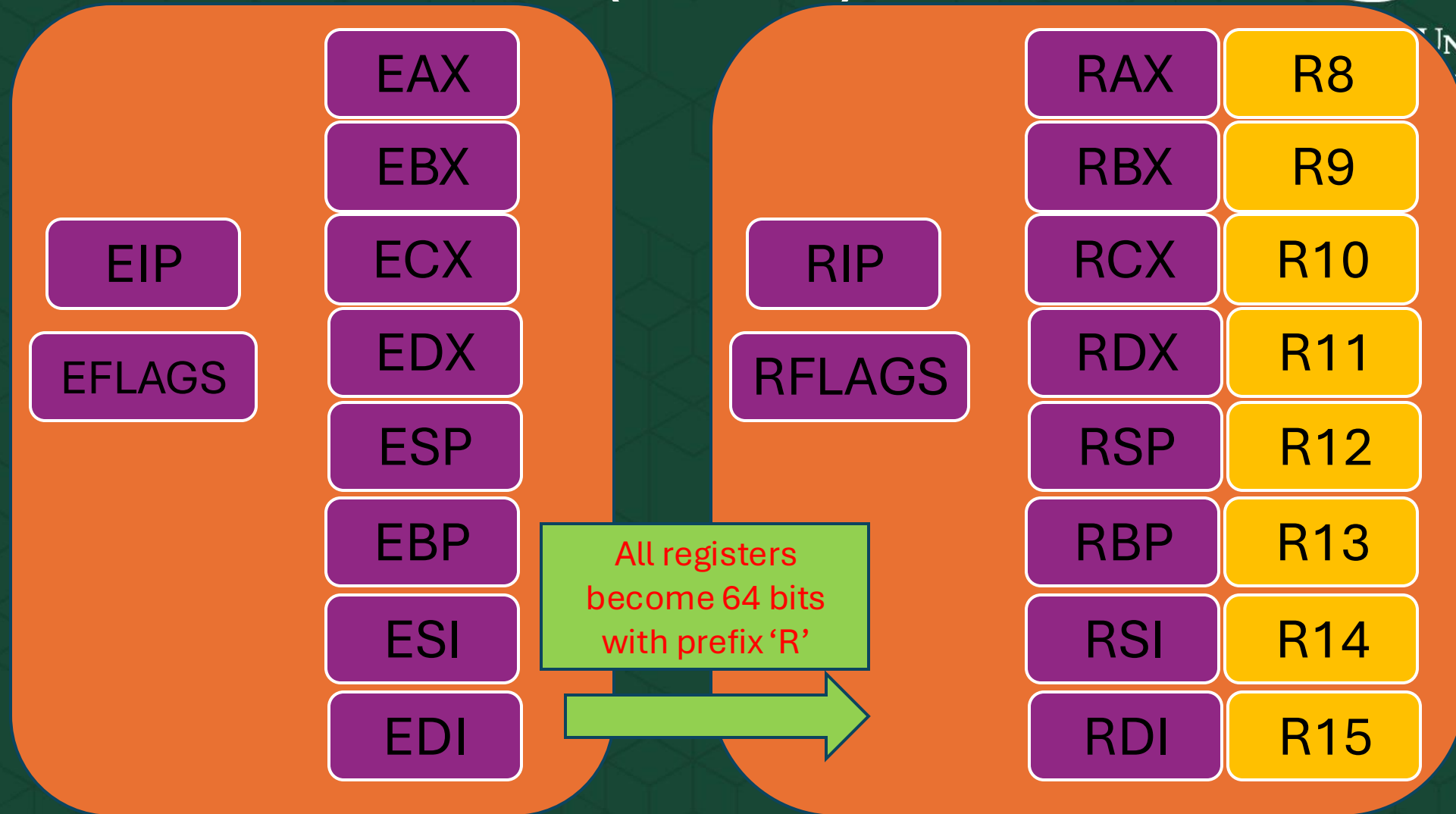
ESP, EBP, ESI, EDI



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —



The x86 Processor (64-bit)





MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Register to register moves



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Register to Register Operations

- AT&T style: <inst> <src> <dest>
 - default for objdump

'%' is needed before a register's name

mov %ebx, %eax
add %ebx, %eax
shl \$2, %ecx

Move contents of ebx into eax
Calculate $\text{eax} = \text{eax} + \text{ebx}$
Calculate $\text{ecx} = \text{ecx} \ll 2$

'\$' is needed to denote a constant (aka immediate)

AT&T vs Intel



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- AT&T
 - mov %ebx, %eax** Move contents of ebx into eax
 - add %ebx, %eax** Calculate $\text{eax} = \text{eax} + \text{ebx}$
 - shl \$2, %ecx** Calculate $\text{ecx} = \text{ecx} \ll 2$
- Intel
 - mov eax, ebx** Move contents of ebx into eax
 - add eax, ebx** Calculate $\text{eax} = \text{eax} + \text{ebx}$
 - shl ecx, 2** Calculate $\text{ecx} = \text{ecx} \ll 2$

Exercise

- Memory can only be read/written
- Registers do all computations
- Question: Describe what instructions are needed to add two numbers in memory.



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Add two numbers in memory.



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

mov address1, %eax Move contents of address1 into eax
mov address2, %ebx Move contents of address 2 into ebx
add %ebx, %eax Calculate $\text{eax} = \text{eax} + \text{ebx}$
mov %eax, address1 Move contents of eax into address1



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Register-memory moves

x86: Byte Addressable Memory



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

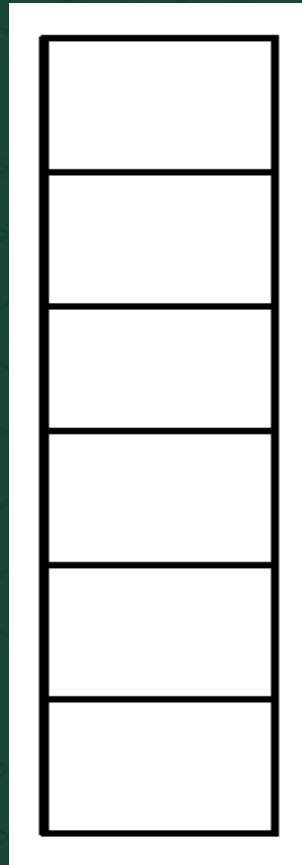
address 4 1 byte

address 3 1 byte

address 2 ...

address 1 ...

address 0 1 byte



Memory is just
like using an array!

It's convention: lower address at the bottom

x86:Addressing Byte

- Addresses are indicated by operands that have a bracket “[]” or paren “()”, depending on Intel vs. AT&T

What does
`mov (%al), %dl`
do?

Move the byte in memory
with address saved in al to
dl

Register	Value
eax	0x3
edx	0x0
ebx	0x5

Addr	
0xff	6
0xee	
0xdd	
0xcc	
0xbb	
0xaa	
0x00	0

mov (%eax), %edx

- Move 4 bytes to edx
- What value can we expect in edx?
 - 1) which 4 bytes do we load?
 - 4-byte load of address a is shorthand for $a, a+1, a+2, a+3$.
 - 2) How are those 4 bytes put a register
 - Endianess

Register	Value
eax	0x3
edx	0x0
ebx	0x5



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Addr	
0xff	6
0xee	
0xdd	
0xcc	
0xbb	
0xaa	
0x00	0

Endianess

- Endianess: Ordering of individually addressable units
- X86 is **Little Endian**:
 - Least significant byte first
 - so ...address a goes in the least significant byte (the littlest bit) $a+1$ goes into the next byte, and so on.

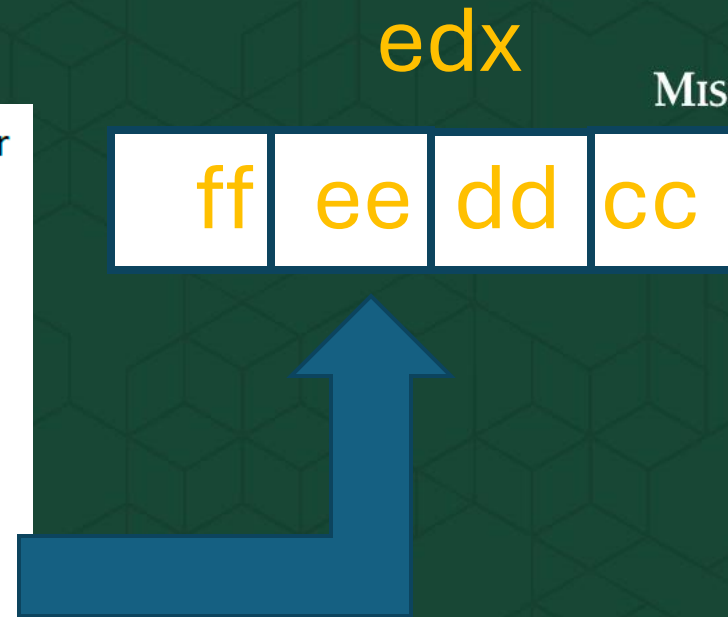


MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

mov(%eax), %edx

Register	Value
eax	0x3
edx	0x0
ebx	0x5

Addr	
6	0xff
	0xee
	0xdd
	0xcc
	0xbb
	0xaa
0	0x00



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
ROLLA, MISSOURI

I want to read memory directly

- `mov 0xdeadbeef,%eax`
 - Load the content at `0xdeadbeef` into `eax`
 - How many bytes are loaded?
 - no “\$” before the address, why?
 - What does `mov $0xdeadbeef,%eax` mean



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Load Effective Address

- Lea instruction
 - Load address not content
- `lea 0xdeadbeef,%eax`
 - compared it with
 - `mov 0xdeadbeef,%eax`
 - `mov $0xdeadbeef, %eax`



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

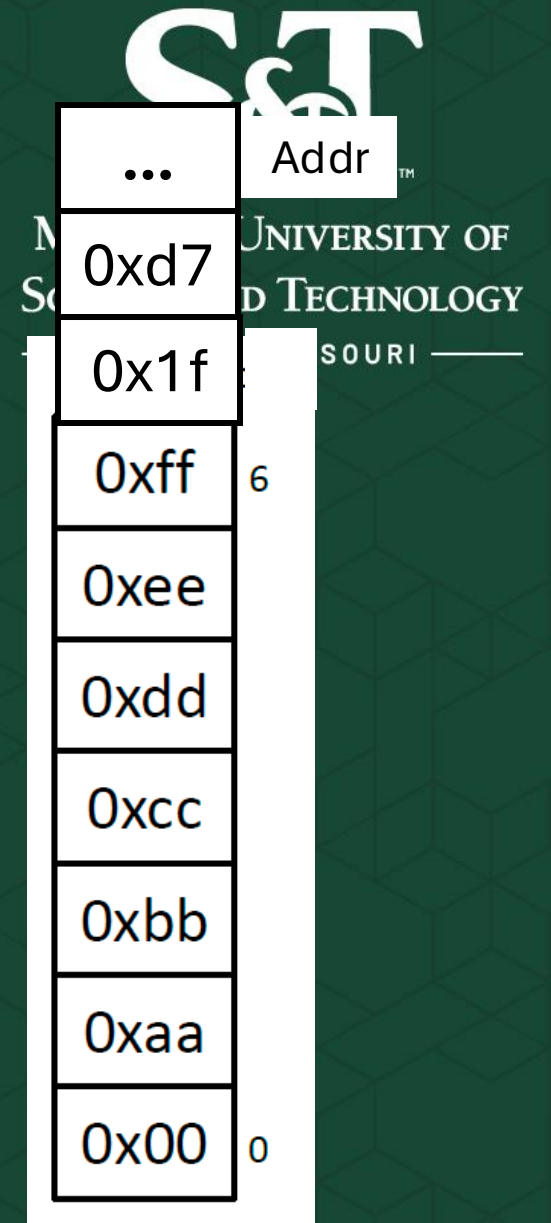
Load Effective Address (II)

- Lea supports address of register + immediate.
 - Starting address plus an offset.
- `lea 0x1(%ebp),%eax`
 - Suppose ebp has value 0xdeadbeef
 - In one instruction, the CPU will compute
 - $ebp + 0x1 = 0xdeadbef0$
 - Then move 0xdeadbef0 to eax
- Effectively support array syntax in C:
 - e.g., `buf[1]`, `buf[20]`.

Examples

- The value of `eax` after
 - `mov $5, %eax`
 - `mov 5, %eax`
 - `mov (%ebx), %eax`
 - `lea (%ebx), %eax`
 - `lea -1(%ebx), %eax`

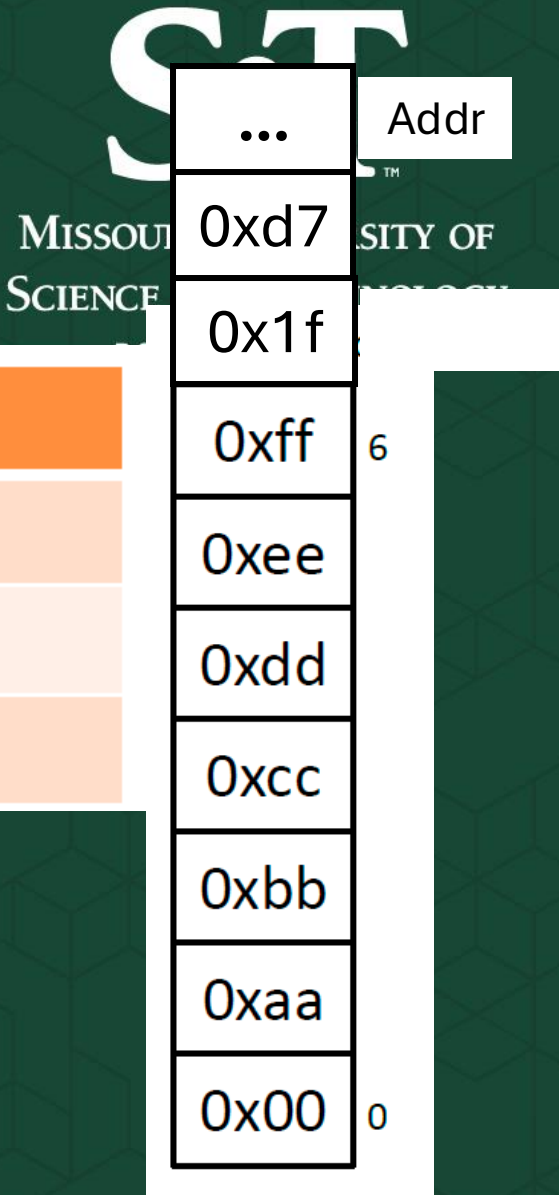
Register	Value
<code>eax</code>	<code>0x3</code>
<code>edx</code>	<code>0x0</code>
<code>ebx</code>	<code>0x5</code>



Examples

- The value of `eax` after
 - `mov $5, %eax` (`eax = 00 00 00 05`)
 - `mov 5, %eax` (`eax = d7 1f ff ee`)
 - `mov (%ebx), %eax` (`eax = d7 1f ff ee`)
 - `lea (%ebx), %eax` (`eax = 00 00 00 05`)
 - `lea -1(%ebx), %eax` (`eax = 00 00 00 04`)

Register	Value
eax	0x3
edx	0x0
ebx	0x5





MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Control Flow

Assembly is “Spaghetti Code”

- **Assembly**
 - Jump
 - Branch
- **Nice C Abstractions**
 - if-then-else
 - while
 - for loops
 - do-while



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Jumps and Branches

No direct way
to get or set EIP

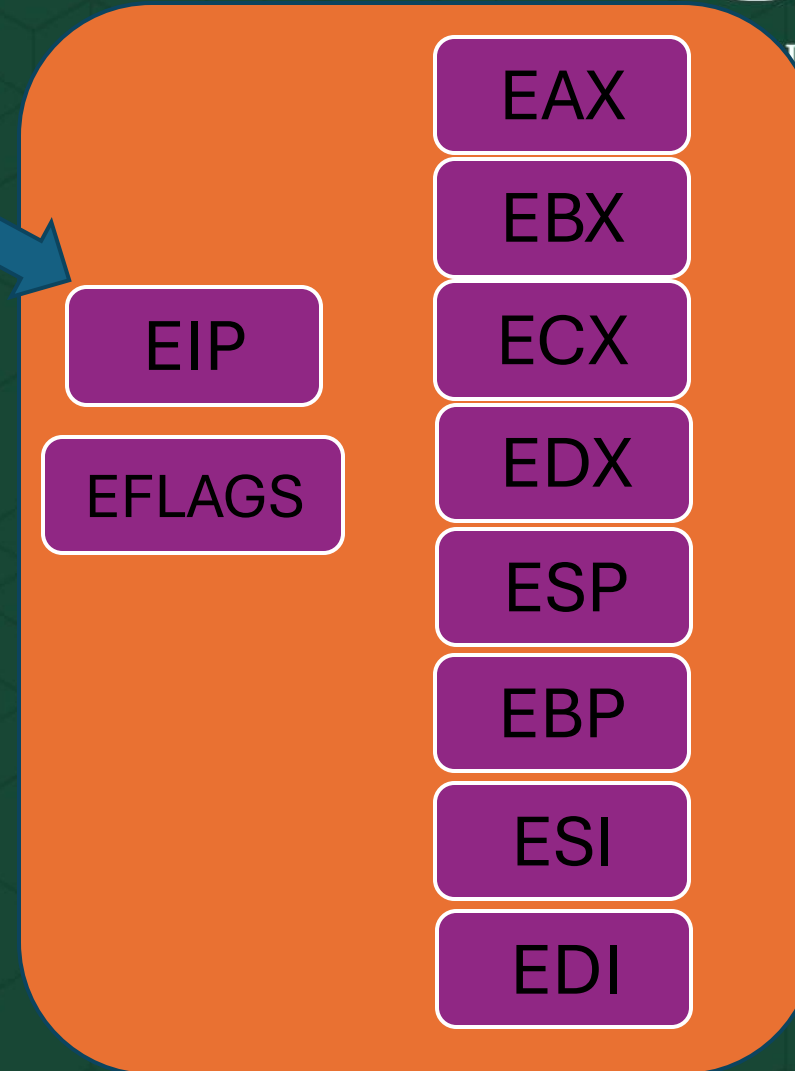


Jumps:

1. **jmp 0x45**, called a *direct jump*
2. **jmp *%eax**, called an *indirect jump*

Branches

- **if (EFLAG) jmp x**
Use one of the 32 EFLAG bits to
determine if jump taken



Control Flow in C

- All C control flow is implemented by
 - A test on operands
 - A branch to a location if true
 - A fall-through to the next instruction if not true



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Implementing “if”



Pseudo-Assembly
MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
MISSOURI

C code

```
If (x <= y)
    z = x;
else
    z = y;
```

A. Test if $x \leq y$ by computing $x - y$.

Set EFLAGS

- CF set if $x < y$
- ZF set if $x == y$

B. Test EFLAGS.

- If both CF and ZF not set, branch to E

C. *Fall-through:* `mov x, z`

D. Jump to F

E. `mov y, z`

F. <end of if-then-else>

Setting EFLAGS

- **Instructions may set an eflag, e.g.,**
- **“cmp” and arithmetic instructions most common**
 - Was there a carry (CF Flag set)
 - Was the result zero (ZF Flag set)
 - What was the parity of the result (PF flag)
 - Did overflow occur (OF Flag)
 - Is the result signed (SF Flag)



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Conditional Jumps



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Instr.	Description
JO	Jump if overflow
JNO	Jump if not overflow
JS	Jump if sign
JZ	Jump if zero
JE	Jump if equal
JL	Jump if less than
JLE	Jump if less than or equal
JB	Jump if below
JP	Jump if parity

Program memory organization and functions



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

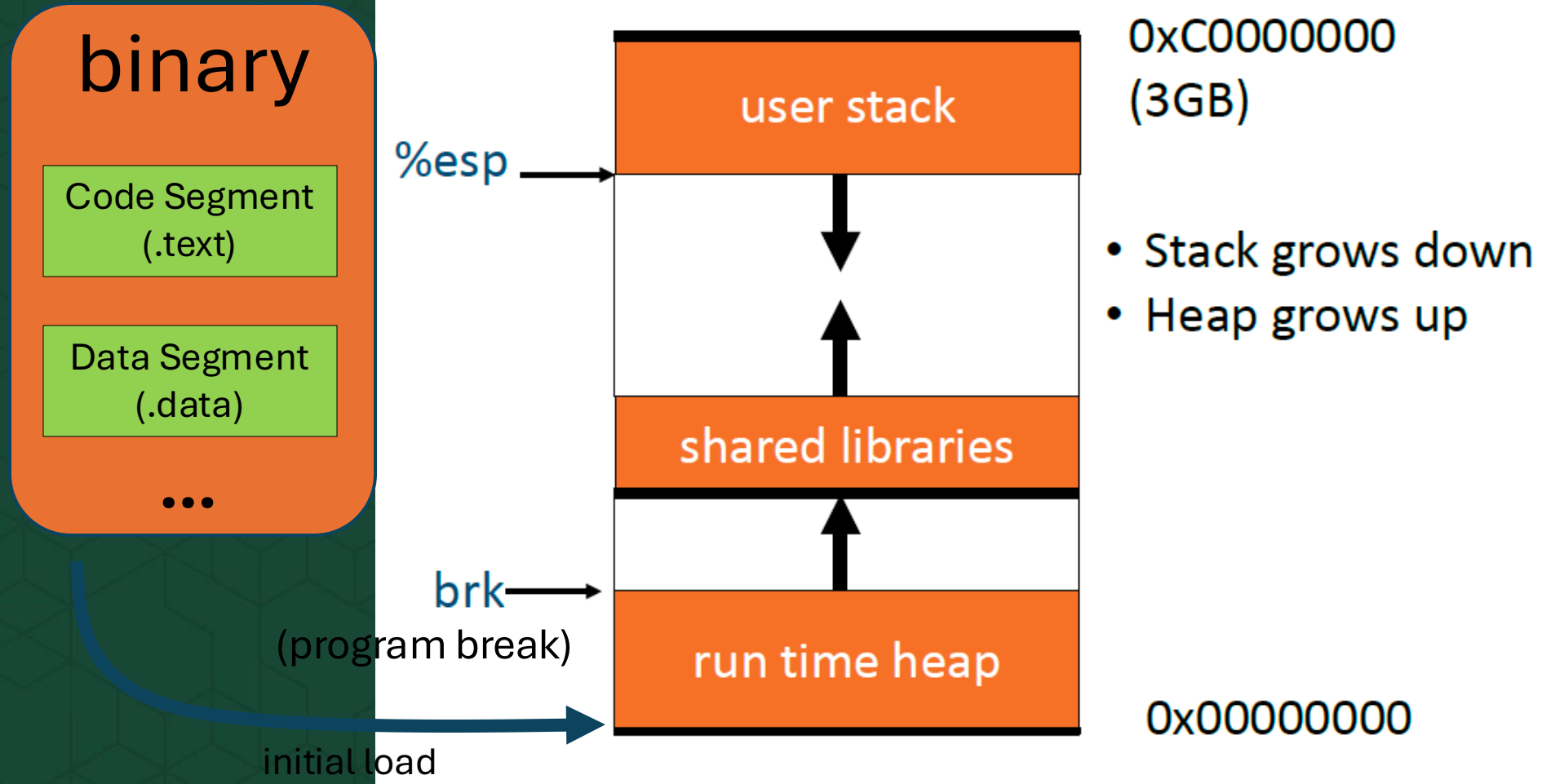
Memory for A Process

- 32-bit in Linux (4GB)
 - First 3GB for user, last 1GB for kernel.
- Each process has an independent view of all 3GB memory.
 - Virtual memory via memory mapping
 - Software and hardware support
 - Benefits?



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

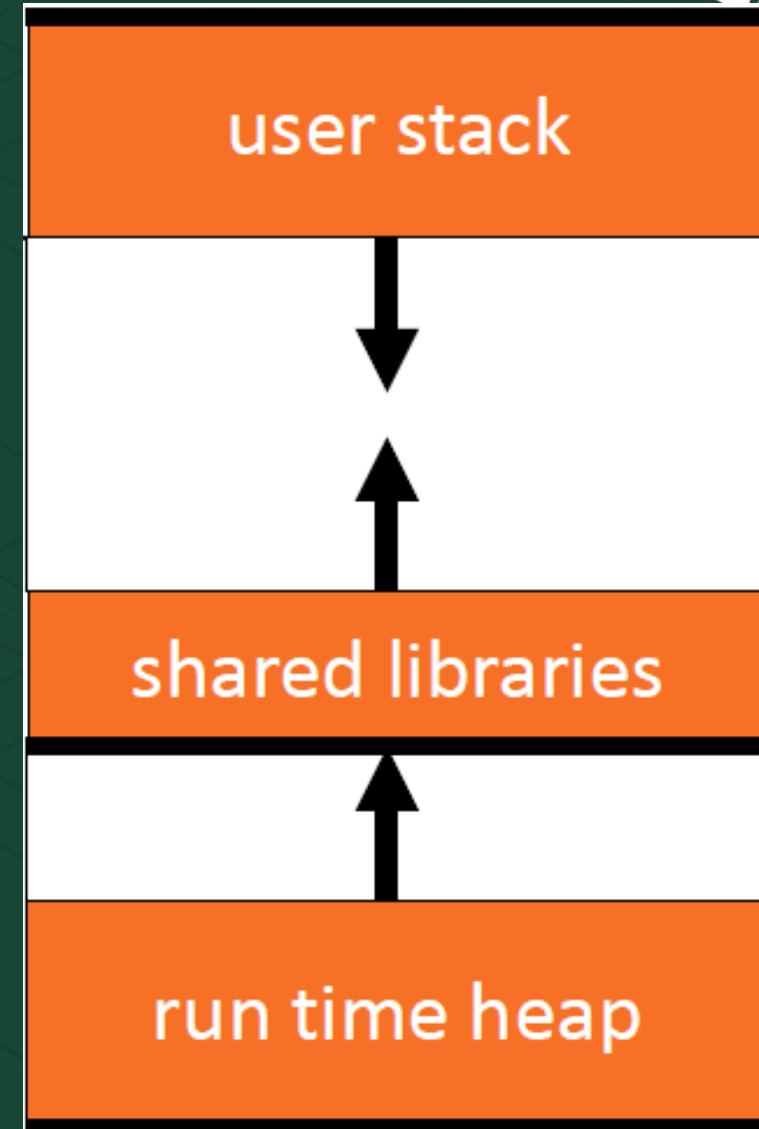
Memory Space for a Process



Variables

- On the stack:
 - Local variables
- On the heap:
 - Dynamically allocated variables

```
void function(int a) {  
    int b, c;  
    int* p = (int*) malloc(  
        sizeof(int) * 100);  
    ...  
}
```



Glibc Call and System Call



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- **The GNU C Library (glibc)**
 - Implementation of all standard C functions
 - E.g.,
 - printf
 - scanf
 - malloc ← Our example
 - ...
 - System call
 - Interface for the program to request service from the operating system.

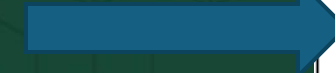
Glibc in Memory

The memory boundaries are determined by the operating system.

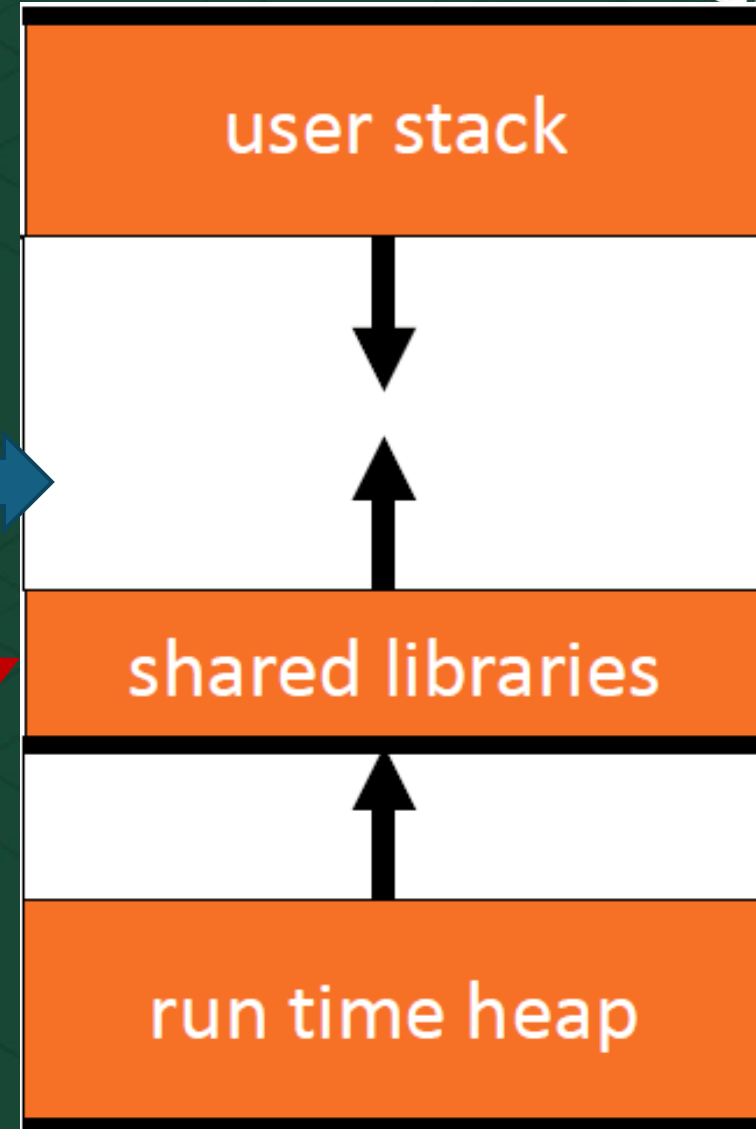
If the user program tried to access memory outside the boundary, the operating system will give “**segmentation fault**” error

```
void function(int a) {  
    int b, c;  
    int* p = (int*) malloc(  
        sizeof(int) * 100);  
    ...  
}
```

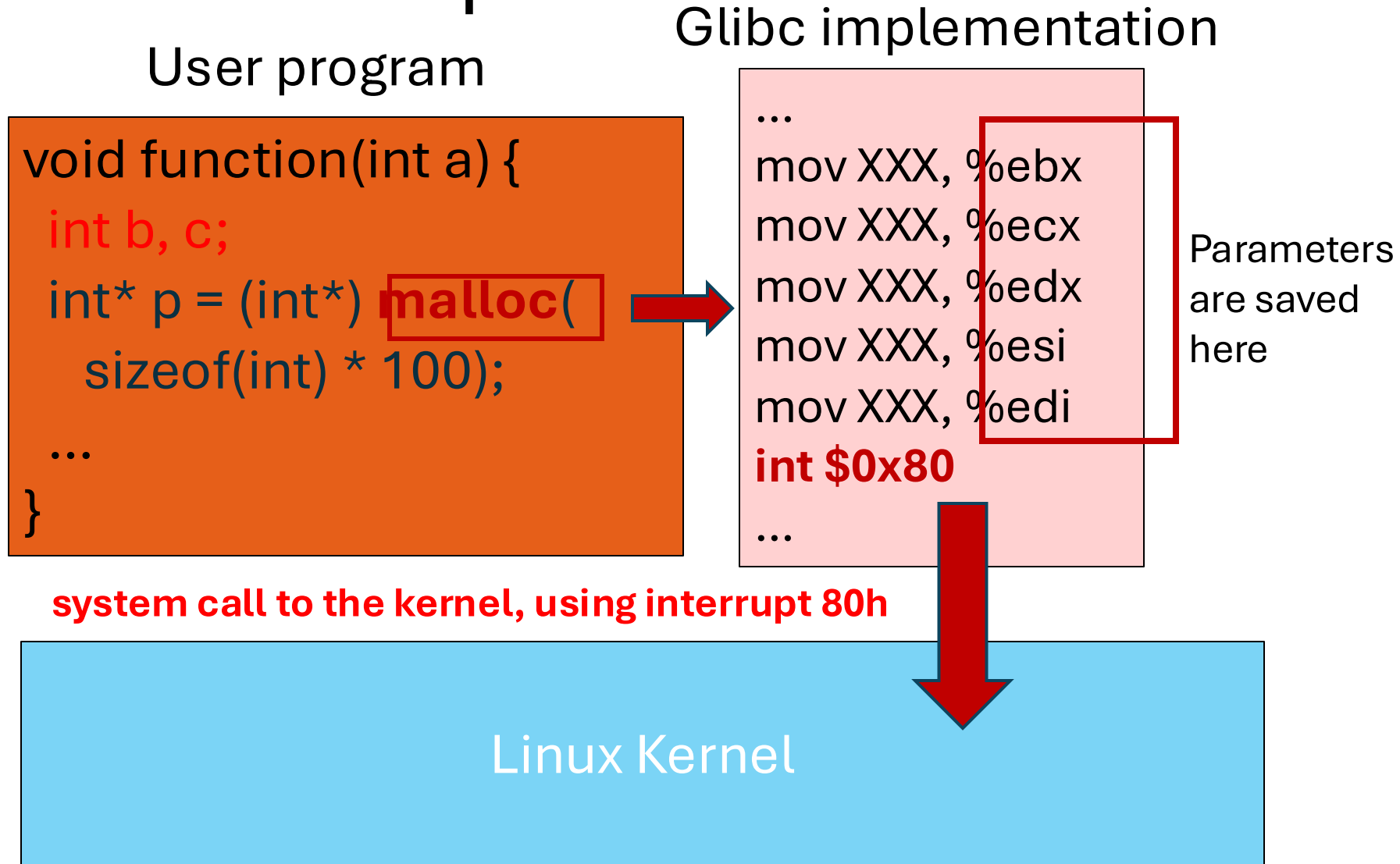
Not accessible!



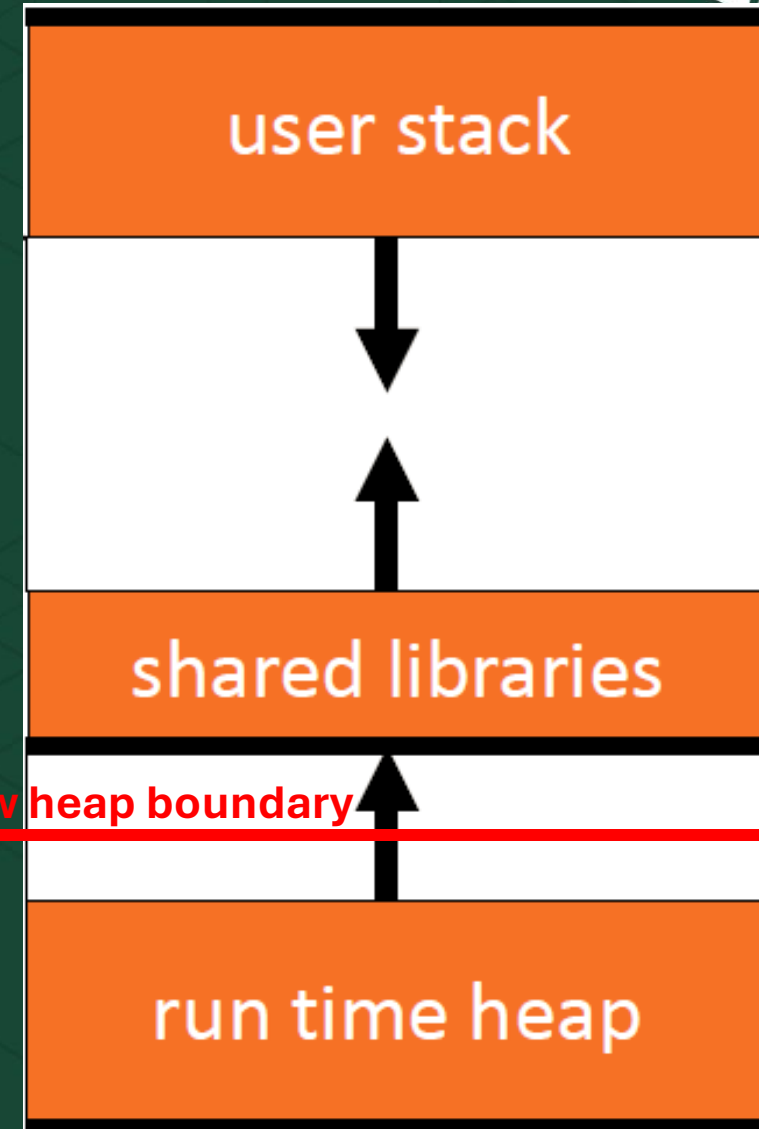
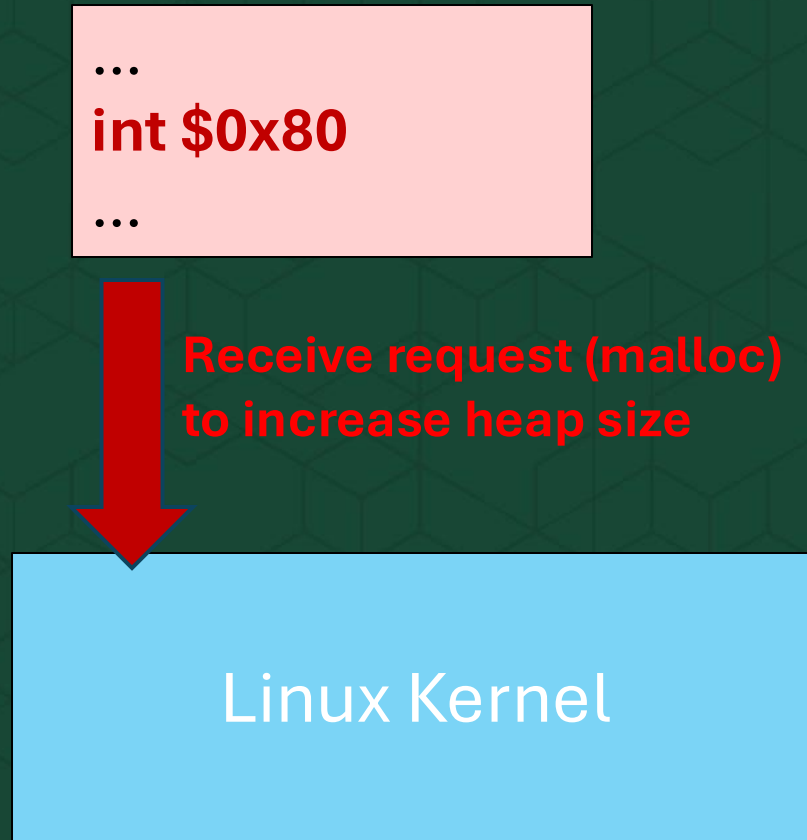
I need more memory!



How Glibc Is Implemented



Handling in Kernel

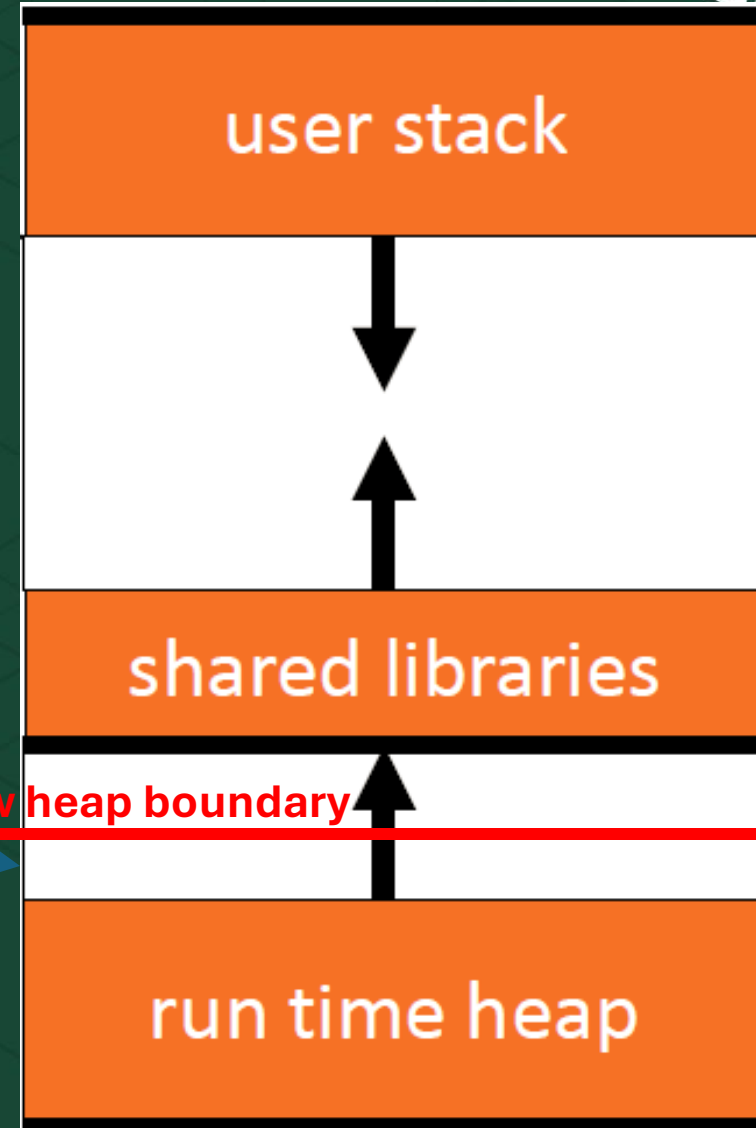


Back to User

User program

```
void function(int a) {  
    int b, c;  
    int* p = (int*) malloc(  
        sizeof(int) * 100);  
  
    ...  
    b = *(p+20) ;  
}
```

access memory here
No violation!

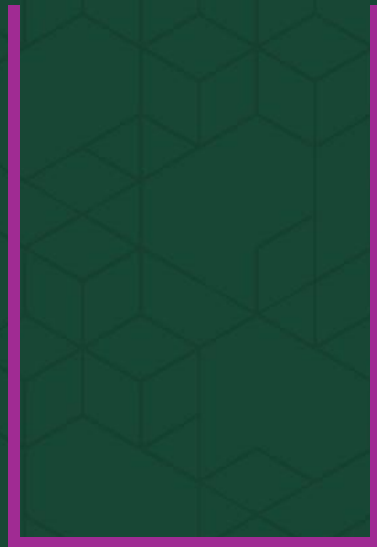




MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

User stack

- User stack in memory is slightly different from the data structure of “stack” that we previously learned

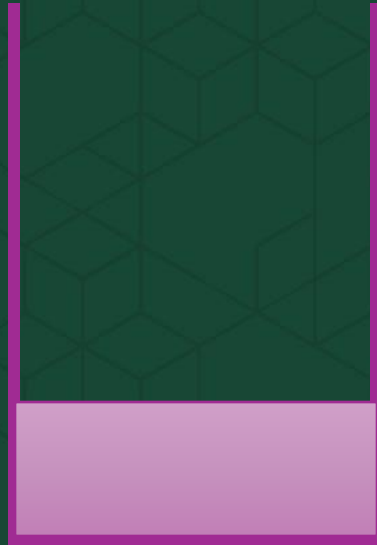


Push elements

Example of stack that we previously learned from data structure course

User stack

- User stack in memory is slightly different from the data structure of “stack” that we previously learned

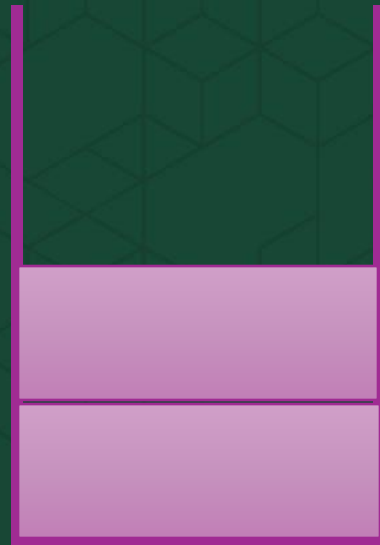


Push elements

Example of stack that we previously learned from data structure course

User stack

- User stack in memory is slightly different from the data structure of “stack” that we previously learned



Push elements

Example of stack that we previously learned from data structure course

User stack

- User stack in memory is slightly different from the data structure of “stack” that we previously learned

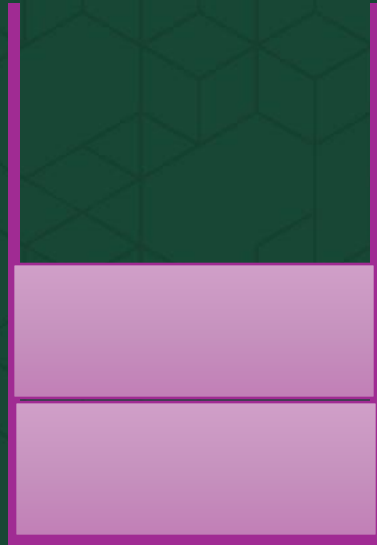


Push elements

Example of stack that we previously learned from data structure course

User stack

- User stack in memory is slightly different from the data structure of “stack” that we previously learned

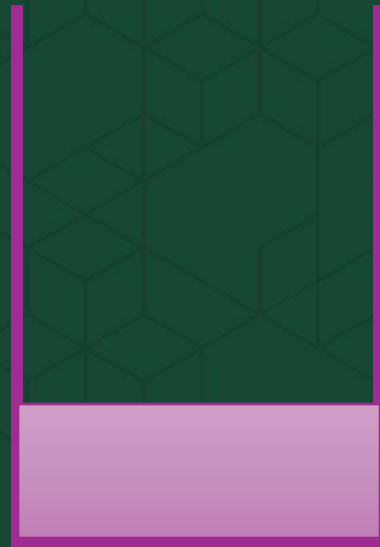


Pop elements

Example of stack that we previously learned from data structure course

User stack

- User stack in memory is slightly different from the data structure of “stack” that we previously learned



Pop elements

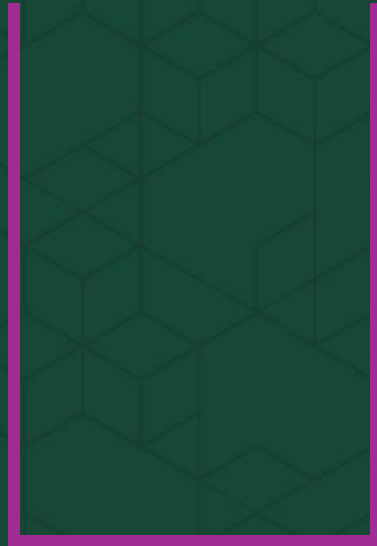
Example of stack that we previously learned from data structure course



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

User stack

- User stack in memory is slightly different from the data structure of “stack” that we previously learned



Pop elements

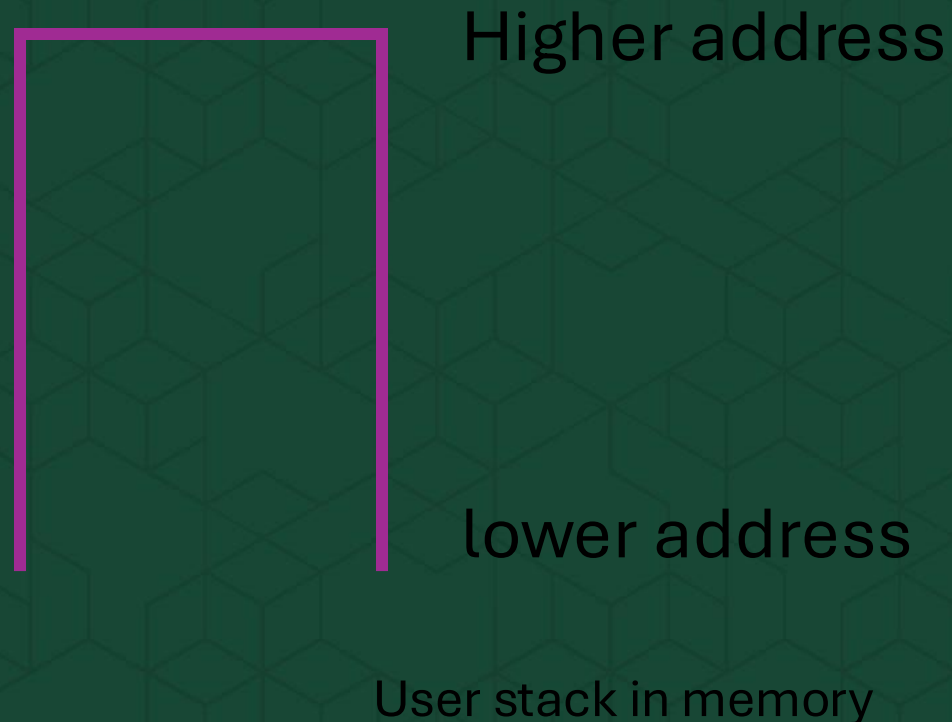
Example of stack that we previously learned from data structure course



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

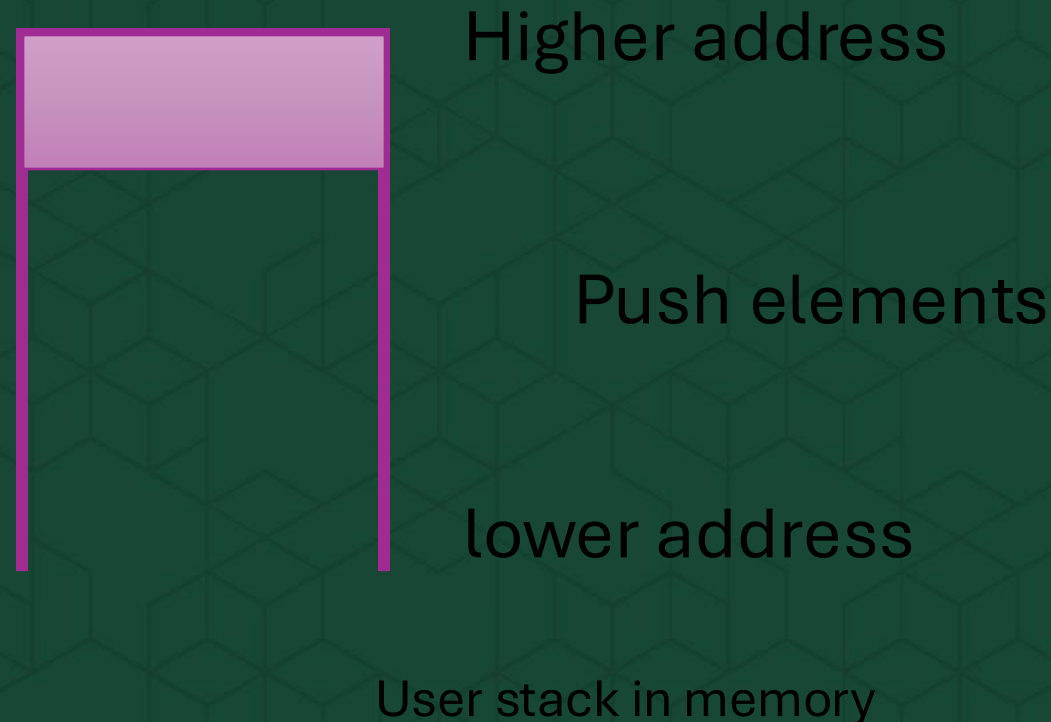
User stack

- User stack in memory grows downwards from higher address to lower address



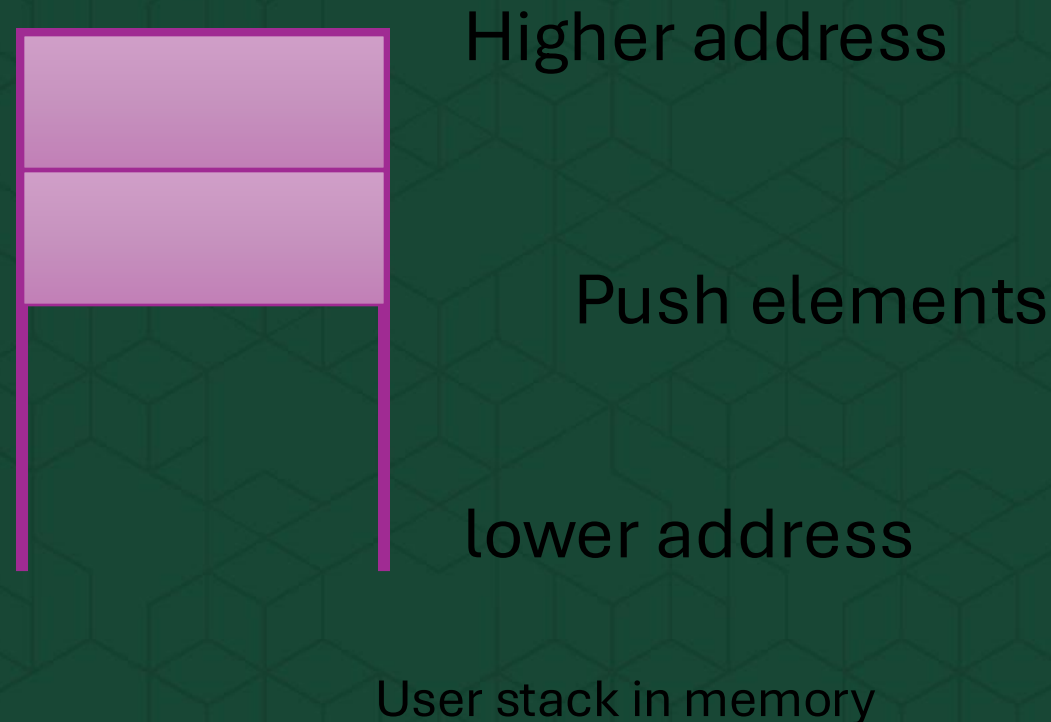
User stack

- User stack in memory grows downwards from higher address to lower address



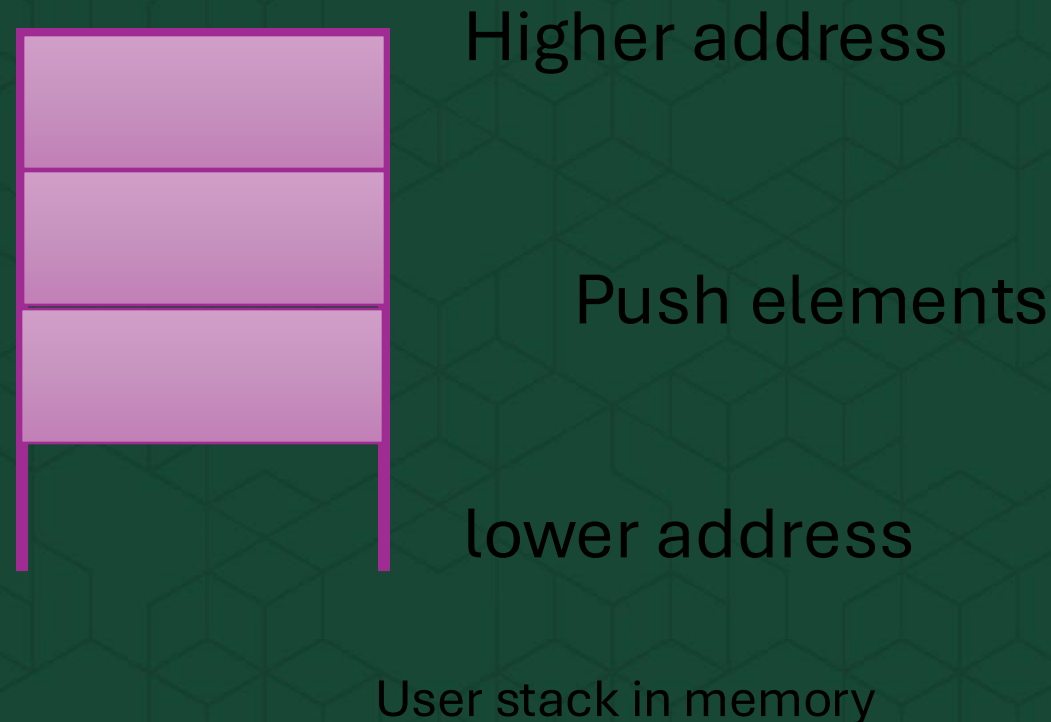
User stack

- User stack in memory grows downwards from higher address to lower address



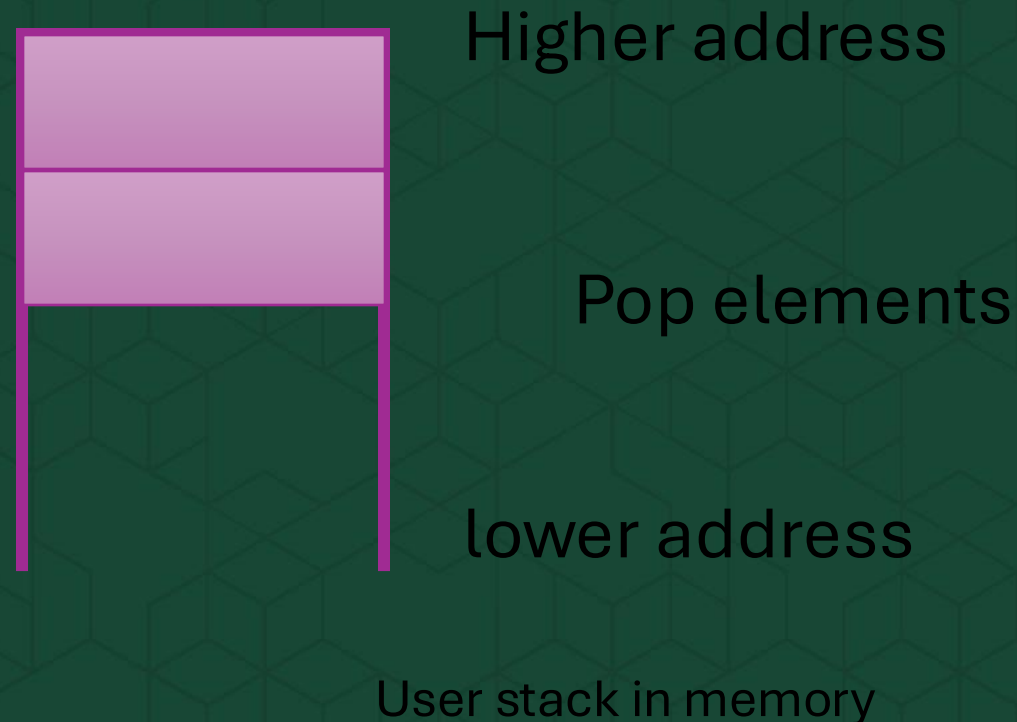
User stack

- User stack in memory grows downwards from higher address to lower address



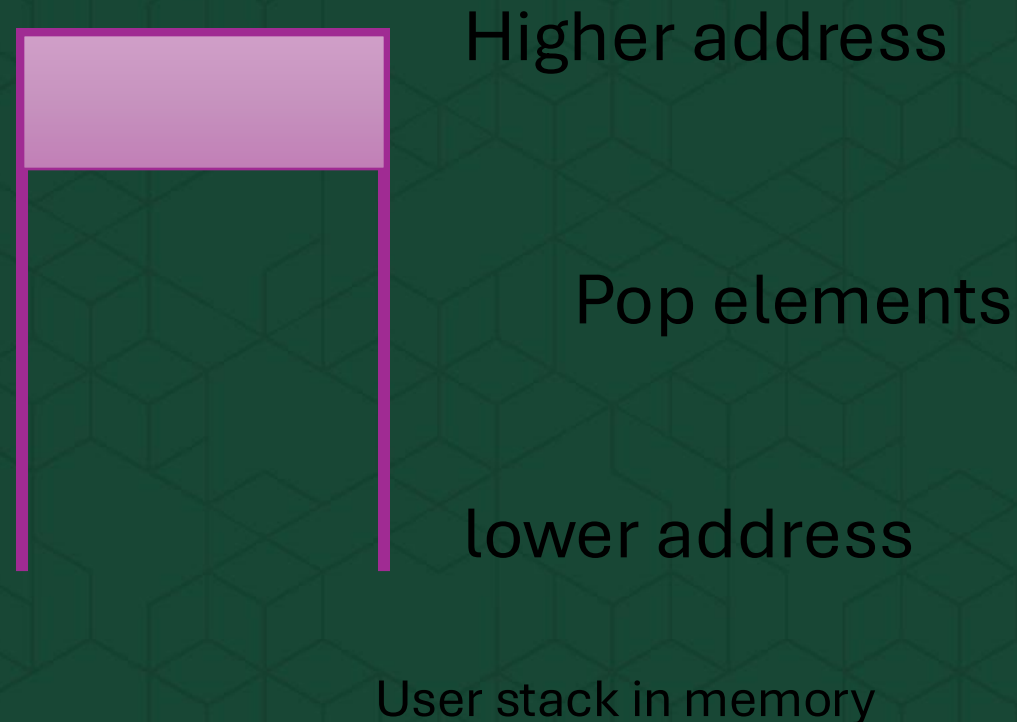
User stack

- User stack in memory grows downwards from higher address to lower address



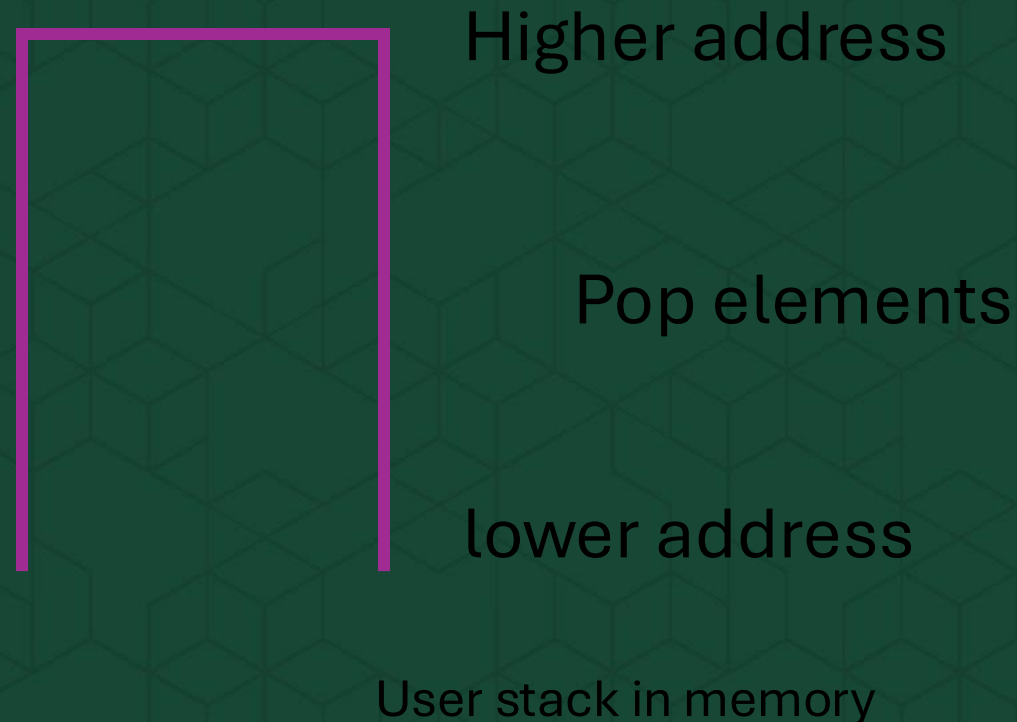
User stack

- User stack in memory grows downwards from higher address to lower address



User stack

- User stack in memory grows downwards from higher address to lower address



User stack

- Why does memory stack adopt the “downwards” style?
- Because of the legacy reason. Early CPUs run assembly instructions starting from the lowest address. For design simplicity, stack that holds content other than the instructions, e.g., return address and local variables, starts from the highest address.

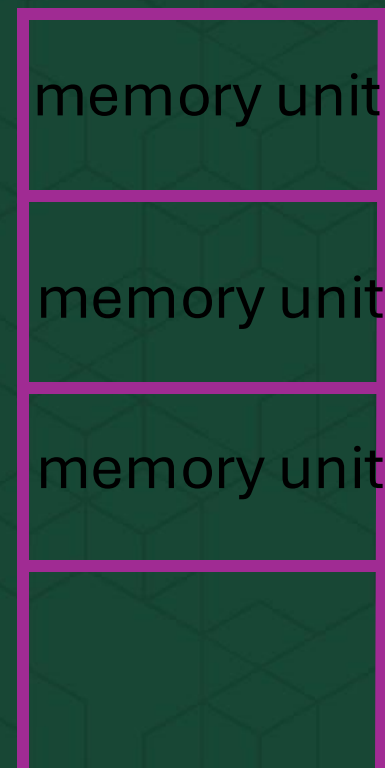


MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Memory unit and the content



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —



Higher address

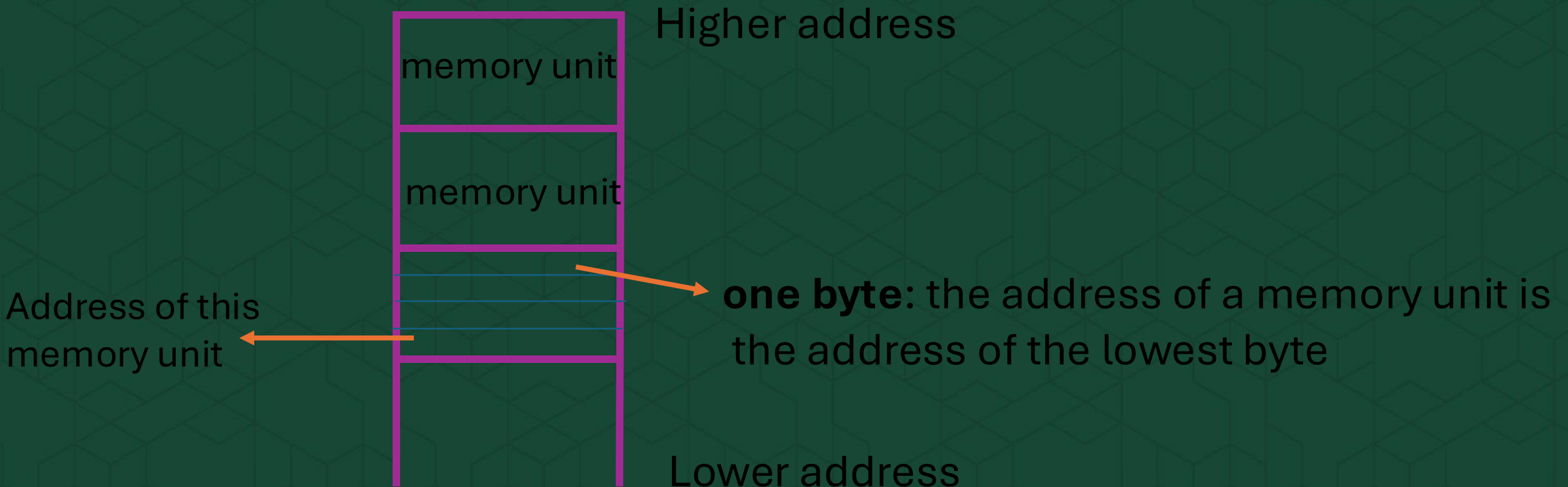
For 32-bit processor, a memory unit
consists of 4 bytes.

Lower address

Memory unit and the content



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —



Functions/Procedures

- Functions/procedures are not native to assembly
- Compilers implement functions
 - On the stack
 - Following the call/return stack discipline



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Implementation Issues of functions

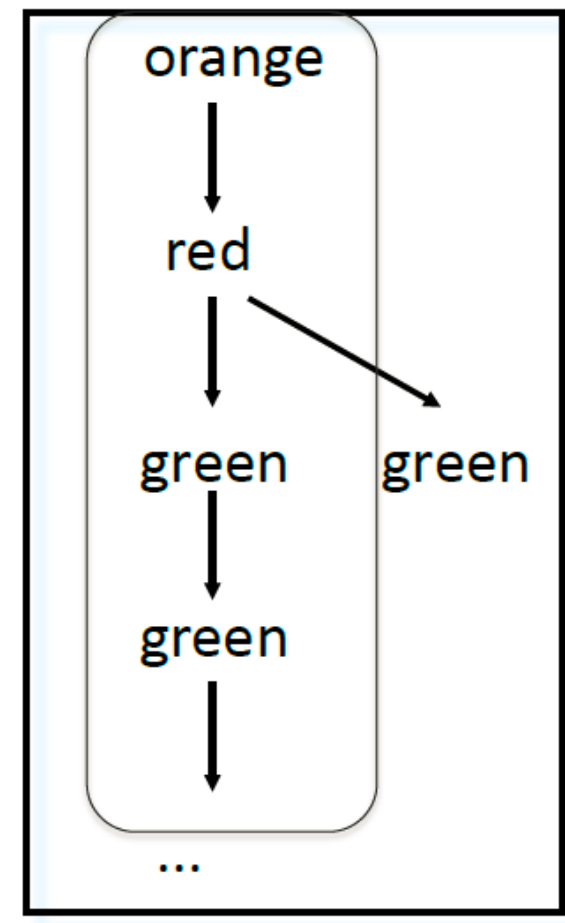
- We need to address several issues:
 - How to allocate space for local variables
 - How to pass parameters
 - How to pass return values
 - How to share 8 registers with local variables
- A stack frame provides space for these values
 - Each procedure invocation has its own stack frame
- Stack discipline is LIFO
 - If procedure A calls B, B's frame must exit before A's

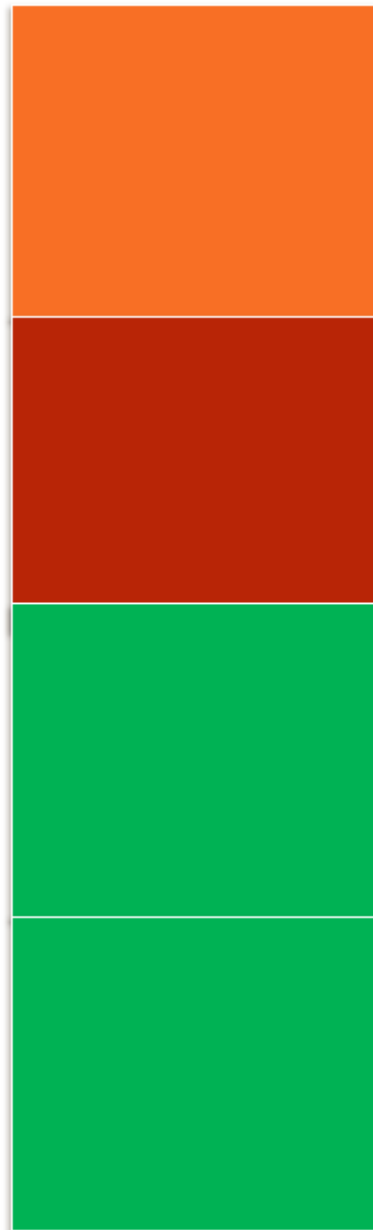
Function Call Chain

```
orange(...)
{
  ...
  red()
  ...
}
```

```
red(...)
{
  ...
  green()
  ...
  green()
}
```

```
green(...)
{
  ...
  green()
  ...
}
```





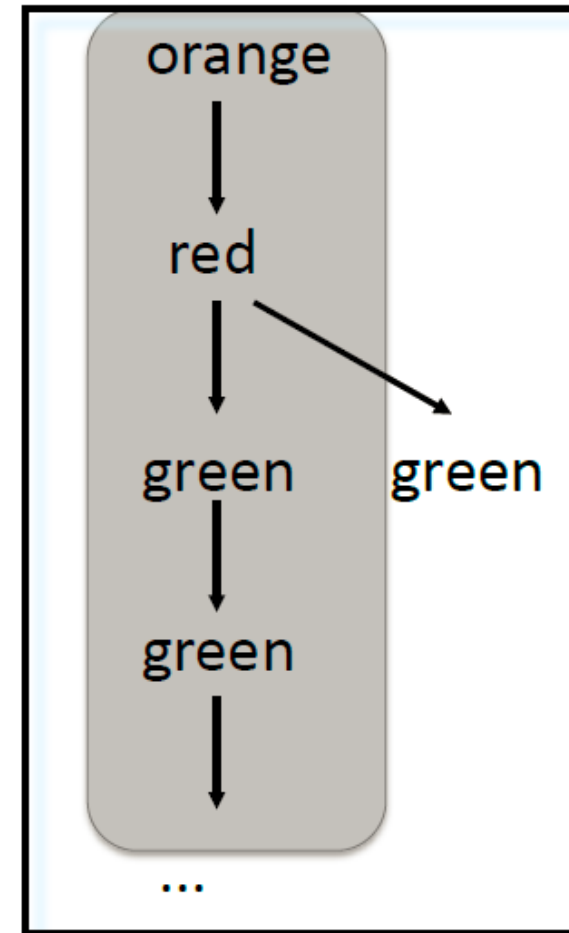
Frame for

- locals
- pushing parameters
- temporary space

Call to red
"pushes"
new frame

When green
returns it
"pops"
its frame

Function Call Chain



On the Stack

```
int orange(int a, int b) {  
    char buf[16];  
    int c, d;  
    if (a > b)  
        c = a;  
    else  
        c = b;  
    d = red(c, buf);  
    return d;  
}
```

arguments

local
variables

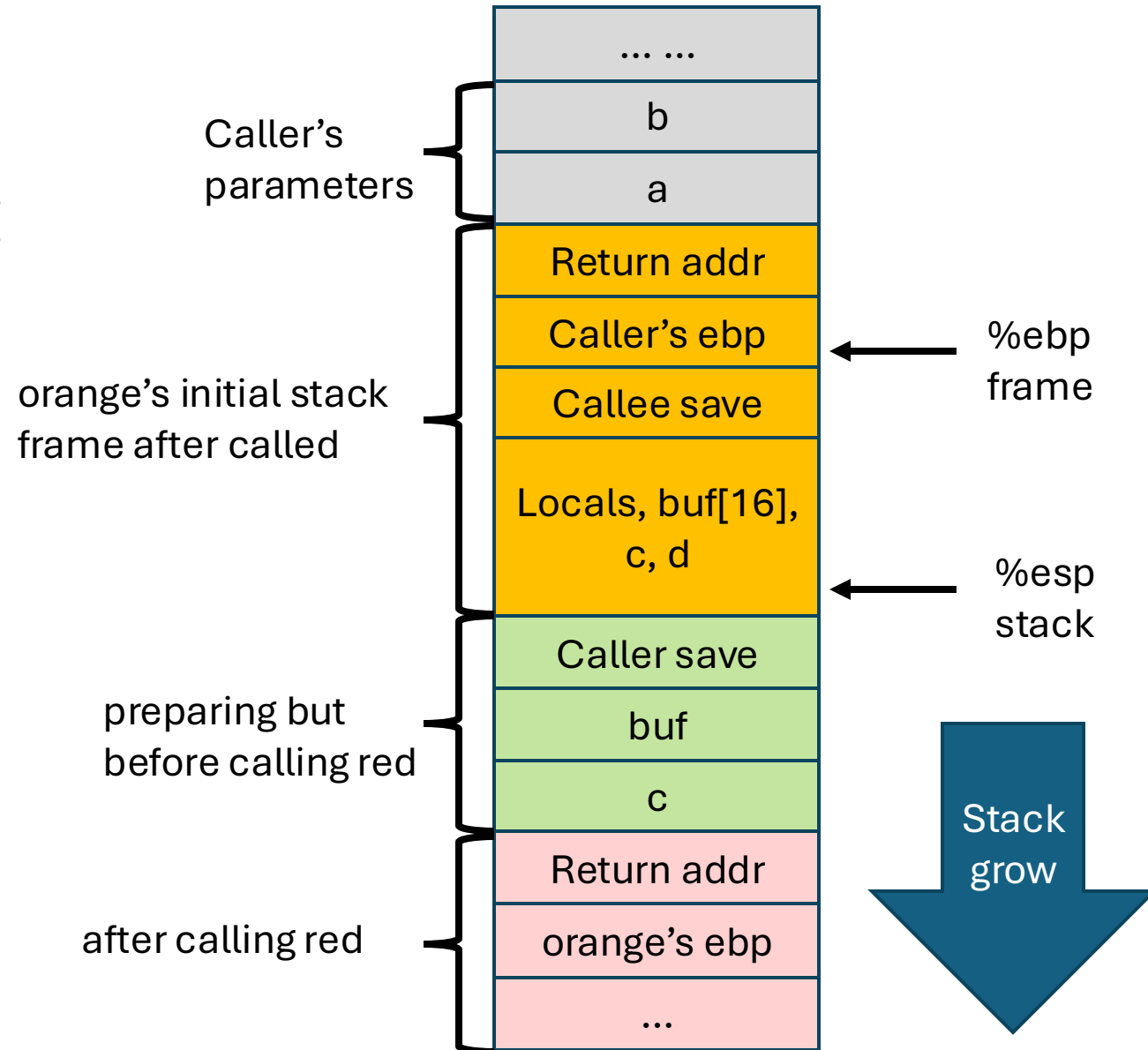
Arguments
for callee

return value

**Calling convention
determines the
above features!**

Cdecl – the default for Linux & GCC

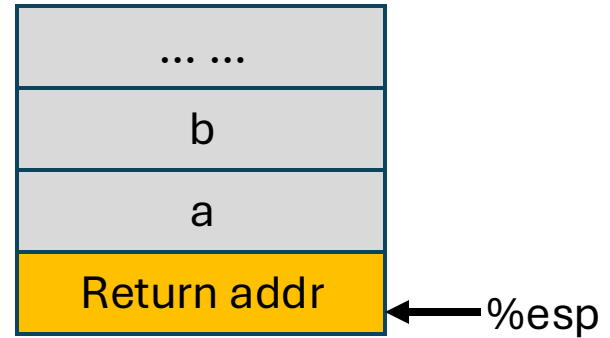
```
int orange(int a, int b) {  
    char buf[16];  
    int c, d;  
    if (a > b)  
        c = a;  
    else  
        c = b;  
    d = red(c, buf);  
    return d;  
}
```



Details: Orange is Called

```
int orange(int a, int b){  
    char buf[16];  
    int c, d;  
    if (a > b)  
        c = a;  
    else  
        c = b;  
    d = red(c, buf);  
    return d;  
}
```

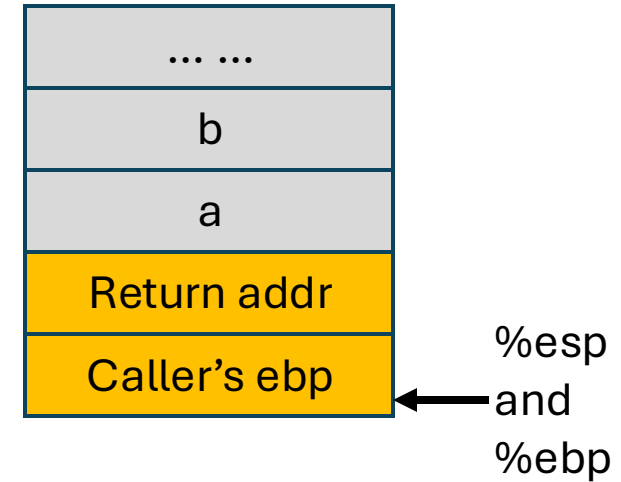
1. return address has already been pushed onto stack by caller



Details: Orange is Called

```
int orange(int a, int b){  
    char buf[16];  
    int c, d;  
    if (a > b)  
        c = a;  
    else  
        c = b;  
    d = red(c, buf);  
    return d;  
}
```

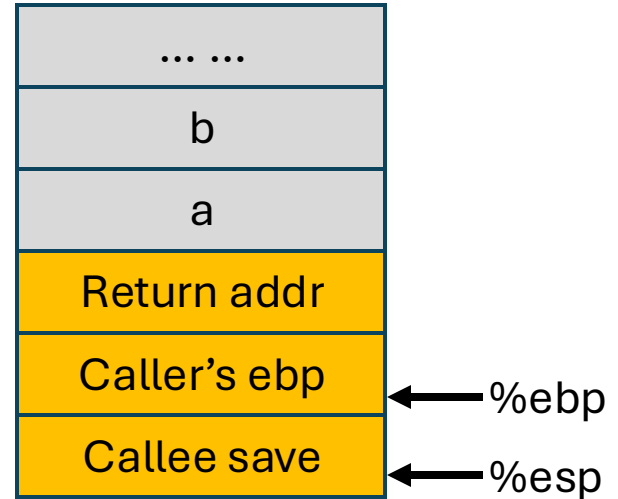
1. return address has already been pushed onto stack by caller
2. Own the frame pointer
 1. Push caller's ebp
 2. Copy current esp to ebp
 3. First augment is ebp+8



Details: Orange is Called

```
int orange(int a, int b){  
    char buf[16];  
    int c, d;  
    if (a > b)  
        c = a;  
    else  
        c = b;  
    d = red(c, buf);  
    return d;  
}
```

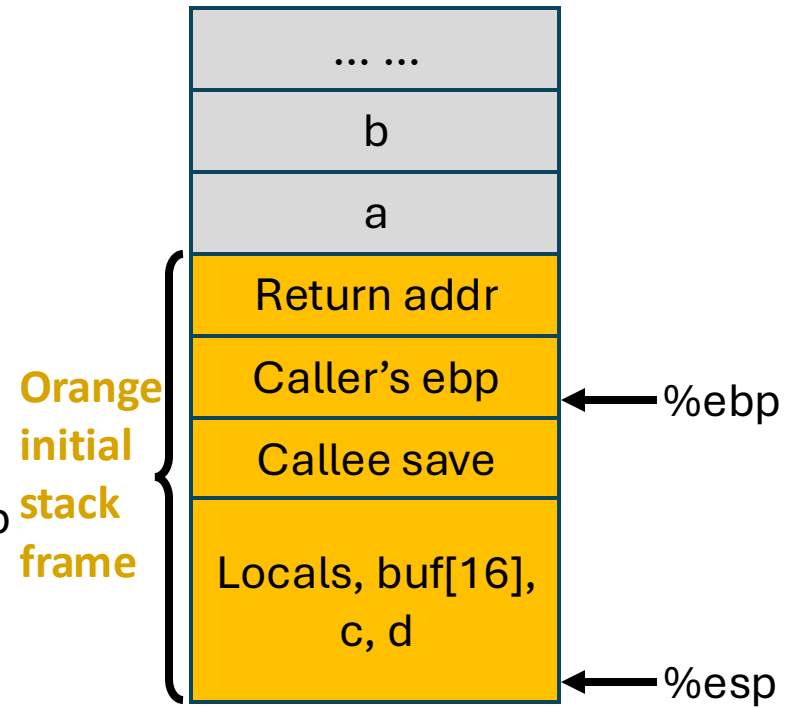
1. return address has already been pushed onto stack by caller
2. Own the frame pointer
 1. Push caller's ebp
 2. Copy current esp to ebp
 3. First augment is ebp+8
3. save values of other callee-save registers if used



Details: Orange is Called

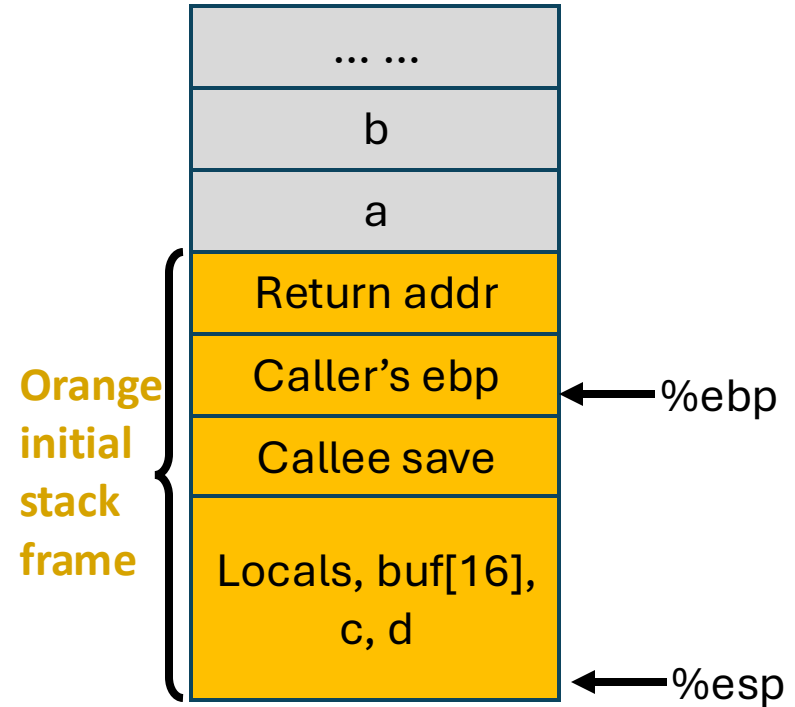
```
int orange(int a, int b){  
    char buf[16];  
    int c, d;  
    if (a > b)  
        c = a;  
    else  
        c = b;  
    d = red(c, buf);  
    return d;  
}
```

1. return address has already been pushed onto stack by caller
2. Own the frame pointer
 1. Push caller's ebp
 2. Copy current esp to ebp
 3. First augment is ebp+8
3. save values of other callee-save registers if used
4. allocate space for locals



Details: Orange Calls Red

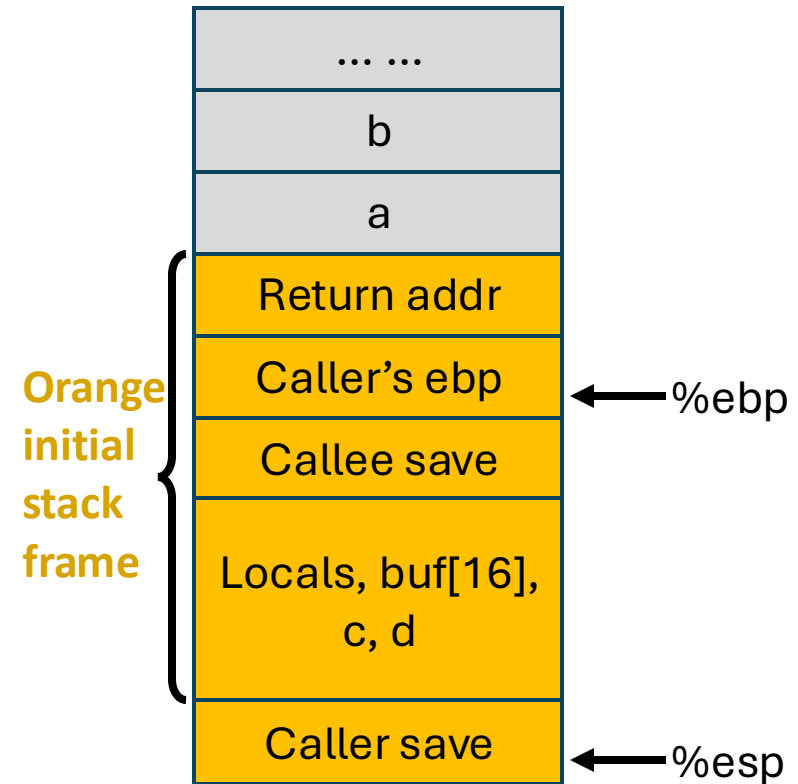
```
int orange(int a, int b){  
    char buf[16];  
    int c, d;  
    if (a > b)  
        c = a;  
    else  
        c = b;  
    d = red(c, buf);  
    return d;  
}
```



Details: Orange Calls Red

```
int orange(int a, int b){  
    char buf[16];  
    int c, d;  
    if (a > b)  
        c = a;  
    else  
        c = b;  
    d = red(c, buf);  
    return d;  
}
```

1. push any caller-save registers if their values are needed after red returns

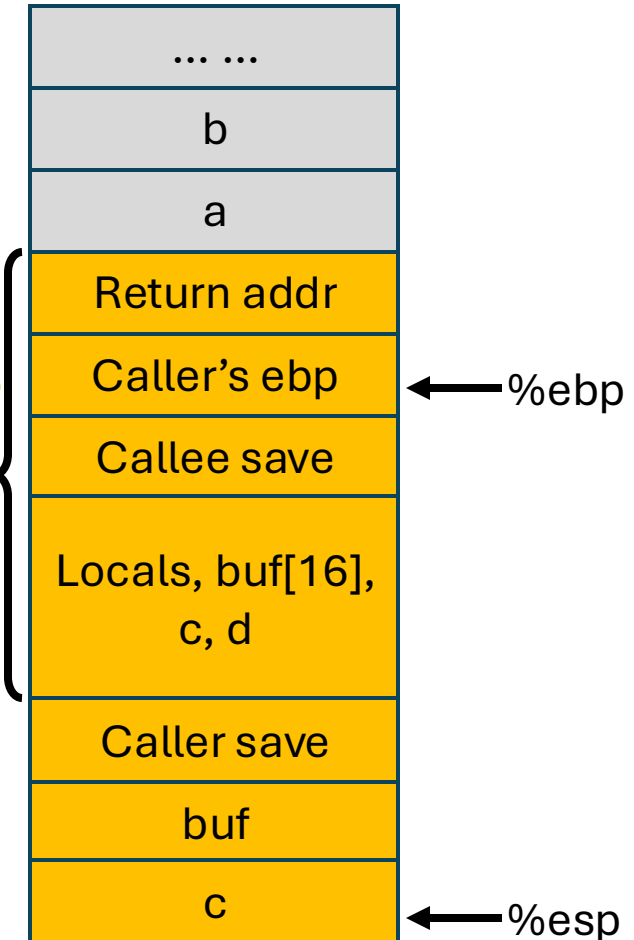


Details: Orange Calls Red

```
int orange(int a, int b){  
    char buf[16];  
    int c, d;  
    if (a > b)  
        c = a;  
    else  
        c = b;  
    d = red(c, buf);  
    return d;  
}
```

1. push any caller-save registers if their values are needed after red returns
2. push arguments to red from right to left (reversed)
 - from callee's perspective, argument 1 is nearest in stack

Orange
initial
stack
frame

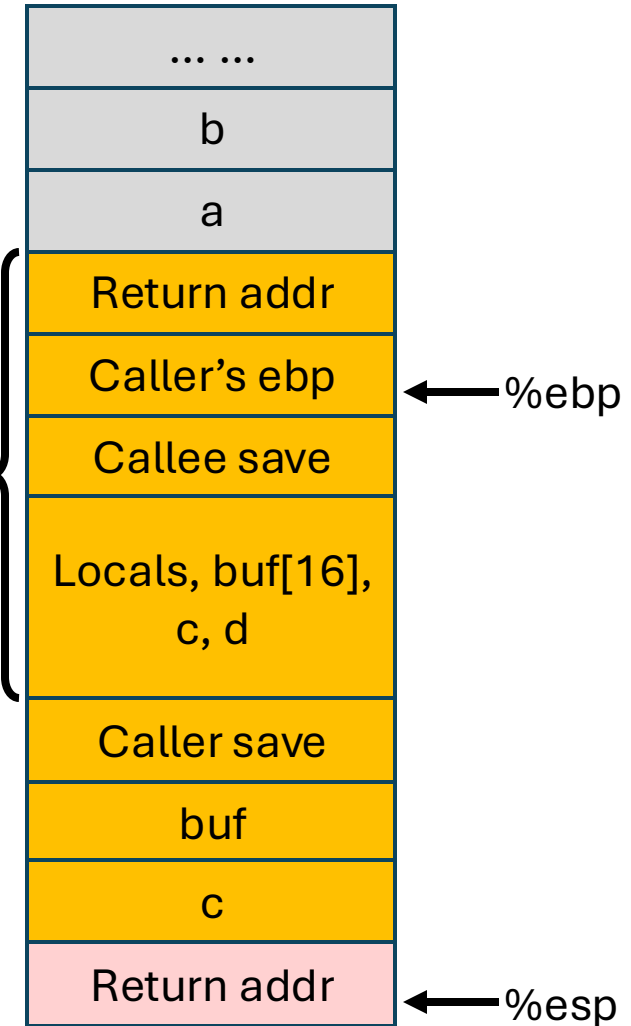


Details: Orange Calls Red

```
int orange(int a, int b){  
    char buf[16];  
    int c, d;  
    if (a > b)  
        c = a;  
    else  
        c = b;  
    d = red(c, buf);  
    return d;  
}
```

1. push any caller-save registers if their values are needed after red returns
2. push arguments to red from right to left (reversed)
 - from callee's perspective, argument 1 is nearest in stack
3. push return address, i.e., the next instruction to execute in orange after red returns

Orange
initial
stack
frame



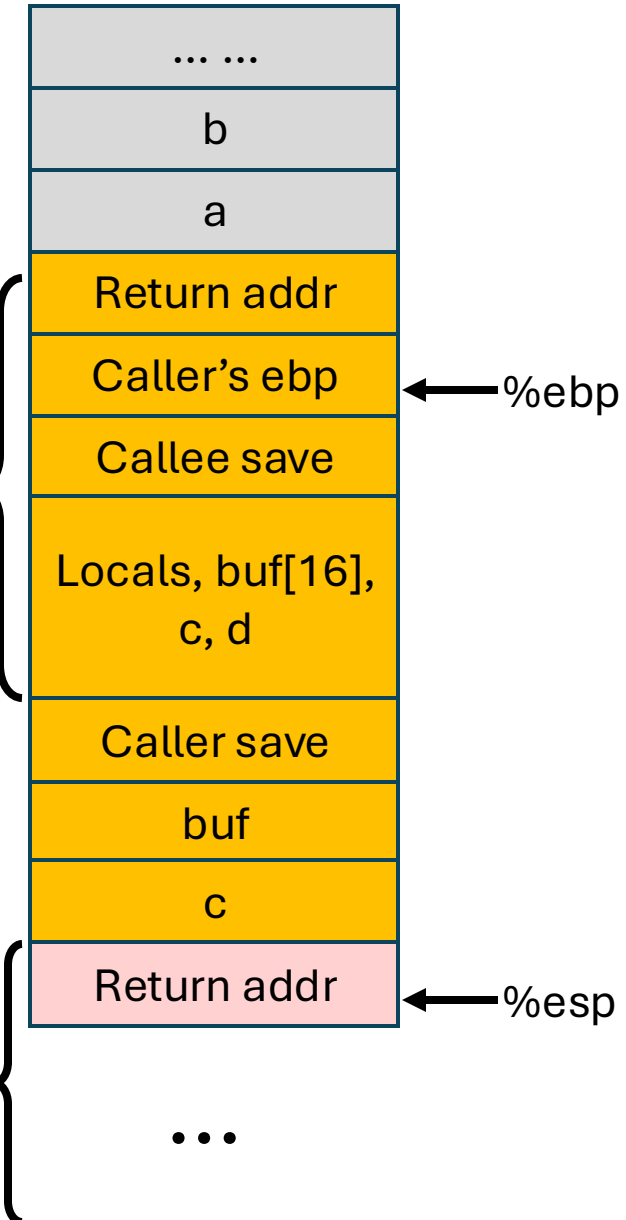
Details: Orange Calls Red

```
int orange(int a, int b){
    char buf[16];
    int c, d;
    if (a > b)
        c = a;
    else
        c = b;
    d = red(c, buf);
    return d;
}
```

1. push any caller-save registers if their values are needed after red returns
2. push arguments to red from right to left (reversed)
 - from callee's perspective, argument 1 is nearest in stack
3. push return address, i.e., the next instruction to execute in orange after red returns
4. Transfer control to red one (usually happens together with step 3 using call)

Orange
initial
stack
frame

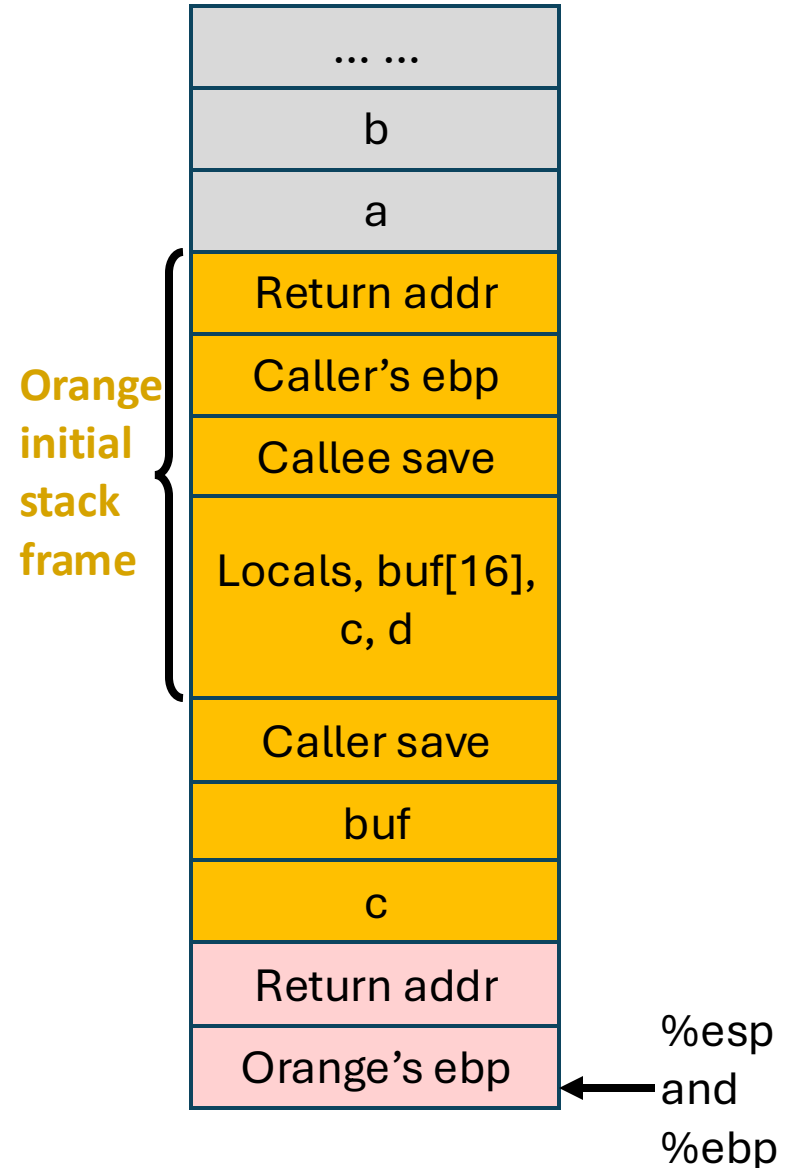
red
stack
frame



Details: Red Attains Control and Returns

When red attains control,

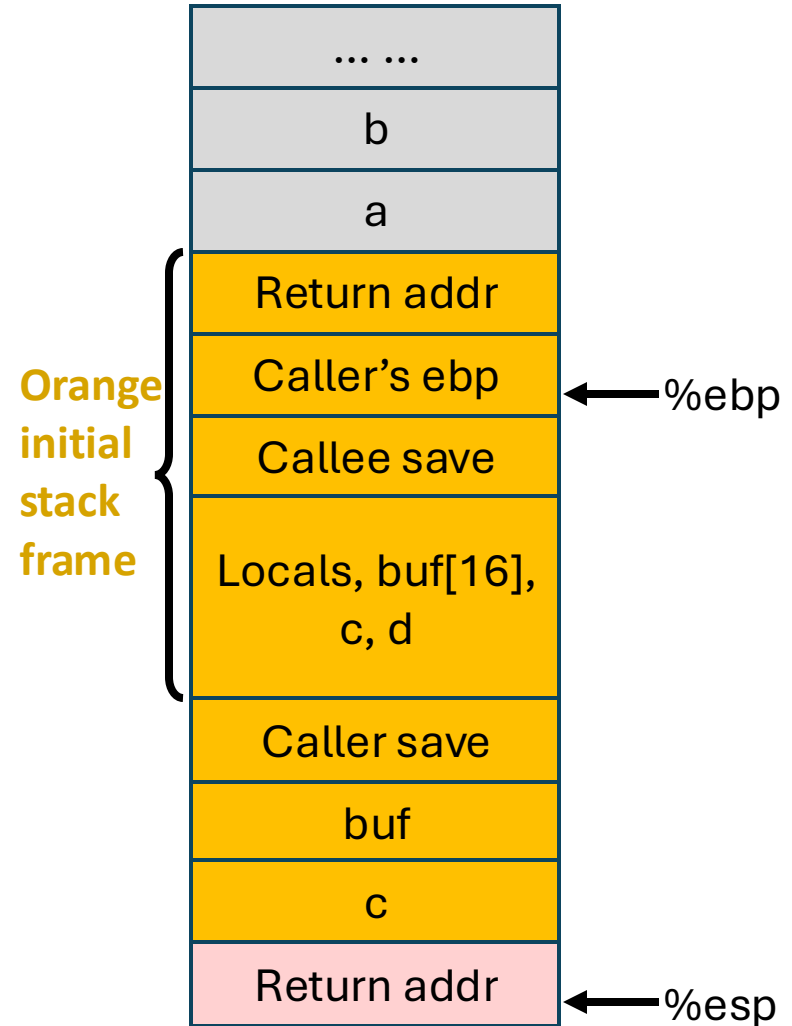
1. return address has already been pushed onto stack by orange
2. own the frame pointer
3. ... (red is doing its stuff) ...
4. store return value, if any, in eax
5. deallocate locals
 - adding to esp
6. restore any callee-save registers



Details: Red Returns

When red attains control,

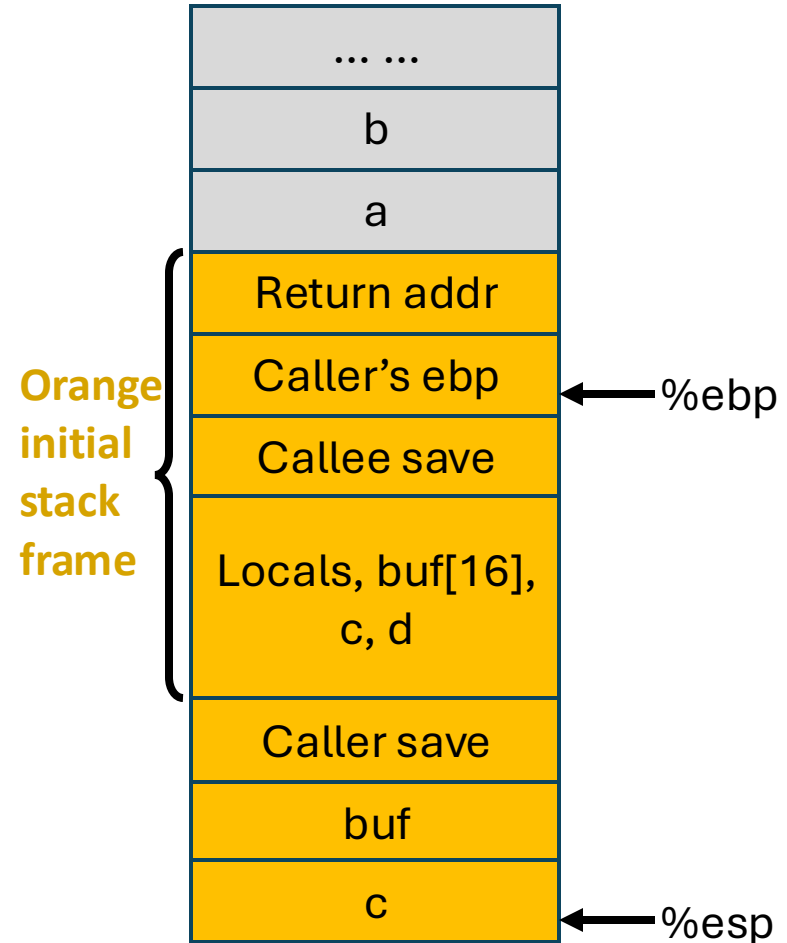
1. return address has already been pushed onto stack by orange
2. own the frame pointer
3. ... (red is doing its stuff) ...
4. store return value, if any, in eax
5. deallocate locals
 - adding to esp
6. restore any callee-save registers
7. restore orange's frame pointer
 - pop %ebp



Details: Red Returns

When red attains control,

1. return address has already been pushed onto stack by orange
2. own the frame pointer
3. ... (red is doing its stuff) ...
4. store return value, if any, in `eax`
5. deallocate locals
 - adding to `esp`
6. restore any callee-save registers
7. restore orange's frame pointer
 - `pop %ebp`
8. return control to orange
 - `ret`
 - pops return address from stack and jumps there

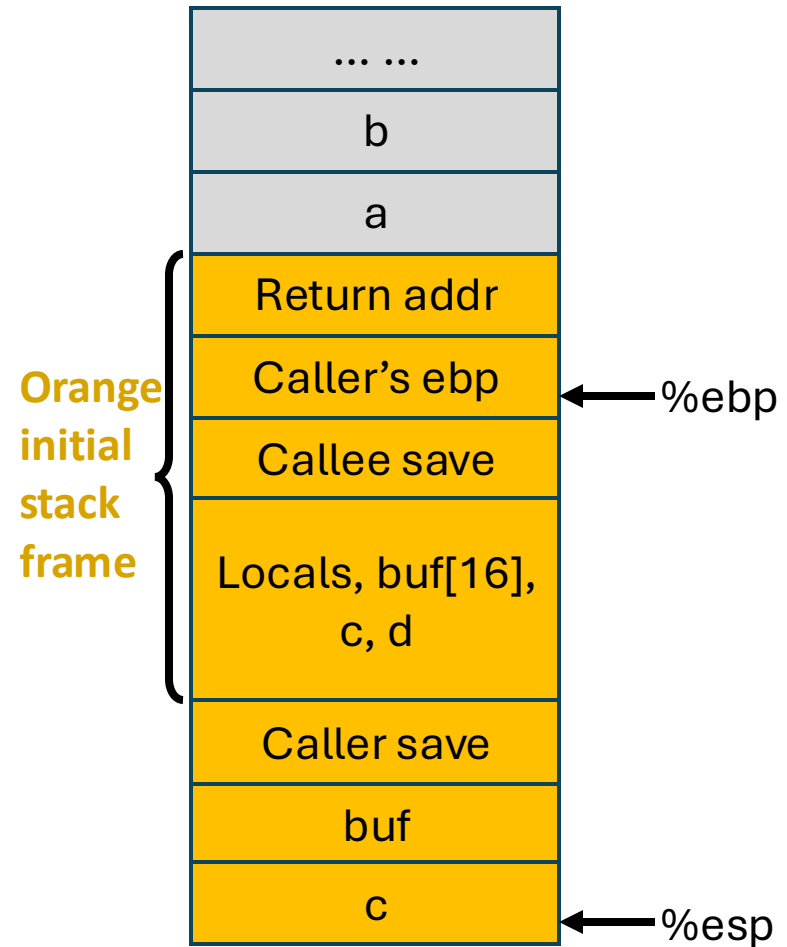


Details: Orange Attains Control Again

When orange attains control,

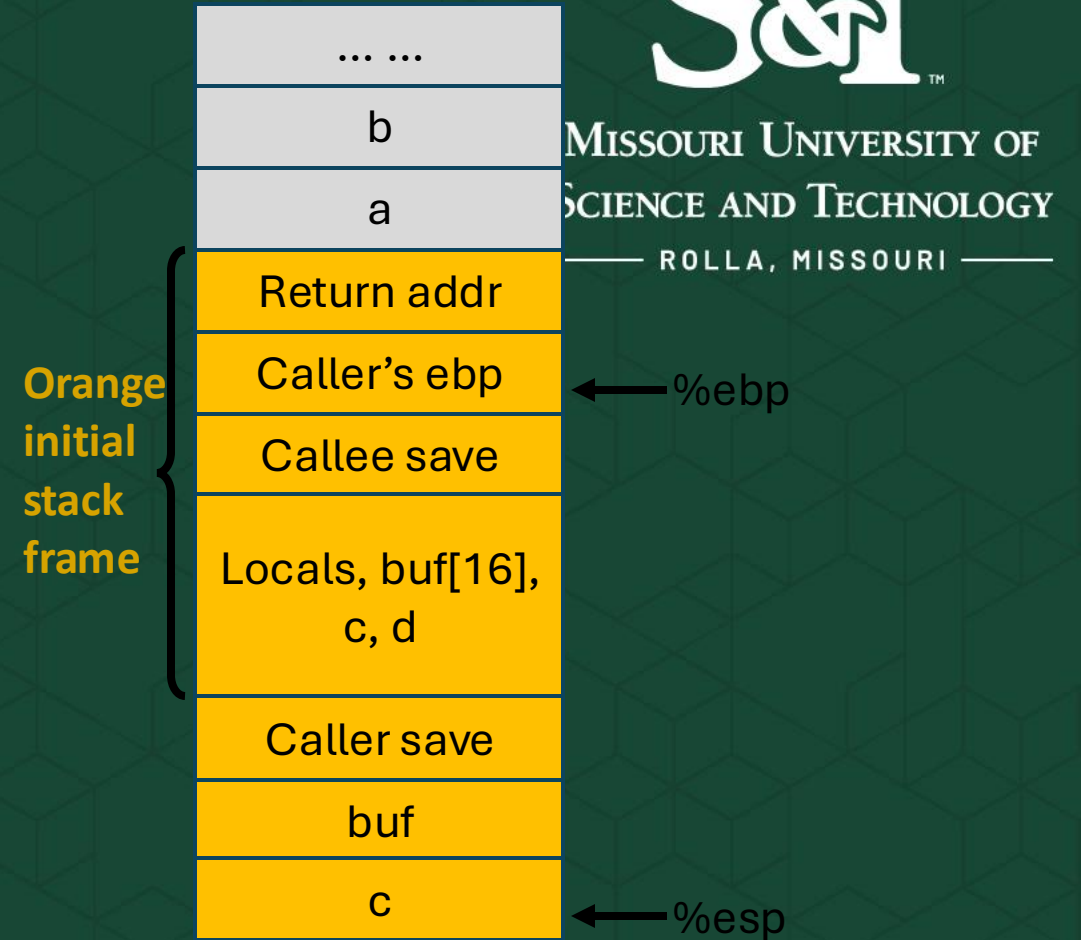
1. clean up arguments to red
 - adding to esp
2. restore any caller-save registers
 - Pop
3. ...

```
int orange(int a, int b) {  
    char buf[16];  
    int c, d;  
    if (a > b)  
        c = a;  
    else  
        c = b;  
    d = red(c, buf);  
    return d;  
}
```



Questions?

- What's the difference between “caller's ebp” and “%ebp”?



Questions?

- Does orange need to pass the number of arguments to red?
- Do we always need to save callee-save and caller-save registers?



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

cdecl – One Slide



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
MISSOURI

Action	Notes
caller saves: eax, edx, ecx	push (old), or mov if esp already adjusted
arguments pushed right-to-left	
linkage data starts new frame	call pushes return addr
callee saves: ebx, esi, edi, ebp, esp	ebp often used to deref args and local vars
return value	pass back using eax
argument cleanup	caller's responsibility

Terminology



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- ***Function Prologue***
 - instructions to set up stack space and save callee saved registers
- ***Function Epilogue***
 - instructions to clean up stack space and restore callee saved registers

Key Concepts

- **Register to register moves**
- **Register/memory**
 - mov and addressing modes for common codes
- **Control flow**
 - EFLAGS
- **Program memory organization**
 - Stack, heap, shared library
- **Functions**
 - Pass arguments, callee and caller saved, stack frame



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

More Info



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- **Overall machine model: Computer Systems, a Programmer's Perspective**
 - by Bryant and O'Hallaron
- **Calling conventions:**
 - http://en.wikipedia.org/wiki/X86_calling_conventions
 -

Real Code Analysis (32-bit)



UNIVERSITY OF
ND TECHNOLOGY
A, MISSOURI

```
int myfun(int a, int b){  
    int c;  
    int d;  
    c = a + b;  
    d = a - b;  
    return c + d;  
}  
int main() {  
    int result = myfun(1,2);  
    printf("the result is %d\n", result);  
}
```

```
08048445 <main>:  
8048445:  push  %ebp  
8048446:  mov   %esp,%ebp  
8048448:  and   $0xffffffff0,%esp  
804844b:  sub   $0x20,%esp  
804844e:  movl  $0x2,0x4(%esp)  
8048456:  movl  $0x1,(%esp)  
804845d:  call  804841d <myfun>  
... ..  
... ..
```

```
0804841d <myfun>:  
804841d:  push  %ebp  
804841e:  mov   %esp,%ebp  
8048420:  sub   $0x10,%esp  
8048423:  mov   0xc(%ebp),%eax  
8048426:  mov   0x8(%ebp),%edx  
8048429:  add   %edx,%eax  
804842b:  mov   %eax,-0x4(%ebp)  
804842e:  mov   0xc(%ebp),%eax  
8048431:  mov   0x8(%ebp),%edx  
8048434:  sub   %eax,%edx  
8048436:  mov   %edx,%eax  
8048438:  mov   %eax,-0x8(%ebp)  
804843b:  mov   -0x8(%ebp),%eax  
804843e:  mov   -0x4(%ebp),%edx  
8048441:  add   %edx,%eax  
8048443:  leave  
8048444:  ret
```

When main gains control

```
int main() {  
    int result = myfun(1,2);  
    printf("the result is %d\n", result);  
}
```

```
08048445 <main>:  
8048445:    push    %ebp  
8048446:    mov     %esp,%ebp  
8048448:    and     $0xffffffff0,%esp  
804844b:    sub     $0x20,%esp  
804844e:    movl    $0x2,0x4(%esp)  
8048456:    movl    $0x1,(%esp)  
804845d:    call    804841d <myfun>  
... ..
```

Main's
frame

.....
Return addr



When main gains control

```
int main() {  
    int result = myfun(1,2);  
    printf("the result is %d\n", result);  
}
```

```
08048445 <main>:  
8048445:    push    %ebp  
8048446:    mov     %esp,%ebp  
8048448:    and     $0xffffffff0,%esp  
804844b:    sub     $0x20,%esp  
804844e:    movl    $0x2,0x4(%esp)  
8048456:    movl    $0x1,(%esp)  
804845d:    call    804841d <myfun>  
... ..
```

Main's
frame

Return addr

Caller's ebp



When main gains control

```
int main() {  
    int result = myfun(1,2);  
    printf("the result is %d\n", result);  
}
```

```
08048445 <main>:  
8048445:    push    %ebp  
8048446:    mov     %esp,%ebp  
8048448:    and     $0xffffffff0,%esp  
804844b:    sub     $0x20,%esp  
804844e:    movl    $0x2,0x4(%esp)  
8048456:    movl    $0x1,(%esp)  
804845d:    call    804841d <myfun>  
... ..
```

Main's
frame

.....
Return addr

Caller's ebp



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
ROLLA, MISSOURI

This code makes sure that the stack is aligned to 16 bytes for efficiency purpose. GCC does this for main.

The new esp will be less than or equal to the old esp.

Let's assume esp doesn't change

When main gains control

```
int main() {  
    int result = myfun(1,2);  
    printf("the result is %d\n", result);  
}
```

```
08048445 <main>:  
8048445:    push    %ebp  
8048446:    mov     %esp,%ebp  
8048448:    and     $0xffffffff0,%esp  
804844b:    sub     $0x20,%esp  
804844e:    movl    $0x2,0x4(%esp)  
8048456:    movl    $0x1,(%esp)  
804845d:    call    804841d <myfun>  
... ..
```

Main's
frame

.....
Return addr

Caller's ebp



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
← $\frac{\%esp}{\text{and}} \frac{\%ebp}{\%ebp}$
ROLLA, MISSOURI

When main gains control

```
int main() {  
    int result = myfun(1,2);  
    printf("the result is %d\n", result);  
}
```

```
08048445 <main>:  
8048445:    push    %ebp  
8048446:    mov     %esp,%ebp  
8048448:    and     $0xffffffff0,%esp  
804844b:    sub     $0x20,%esp  
804844e:    movl    $0x2,0x4(%esp)  
8048456:    movl    $0x1,(%esp)  
804845d:    call    804841d <myfun>  
... ..
```

Main's
frame

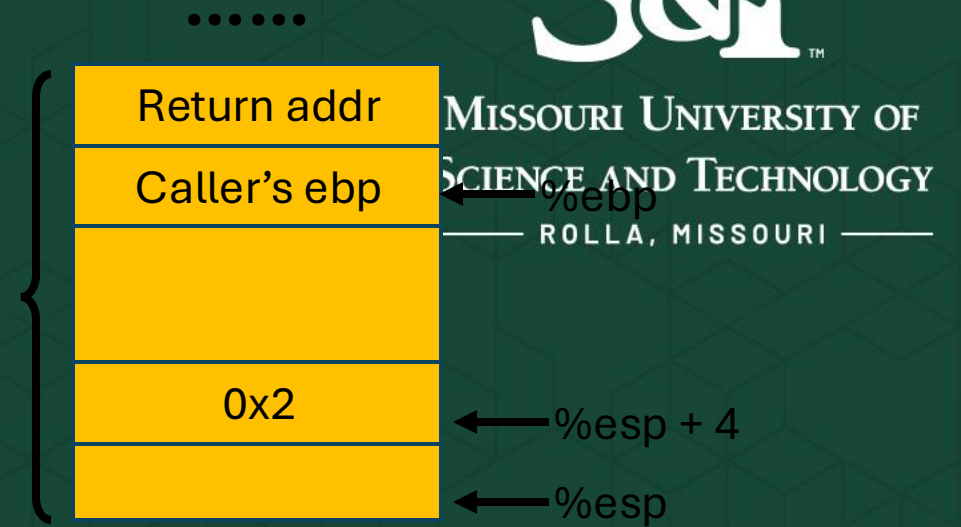


When main gains control

```
int main() {  
    int result = myfun(1,2);  
    printf("the result is %d\n", result);  
}
```

```
08048445 <main>:  
8048445:    push    %ebp  
8048446:    mov     %esp,%ebp  
8048448:    and     $0xffffffff0,%esp  
804844b:    sub     $0x20,%esp  
804844e:    movl    $0x2,0x4(%esp)  
8048456:    movl    $0x1,(%esp)  
804845d:    call    804841d <myfun>  
... ..
```

Main's
frame



When main gains control



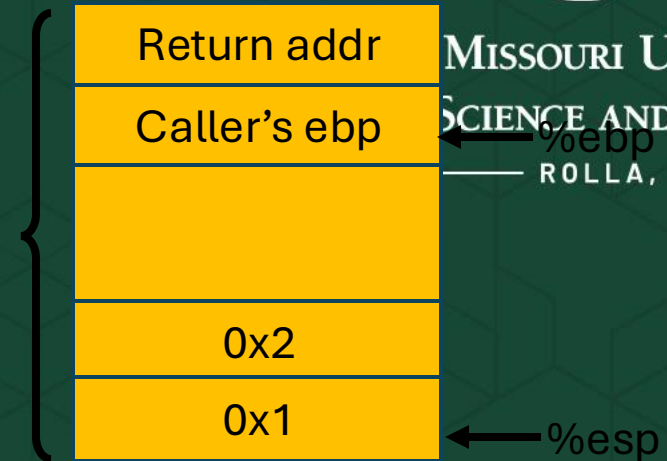
MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
ROLLA, MISSOURI

```
int main() {  
    int result = myfun(1,2);  
    printf("the result is %d\n", result);  
}
```

```
08048445 <main>:  
8048445:    push    %ebp  
8048446:    mov     %esp,%ebp  
8048448:    and     $0xffffffff0,%esp  
804844b:    sub     $0x20,%esp  
804844e:    movl    $0x2,0x4(%esp)  
8048456:    movl    $0x1,(%esp)  
804845d:    call    804841d <myfun>  
... ..
```

Pay attention to this address

Main's
frame



Call:

- 1) Push the current address into stack
- 2) Jump to <myfun> (transfer control)

Now myfun gains control

```
int myfun(int a, int b) {  
    int c;  
    int d;  
    c = a + b;  
    d = a - b;  
    return c + d; }
```

0804841d <myfun>:

```
804841d:  push  %ebp  
804841e:  mov   %esp,%ebp  
8048420:  sub   $0x10,%esp  
8048423:  mov   0xc(%ebp),%eax  
8048426:  mov   0x8(%ebp),%edx  
8048429:  add   %edx,%eax  
804842b:  mov   %eax,-0x4(%ebp)  
804842e:  mov   0xc(%ebp),%eax  
8048431:  mov   0x8(%ebp),%edx  
8048434:  sub   %eax,%edx  
8048436:  mov   %edx,%eax  
8048438:  mov   %eax,-0x8(%ebp)  
804843b:  mov   -0x8(%ebp),%eax  
804843e:  mov   -0x4(%ebp),%edx  
8048441:  add   %edx,%eax  
8048443:  leave  
8048444:  ret
```

Main's
frame

myfun's
frame

.....

Return addr

Caller's ebp

b: 0x2

a: 0x1

main's ret addr



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
ROLLA, MISSOURI

Now myfun gains control

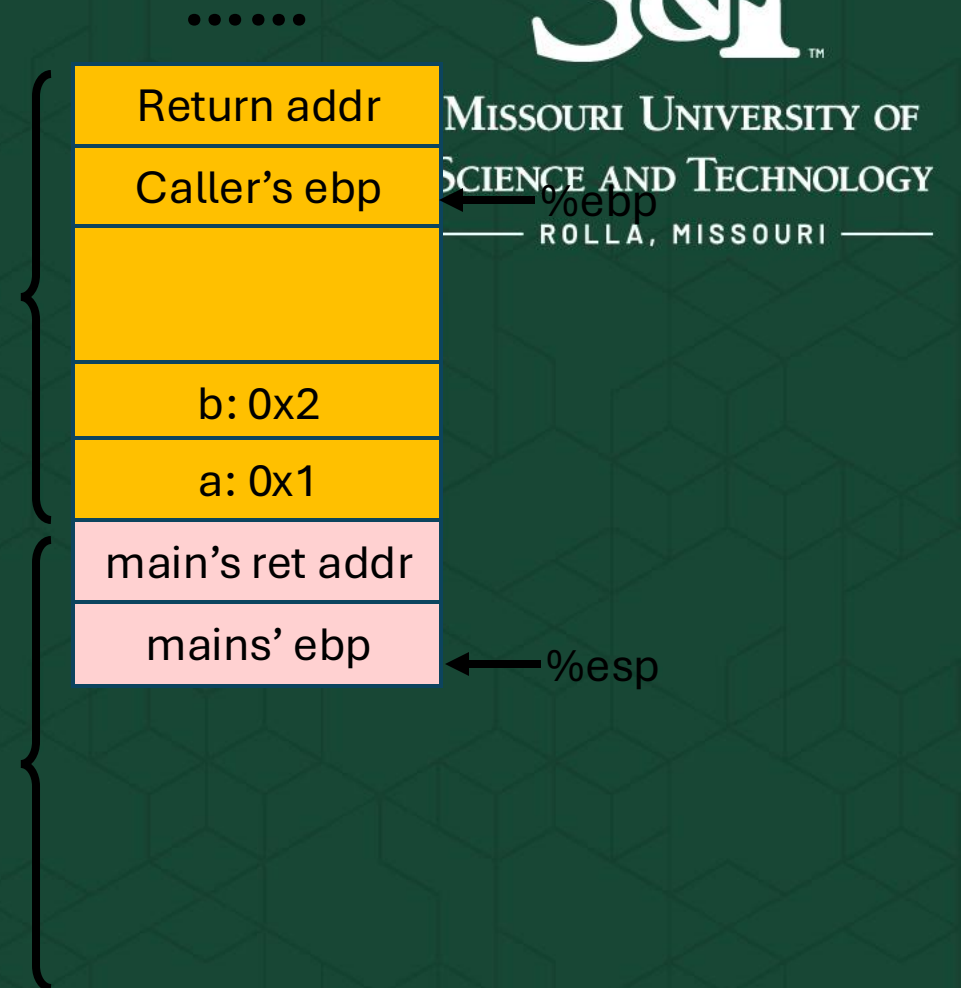
```
int myfun(int a, int b) {  
    int c;  
    int d;  
    c = a + b;  
    d = a - b;  
    return c + d; }
```

0804841d <myfun>:

```
804841d:  push  %ebp  
804841e:  mov   %esp,%ebp  
8048420:  sub   $0x10,%esp  
8048423:  mov   0xc(%ebp),%eax  
8048426:  mov   0x8(%ebp),%edx  
8048429:  add   %edx,%eax  
804842b:  mov   %eax,-0x4(%ebp)  
804842e:  mov   0xc(%ebp),%eax  
8048431:  mov   0x8(%ebp),%edx  
8048434:  sub   %eax,%edx  
8048436:  mov   %edx,%eax  
8048438:  mov   %eax,-0x8(%ebp)  
804843b:  mov   -0x8(%ebp),%eax  
804843e:  mov   -0x4(%ebp),%edx  
8048441:  add   %edx,%eax  
8048443:  leave  
8048444:  ret
```

Main's
frame

myfun's
frame



Now myfun gains control

```
int myfun(int a, int b) {  
    int c;  
    int d;  
    c = a + b;  
    d = a - b;  
    return c + d; }
```

0804841d <myfun>:

```
804841d:  push  %ebp  
804841e:  mov   %esp,%ebp  
8048420:  sub   $0x10,%esp  
8048423:  mov   0xc(%ebp),%eax  
8048426:  mov   0x8(%ebp),%edx  
8048429:  add   %edx,%eax  
804842b:  mov   %eax,-0x4(%ebp)  
804842e:  mov   0xc(%ebp),%eax  
8048431:  mov   0x8(%ebp),%edx  
8048434:  sub   %eax,%edx  
8048436:  mov   %edx,%eax  
8048438:  mov   %eax,-0x8(%ebp)  
804843b:  mov   -0x8(%ebp),%eax  
804843e:  mov   -0x4(%ebp),%edx  
8048441:  add   %edx,%eax  
8048443:  leave  
8048444:  ret
```

Main's
frame

myfun's
frame

Return addr

Caller's ebp

b: 0x2

a: 0x1

main's ret addr

mains' ebp

MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

← %ebp + 12

← %ebp + 8

← %esp
and
%ebp

c and d are 4 bytes each, but esp is subtracted by 0x10 (16) bytes. 16 bytes alignment is really important for efficiency.

Now myfun gains control

```
int myfun(int a, int b) {  
    int c;  
    int d;  
    c = a + b;  
    d = a - b;  
    return c + d; }
```

0804841d <myfun>:

```
804841d:  push  %ebp  
804841e:  mov   %esp,%ebp  
8048420:  sub   $0x10,%esp  
8048423:  mov   0xc(%ebp),%eax  
8048426:  mov   0x8(%ebp),%edx  
8048429:  add   %edx,%eax  
804842b:  mov   %eax,-0x4(%ebp)  
804842e:  mov   0xc(%ebp),%eax  
8048431:  mov   0x8(%ebp),%edx  
8048434:  sub   %eax,%edx  
8048436:  mov   %edx,%eax  
8048438:  mov   %eax,-0x8(%ebp)  
804843b:  mov   -0x8(%ebp),%eax  
804843e:  mov   -0x4(%ebp),%edx  
8048441:  add   %edx,%eax  
8048443:  leave  
8048444:  ret
```

Main's
frame

myfun's
frame

Return addr

Caller's ebp

b: 0x2

a: 0x1

main's ret addr

mains' ebp



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

eax

← %ebp

← %esp

Now myfun gains control

```
int myfun(int a, int b) {  
    int c;  
    int d;  
    c = a + b;  
    d = a - b;  
    return c + d; }
```

0804841d <myfun>:

```
804841d:  push  %ebp  
804841e:  mov   %esp,%ebp  
8048420:  sub   $0x10,%esp  
8048423:  mov   0xc(%ebp),%eax  
8048426:  mov   0x8(%ebp),%edx  
8048429:  add   %edx,%eax  
804842b:  mov   %eax,-0x4(%ebp)  
804842e:  mov   0xc(%ebp),%eax  
8048431:  mov   0x8(%ebp),%edx  
8048434:  sub   %eax,%edx  
8048436:  mov   %edx,%eax  
8048438:  mov   %eax,-0x8(%ebp)  
804843b:  mov   -0x8(%ebp),%eax  
804843e:  mov   -0x4(%ebp),%edx  
8048441:  add   %edx,%eax  
8048443:  leave  
8048444:  ret
```

Main's
frame

myfun's
frame

Return addr

Caller's ebp

b: 0x2

a: 0x1

main's ret addr

mains' ebp



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

eax

edx

← %ebp

← %esp

Now myfun gains control

```
int myfun(int a, int b) {  
    int c;  
    int d;  
    c = a + b;  
    d = a - b;  
    return c + d; }
```

0804841d <myfun>:

```
804841d:  push  %ebp  
804841e:  mov   %esp,%ebp  
8048420:  sub   $0x10,%esp  
8048423:  mov   0xc(%ebp),%eax  
8048426:  mov   0x8(%ebp),%edx  
8048429:  add   %edx,%eax  
804842b:  mov   %eax,-0x4(%ebp)  
804842e:  mov   0xc(%ebp),%eax  
8048431:  mov   0x8(%ebp),%edx  
8048434:  sub   %eax,%edx  
8048436:  mov   %edx,%eax  
8048438:  mov   %eax,-0x8(%ebp)  
804843b:  mov   -0x8(%ebp),%eax  
804843e:  mov   -0x4(%ebp),%edx  
8048441:  add   %edx,%eax  
8048443:  leave  
8048444:  ret
```

Main's
frame

myfun's
frame

Return addr

Caller's ebp

b: 0x2

a: 0x1

main's ret addr

main's ebp



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

eax

edx

← %ebp

← %esp

Now myfun gains control

```
int myfun(int a, int b) {  
    int c;  
    int d;  
    c = a + b;  
    d = a - b;  
    return c + d; }
```

0804841d <myfun>:

```
804841d:  push  %ebp  
804841e:  mov   %esp,%ebp  
8048420:  sub   $0x10,%esp  
8048423:  mov   0xc(%ebp),%eax  
8048426:  mov   0x8(%ebp),%edx  
8048429:  add   %edx,%eax  
804842b:  mov   %eax,-0x4(%ebp)  
804842e:  mov   0xc(%ebp),%eax  
8048431:  mov   0x8(%ebp),%edx  
8048434:  sub   %eax,%edx  
8048436:  mov   %edx,%eax  
8048438:  mov   %eax,-0x8(%ebp)  
804843b:  mov   -0x8(%ebp),%eax  
804843e:  mov   -0x4(%ebp),%edx  
8048441:  add   %edx,%eax  
8048443:  leave  
8048444:  ret
```

Main's
frame

myfun's
frame

Return addr

Caller's ebp

b: 0x2

a: 0x1

main's ret addr

main's ebp

c:0x3



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

← %ebp

← %esp

Now myfun gains control

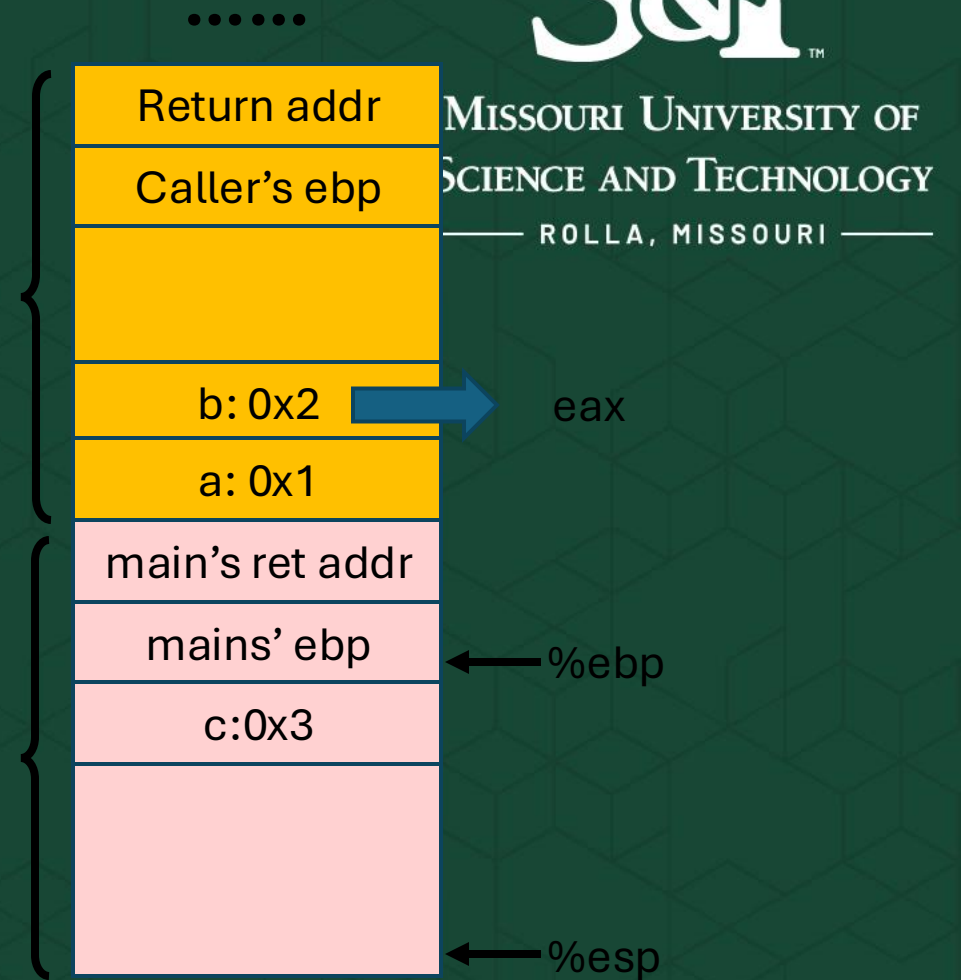
```
int myfun(int a, int b) {  
    int c;  
    int d;  
    c = a + b;  
    d = a - b;  
    return c + d; }
```

0804841d <myfun>:

```
804841d:  push  %ebp  
804841e:  mov   %esp,%ebp  
8048420:  sub   $0x10,%esp  
8048423:  mov   0xc(%ebp),%eax  
8048426:  mov   0x8(%ebp),%edx  
8048429:  add   %edx,%eax  
804842b:  mov   %eax,-0x4(%ebp)  
804842e:  mov   0xc(%ebp),%eax  
8048431:  mov   0x8(%ebp),%edx  
8048434:  sub   %eax,%edx  
8048436:  mov   %edx,%eax  
8048438:  mov   %eax,-0x8(%ebp)  
804843b:  mov   -0x8(%ebp),%eax  
804843e:  mov   -0x4(%ebp),%edx  
8048441:  add   %edx,%eax  
8048443:  leave  
8048444:  ret
```

Main's
frame

myfun's
frame



Now myfun gains control

```
int myfun(int a, int b) {  
    int c;  
    int d;  
    c = a + b;  
    d = a - b;  
    return c + d; }
```

0804841d <myfun>:

```
804841d:  push  %ebp  
804841e:  mov   %esp,%ebp  
8048420:  sub   $0x10,%esp  
8048423:  mov   0xc(%ebp),%eax  
8048426:  mov   0x8(%ebp),%edx  
8048429:  add   %edx,%eax  
804842b:  mov   %eax,-0x4(%ebp)  
804842e:  mov   0xc(%ebp),%eax  
8048431:  mov   0x8(%ebp),%edx  
8048434:  sub   %eax,%edx  
8048436:  mov   %edx,%eax  
8048438:  mov   %eax,-0x8(%ebp)  
804843b:  mov   -0x8(%ebp),%eax  
804843e:  mov   -0x4(%ebp),%edx  
8048441:  add   %edx,%eax  
8048443:  leave  
8048444:  ret
```

Main's
frame

myfun's
frame

Return addr

Caller's ebp

b: 0x2

a: 0x1

main's ret addr

mains' ebp

c:0x3



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

eax

edx

← %ebp

← %esp

Now myfun gains control

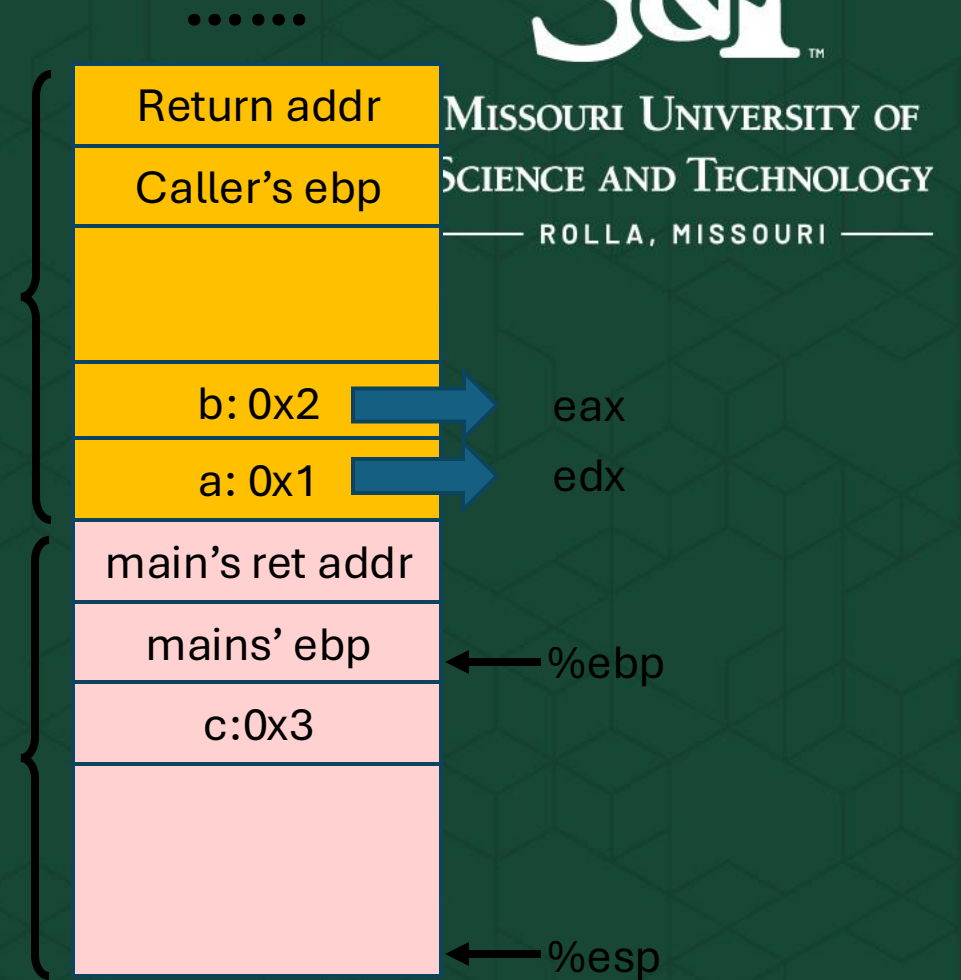
```
int myfun(int a, int b) {  
    int c;  
    int d;  
    c = a + b;  
    d = a - b;  
    return c + d; }
```

0804841d <myfun>:

```
804841d:  push  %ebp  
804841e:  mov   %esp,%ebp  
8048420:  sub   $0x10,%esp  
8048423:  mov   0xc(%ebp),%eax  
8048426:  mov   0x8(%ebp),%edx  
8048429:  add   %edx,%eax  
804842b:  mov   %eax,-0x4(%ebp)  
804842e:  mov   0xc(%ebp),%eax  
8048431:  mov   0x8(%ebp),%edx  
8048434:  sub   %eax,%edx  
8048436:  mov   %edx,%eax  
8048438:  mov   %eax,-0x8(%ebp)  
804843b:  mov   -0x8(%ebp),%eax  
804843e:  mov   -0x4(%ebp),%edx  
8048441:  add   %edx,%eax  
8048443:  leave  
8048444:  ret
```

Main's
frame

myfun's
frame



Now myfun gains control

```
int myfun(int a, int b) {  
    int c;  
    int d;  
    c = a + b;  
    d = a - b;  
    return c + d; }
```

0804841d <myfun>:

```
804841d:  push  %ebp  
804841e:  mov   %esp,%ebp  
8048420:  sub   $0x10,%esp  
8048423:  mov   0xc(%ebp),%eax  
8048426:  mov   0x8(%ebp),%edx  
8048429:  add   %edx,%eax  
804842b:  mov   %eax,-0x4(%ebp)  
804842e:  mov   0xc(%ebp),%eax  
8048431:  mov   0x8(%ebp),%edx  
8048434:  sub   %eax,%edx  
8048436:  mov   %edx,%eax  
8048438:  mov   %eax,-0x8(%ebp)  
804843b:  mov   -0x8(%ebp),%eax  
804843e:  mov   -0x4(%ebp),%edx  
8048441:  add   %edx,%eax  
8048443:  leave  
8048444:  ret
```

Main's
frame

myfun's
frame

Return addr

Caller's ebp

b: 0x2

a: 0x1

main's ret addr

main's ebp

c:0x3



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

← %ebp

← %esp

Now myfun gains control



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

```
int myfun(int a, int b) {  
    int c;  
    int d;  
    c = a + b;  
    d = a - b;  
    return c + d; }
```

0804841d <myfun>:

```
804841d:  push  %ebp  
804841e:  mov   %esp,%ebp  
8048420:  sub   $0x10,%esp  
8048423:  mov   0xc(%ebp),%eax  
8048426:  mov   0x8(%ebp),%edx  
8048429:  add   %edx,%eax  
804842b:  mov   %eax,-0x4(%ebp)  
804842e:  mov   0xc(%ebp),%eax  
8048431:  mov   0x8(%ebp),%edx  
8048434:  sub   %eax,%edx  
8048436:  mov   %edx,%eax  
8048438:  mov   %eax,-0x8(%ebp)  
804843b:  mov   -0x8(%ebp),%eax  
804843e:  mov   -0x4(%ebp),%edx  
8048441:  add   %edx,%eax  
8048443:  leave  
8048444:  ret
```

Main's
frame

myfun's
frame

Return addr

Caller's ebp

b: 0x2

a: 0x1

main's ret addr

mains' ebp

c:0x3

d:-1

← %ebp

← %ebp - 4

← %ebp - 8

← %esp

Now myfun gains control

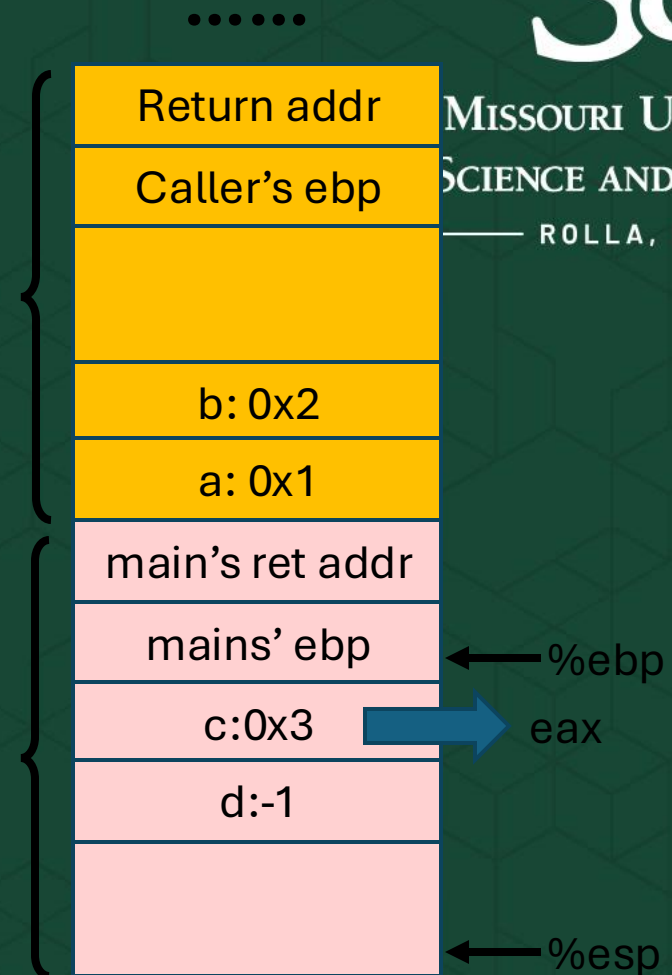
```
int myfun(int a, int b) {  
    int c;  
    int d;  
    c = a + b;  
    d = a - b;  
    return c + d; }
```

0804841d <myfun>:

```
804841d:  push  %ebp  
804841e:  mov   %esp,%ebp  
8048420:  sub   $0x10,%esp  
8048423:  mov   0xc(%ebp),%eax  
8048426:  mov   0x8(%ebp),%edx  
8048429:  add   %edx,%eax  
804842b:  mov   %eax,-0x4(%ebp)  
804842e:  mov   0xc(%ebp),%eax  
8048431:  mov   0x8(%ebp),%edx  
8048434:  sub   %eax,%edx  
8048436:  mov   %edx,%eax  
8048438:  mov   %eax,-0x8(%ebp)  
804843b:  mov   -0x8(%ebp),%eax  
804843e:  mov   -0x4(%ebp),%edx  
8048441:  add   %edx,%eax  
8048443:  leave  
8048444:  ret
```

Main's
frame

myfun's
frame



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Now myfun gains control

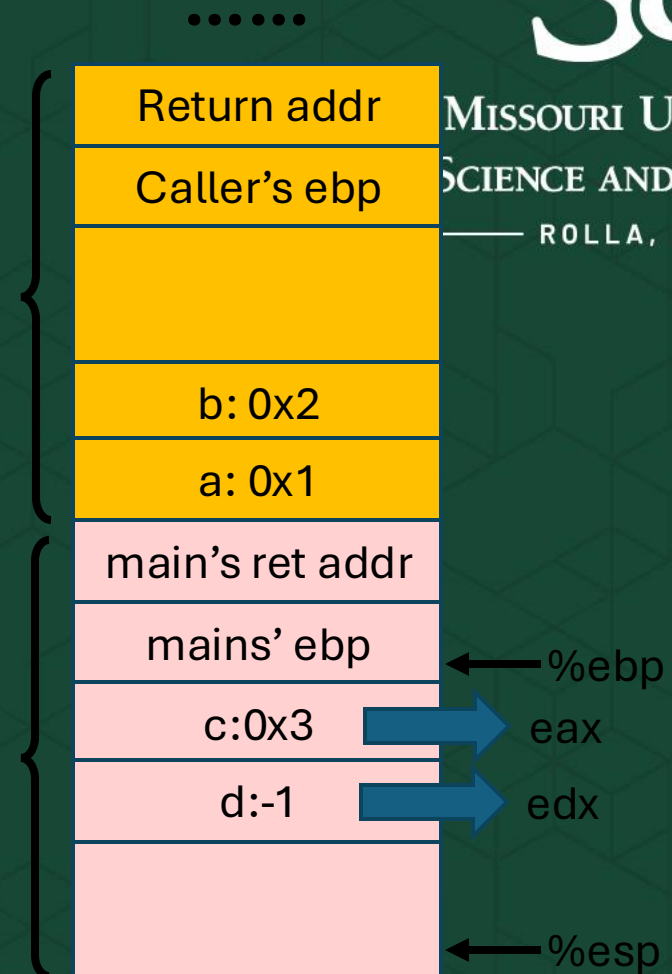
```
int myfun(int a, int b) {  
    int c;  
    int d;  
    c = a + b;  
    d = a - b;  
    return c + d; }
```

0804841d <myfun>:

```
804841d:  push  %ebp  
804841e:  mov   %esp,%ebp  
8048420:  sub   $0x10,%esp  
8048423:  mov   0xc(%ebp),%eax  
8048426:  mov   0x8(%ebp),%edx  
8048429:  add   %edx,%eax  
804842b:  mov   %eax,-0x4(%ebp)  
804842e:  mov   0xc(%ebp),%eax  
8048431:  mov   0x8(%ebp),%edx  
8048434:  sub   %eax,%edx  
8048436:  mov   %edx,%eax  
8048438:  mov   %eax,-0x8(%ebp)  
804843b:  mov   -0x8(%ebp),%eax  
804843e:  mov   -0x4(%ebp),%edx  
8048441:  add   %edx,%eax  
8048443:  leave  
8048444:  ret
```

Main's
frame

myfun's
frame



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Now myfun gains control

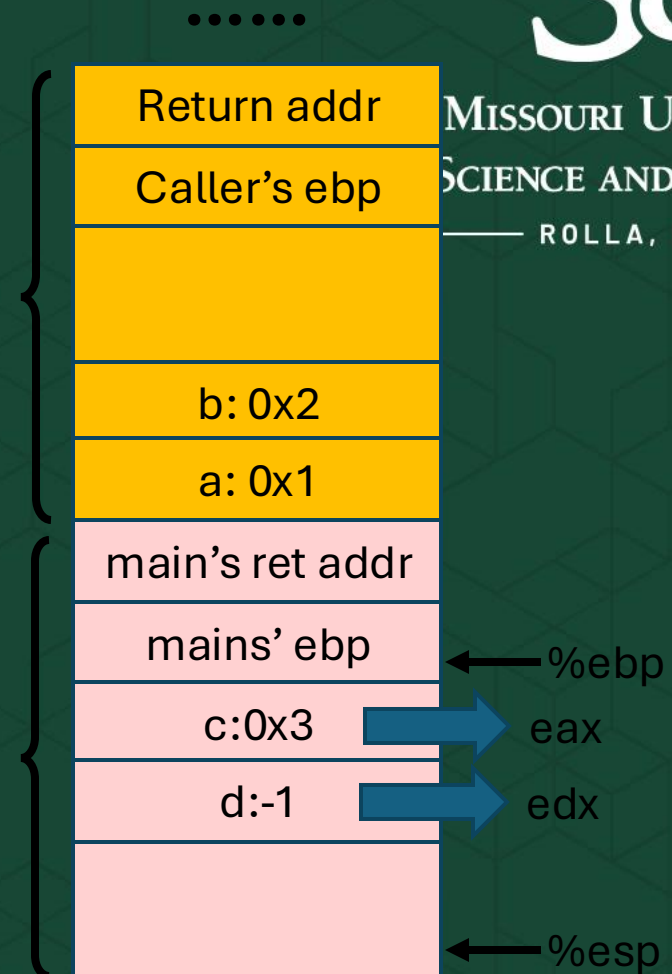
```
int myfun(int a, int b) {  
    int c;  
    int d;  
    c = a + b;  
    d = a - b;  
    return c + d; }
```

0804841d <myfun>:

```
804841d:  push  %ebp  
804841e:  mov   %esp,%ebp  
8048420:  sub   $0x10,%esp  
8048423:  mov   0xc(%ebp),%eax  
8048426:  mov   0x8(%ebp),%edx  
8048429:  add   %edx,%eax  
804842b:  mov   %eax,-0x4(%ebp)  
804842e:  mov   0xc(%ebp),%eax  
8048431:  mov   0x8(%ebp),%edx  
8048434:  sub   %eax,%edx  
8048436:  mov   %edx,%eax  
8048438:  mov   %eax,-0x8(%ebp)  
804843b:  mov   -0x8(%ebp),%eax  
804843e:  mov   -0x4(%ebp),%edx  
8048441:  add   %edx,%eax  
8048443:  leave  
8048444:  ret
```

Main's
frame

myfun's
frame



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Now myfun gains control



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

```
int myfun(int a, int b) {  
    int c;  
    int d;  
    c = a + b;  
    d = a - b;  
    return c + d; }
```

0804841d <myfun>:

```
804841d:  push  %ebp  
804841e:  mov   %esp,%ebp  
8048420:  sub   $0x10,%esp  
8048423:  mov   0xc(%ebp),%eax  
8048426:  mov   0x8(%ebp),%edx  
8048429:  add   %edx,%eax  
804842b:  mov   %eax,-0x4(%ebp)  
804842e:  mov   0xc(%ebp),%eax  
8048431:  mov   0x8(%ebp),%edx  
8048434:  sub   %eax,%edx  
8048436:  mov   %edx,%eax  
8048438:  mov   %eax,-0x8(%ebp)  
804843b:  mov   -0x8(%ebp),%eax  
804843e:  mov   -0x4(%ebp),%edx  
8048441:  add   %edx,%eax  
8048443:  leave ←  
8048444:  ret
```

Main's
frame

myfun's
frame

Return addr

Caller's ebp

b: 0x2

a: 0x1

main's ret addr

mains' ebp

c:0x3

d:-1

← %ebp

← %esp

```
mov %ebp, %esp  
pop %ebp
```

Now myfun gains control



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

```
int myfun(int a, int b) {  
    int c;  
    int d;  
    c = a + b;  
    d = a - b;  
    return c + d; }
```

0804841d <myfun>:

```
804841d:  push  %ebp  
804841e:  mov   %esp,%ebp  
8048420:  sub   $0x10,%esp  
8048423:  mov   0xc(%ebp),%eax  
8048426:  mov   0x8(%ebp),%edx  
8048429:  add   %edx,%eax  
804842b:  mov   %eax,-0x4(%ebp)  
804842e:  mov   0xc(%ebp),%eax  
8048431:  mov   0x8(%ebp),%edx  
8048434:  sub   %eax,%edx  
8048436:  mov   %edx,%eax  
8048438:  mov   %eax,-0x8(%ebp)  
804843b:  mov   -0x8(%ebp),%eax  
804843e:  mov   -0x4(%ebp),%edx  
8048441:  add   %edx,%eax  
8048443:  leave ←  
8048444:  ret
```

Main's
frame

myfun's
frame

Return addr

Caller's ebp

b: 0x2

a: 0x1

main's ret addr

mains' ebp

c:0x3

d:-1

%esp
and
%ebp

```
mov %ebp, %esp  
pop %ebp
```

Now myfun gains control

```
int myfun(int a, int b) {  
    int c;  
    int d;  
    c = a + b;  
    d = a - b;  
    return c + d; }
```

0804841d <myfun>:

```
804841d:  push %ebp  
804841e:  mov  %esp,%ebp  
8048420:  sub  $0x10,%esp  
8048423:  mov  0xc(%ebp),%eax  
8048426:  mov  0x8(%ebp),%edx  
8048429:  add  %edx,%eax  
804842b:  mov  %eax,-0x4(%ebp)  
804842e:  mov  0xc(%ebp),%eax  
8048431:  mov  0x8(%ebp),%edx  
8048434:  sub  %eax,%edx  
8048436:  mov  %edx,%eax  
8048438:  mov  %eax,-0x8(%ebp)  
804843b:  mov  -0x8(%ebp),%eax  
804843e:  mov  -0x4(%ebp),%edx  
8048441:  add  %edx,%eax  
8048443:  leave  
8048444:  ret
```

Main's
frame

myfun's
frame

Return addr

Caller's ebp

b: 0x2

a: 0x1

main's ret addr

main's ebp

c:0x3

d:-1

MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
ROLLA, MISSOURI

Return to main (give control again to main)
Return value is always saved to eax

Summary



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- Cdecl calling convention:
 - 32-bit x86 default for Linux and GCC
 - Use the stack frame pointer (ebp register) to reference arguments and local variables
 - The compiler can optimize in many aspects
 - Callee/caller's saves
 - Address alignment
 - ...
- 64-bit is different!
 - take advantage of the additional register space to pass more arguments in registers
 - Read arguments from registers, not memory → efficiency!



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Control Hijacking Attacks

materials are based on security courses at other institutions

Today's attacks

- **Much of software security is motivated by common attacks**
 - Topic: control hijacking attacks + some defenses
- Goals:
 - Understand in general terms what we're up against
 - Understand key characteristics of attacks
 - Understand vulnerabilities in software that make attacks possible
 - **so that we can better defend against attacks**



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Control hijacking attacks

- Attacker's goal:
 - Take over target machine, e.g., web server
 - Execute arbitrary attack code on target by hijacking application control flow
- Three typical examples
 - –Buffer overflow attacks
 - –Integer overflow attacks
 - –Format string vulnerabilities



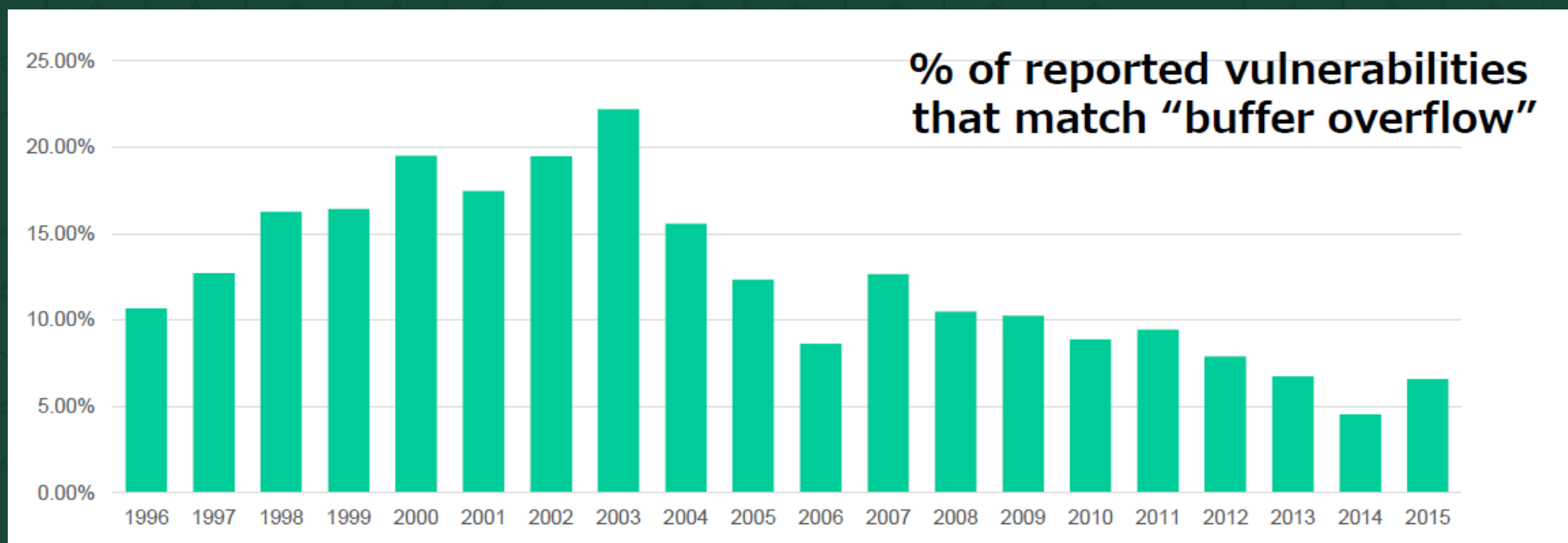
MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Buffer overflows

- Extremely common bug
 - First major exploit: 1988 Internet Worm (fingerd)



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —



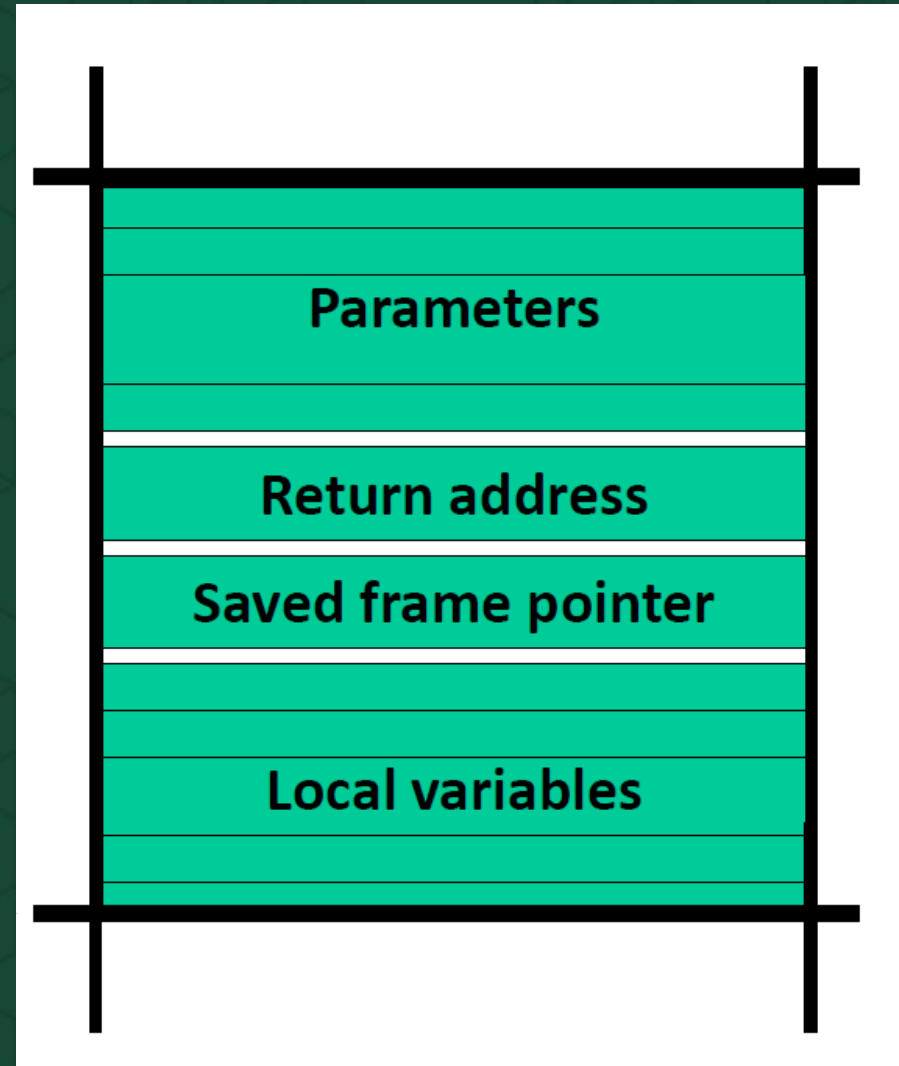
Anatomy of a Buffer Overflow

- Buffer: memory used to store user input, has fixed maximum size.
- Buffer overflow: when user input exceeds max buffer size, extra input goes into unexpected memory locations



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Stack Frame



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

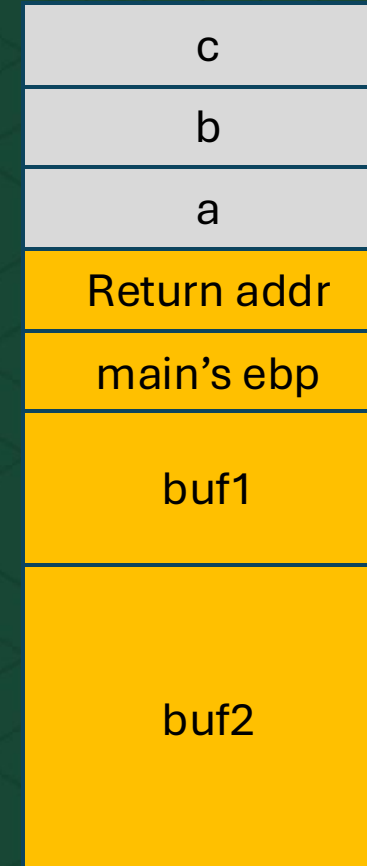
Stack frame of a function

```
void main()
{ func(1,2,3);
}

void func(int a, int b, int c)
{ char buffer1[5];
  char buffer2[10];
}
```

Function call: push

- Parameters, current %EIP as return address, saved frame pointer (ebp), local vars



Assume no alignment



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
ROLLA, MISSOURI

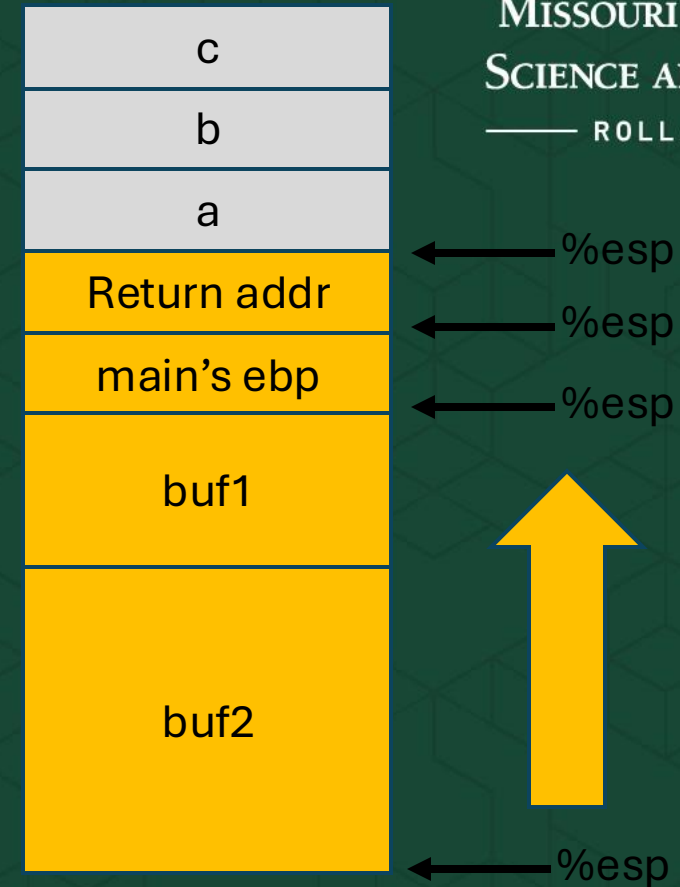
Stack frame of a function

```
void main()
{ func(1,2,3);
}

void func(int a, int b, int c)
{ char buffer1[5];
  char buffer2[10];
}
```

Function call: pop

- update frame pointer using ebp, update %EIP using RET

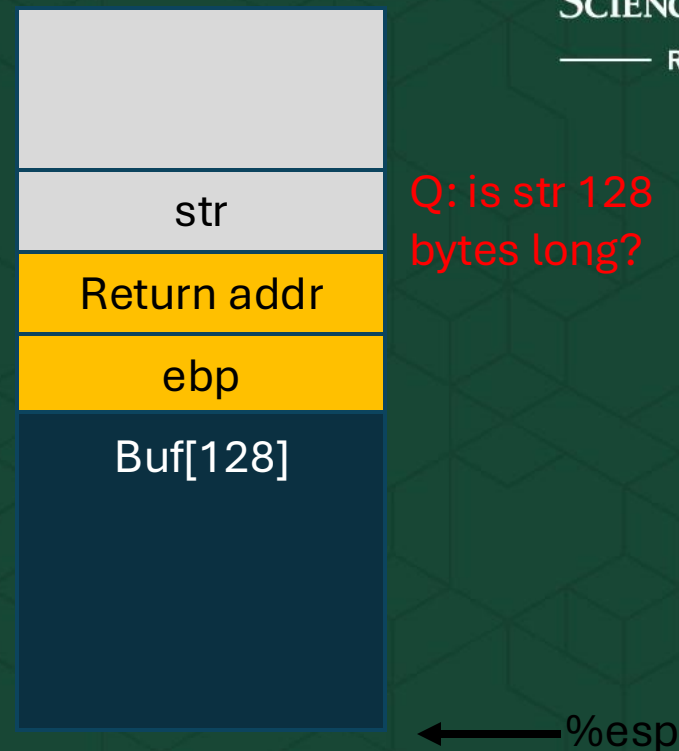


MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

What are buffer overflows?

- Suppose a web server contains a function:

```
void func(char *str)
{ char buf[128];
  strcpy(buf, str);
  do-something(buf);
}
```
- When the function is invoked the stack looks like:

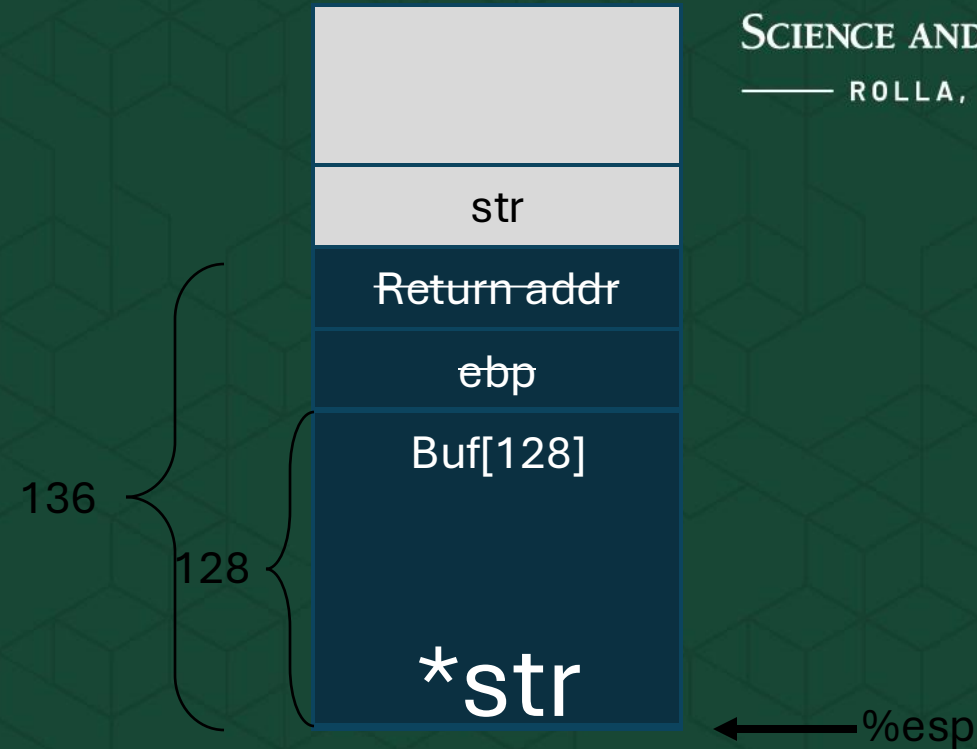


What are buffer overflows?

- Suppose a web server contains a function:

```
void func(char *str)
{ char buf[128];
  strcpy(buf, str);
  do-something(buf);
}
```

- What if str is 136 bytes long
 - $128 + 8$

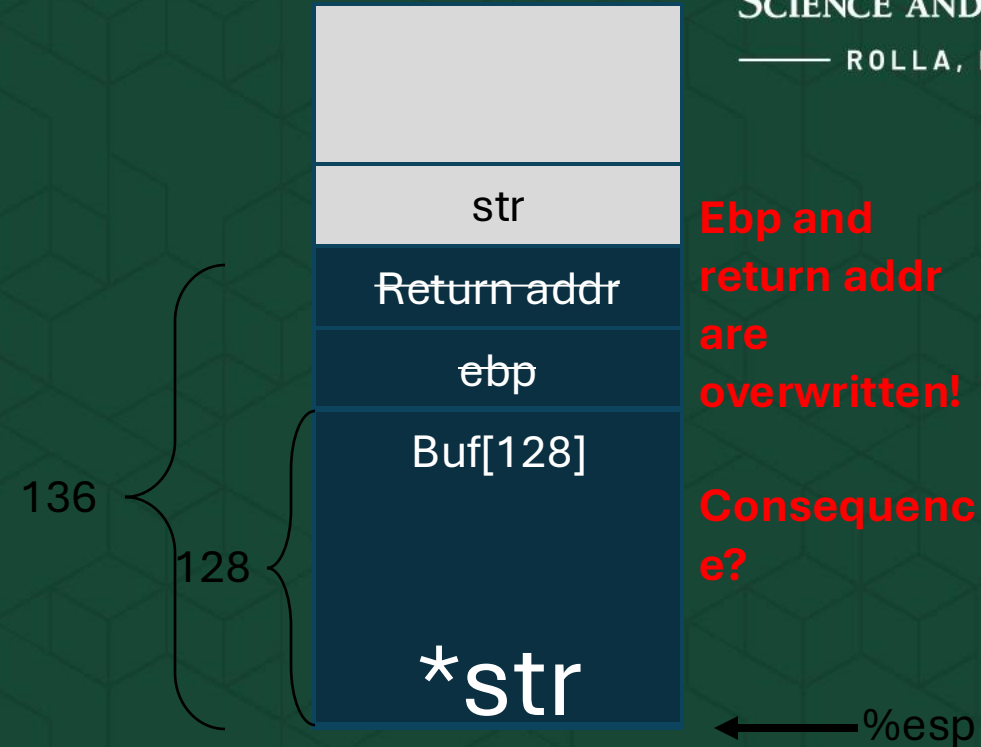


What are buffer overflows?

- Suppose a web server contains a function:

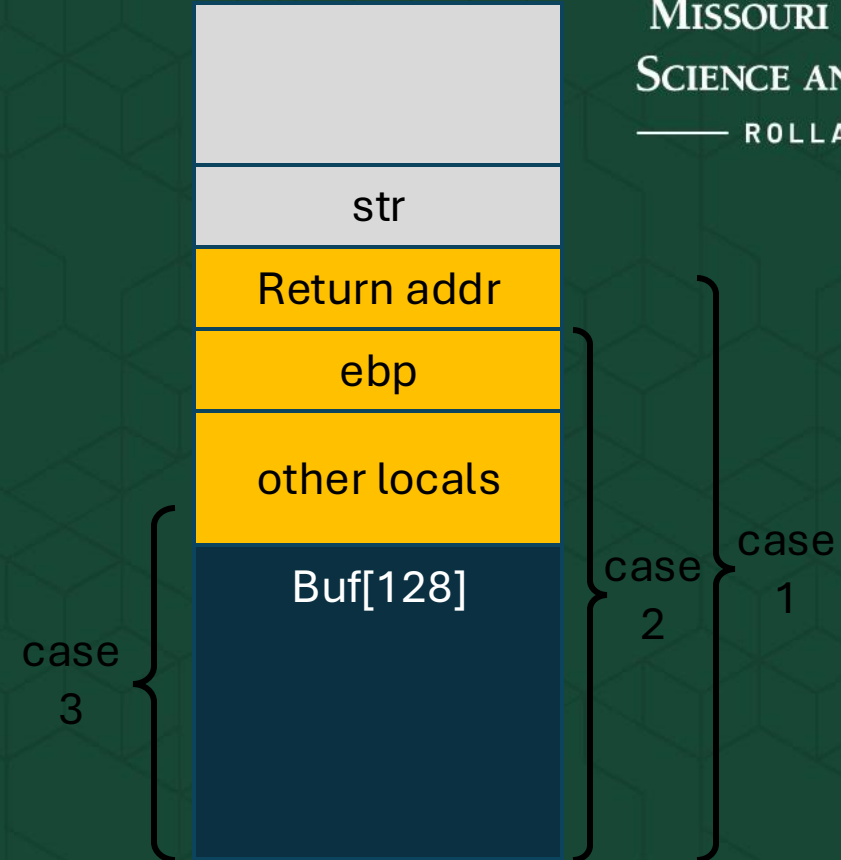
```
void func(char *str)
{ char buf[128];
  strcpy(buf, str);
  do-something(buf);
}
```

- What if str is 136 bytes long
 - 128 + 8



Some cases and consequences

1. The return address is overflowed
2. The memory that saves caller's ebp is overflowed
3. Some local variables are overflowed



Many unsafe C lib functions

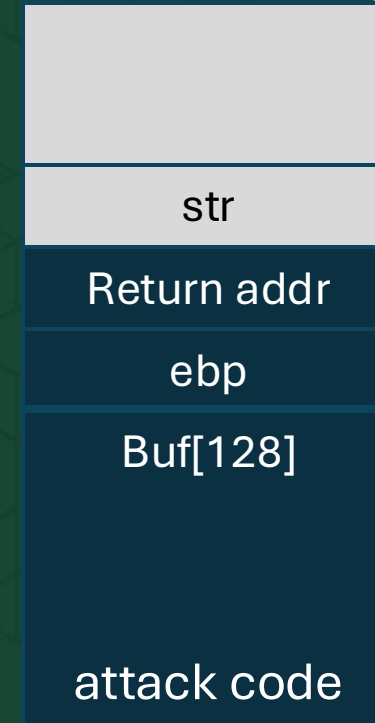
- **strcpy(char *dest, constchar *src)**
- **strcat(char *dest, constchar *src)**
- **gets(char *s)**
- **scanf(constchar *format, ...)**
- ...
- Why:
 - All have no boundary checking!



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Basic stack exploit: run a shell

- Problem: no range checking in strcpy()
- Suppose *str is such that after strcpy stack looks like:
 - **Program P: exec("/bin/sh")**
- When func() exits, the user will be given a shell!
 - Note: attack code runs in stack
 - To determine ret, guess position of stack when func() is called



The return addr points
to the start of attack
code

Run a shell

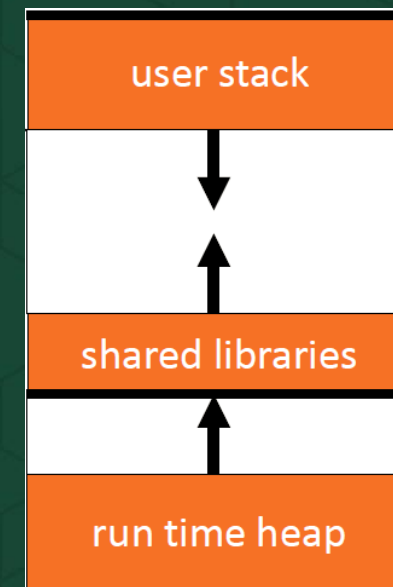
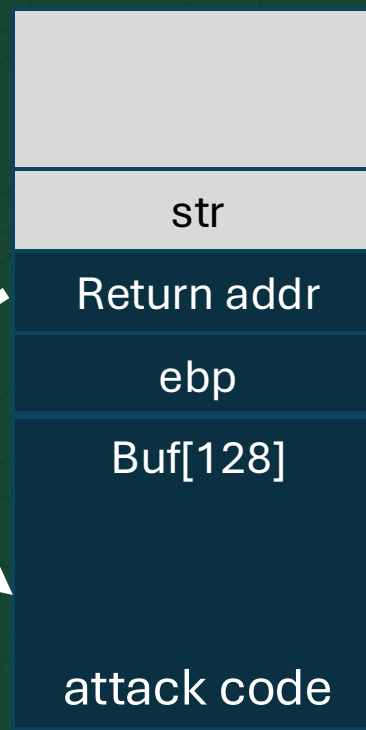


MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- Attacker carefully designs str to overflow the buffer such that

```
void func(char *str)
{ char buf[128];
  strcpy(buf, str);
  do-something(buf);
}
```

1. The return address points to the code in *str
 - Then the CPU starts to run the code in str



2. The buffer code calls 'exec' glibc function with '/bin/sh' argument



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Exploiting buffer overflows

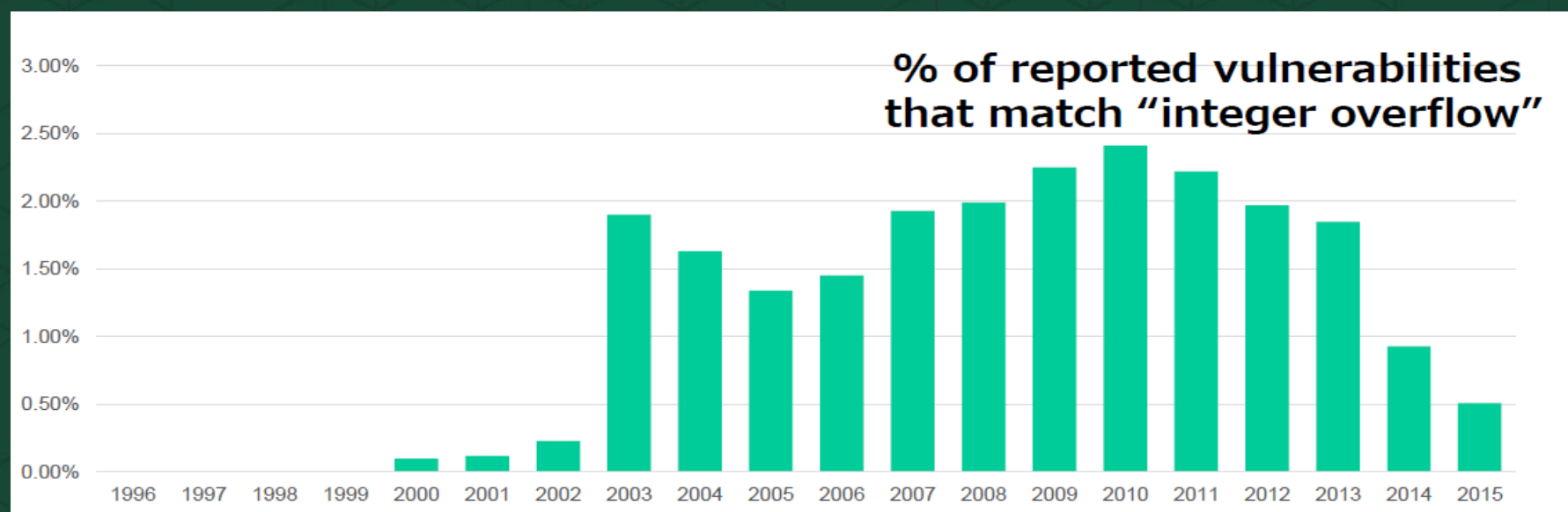
- **Suppose web server calls `func()` with *URL provided by the attacker***
 - Attacker sends a 200 byte URL, gets shell on web server
- **Some complications:**
 - Program P should not contain the '`\0`' character
 - why?
 - Overflow should not crash program before **`func()`** exits
 - Why?
- **Sample *remote* buffer overflows of this type:**
 - Picasa3 (1/2014)
 - Real Networks Real Player (1/2014)

Integer overflows



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- Integers are represented using a fixed number of bits
- Wrap around (modulo max value + 1) if value greater than max
 - $(\text{Value mod max value}) + 1$



Wrap around example

```
Main()
```

```
{
```

```
    example(12, 24);
```

```
}
```

The function example returns 0,
because 12+24 is less than 256

```
int example(unsigned char a, unsigned char b)
```

```
{
```

```
    if (a + b >= 256)
```

```
        return -1;
```

```
    return 0;
```

```
}
```



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Wrap around example

```
Main()
{
    example(12, 254);
}
```

```
int example(unsigned char a, unsigned char b)
{
    if (a + b >= 256)
        return -1;
    return 0;
}
```

The function example should return -1, because 12+254 is larger than 256. However, it indeed returns 0. Why?



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Wrap around example

```
Main()
{
    example(12, 254);
}

int example(unsigned char a, unsigned char b)
{
    if (a + b >= 256)
        return -1;
    return 0;
}
```

a and b are of type unsigned char, which is of size 8 bits. So, the maximum value of a and b is 0xff, or 255 in decimal.

C language by default considers $a + b$ the same type as a and b, i.e., unsigned char type. In this case, $a = 12$, $b = 254$, and $a + b = 266$, which is 100001010. The highest bit 1 is discarded to fit into the unsigned char type and therefore the result is 00001010, which is equal to 10 and less than 255. So, the function returns 0

What's wrong in the code

```
int intcatvars(char *buf1, char *buf2,  
               unsigned int len1, unsigned int len2)  
{  
    char mybuf[256];  
    if ((len1 + len2) > 256)  
        return -1;  
    memcpy(mybuf, buf1, len1);  
    memcpy(mybuf + len1, buf2, len2);  
    do  
    re  
}
```

Problem:

If $len1 = 0x104$ and $len2 = 0xfffffffffc$,
then $len1 + len2 = 0x100$ (decimal 256),
which allows buffer overflow attack



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

How to fix the code?

```
int intcatvars(char *buf1, char *buf2,  
              unsigned int len1, unsigned int len2)  
{  
    char mybuf[256];  
    if ((len1 + len2) > 256) /* Need a fix */  
        return -1;  
    memcpy(mybuf, buf1, len1);  
    memcpy(mybuf + len1, buf2, len2);  
    do_some_stuff(mybuf);  
    return 0;  
}
```



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Real Bug



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY

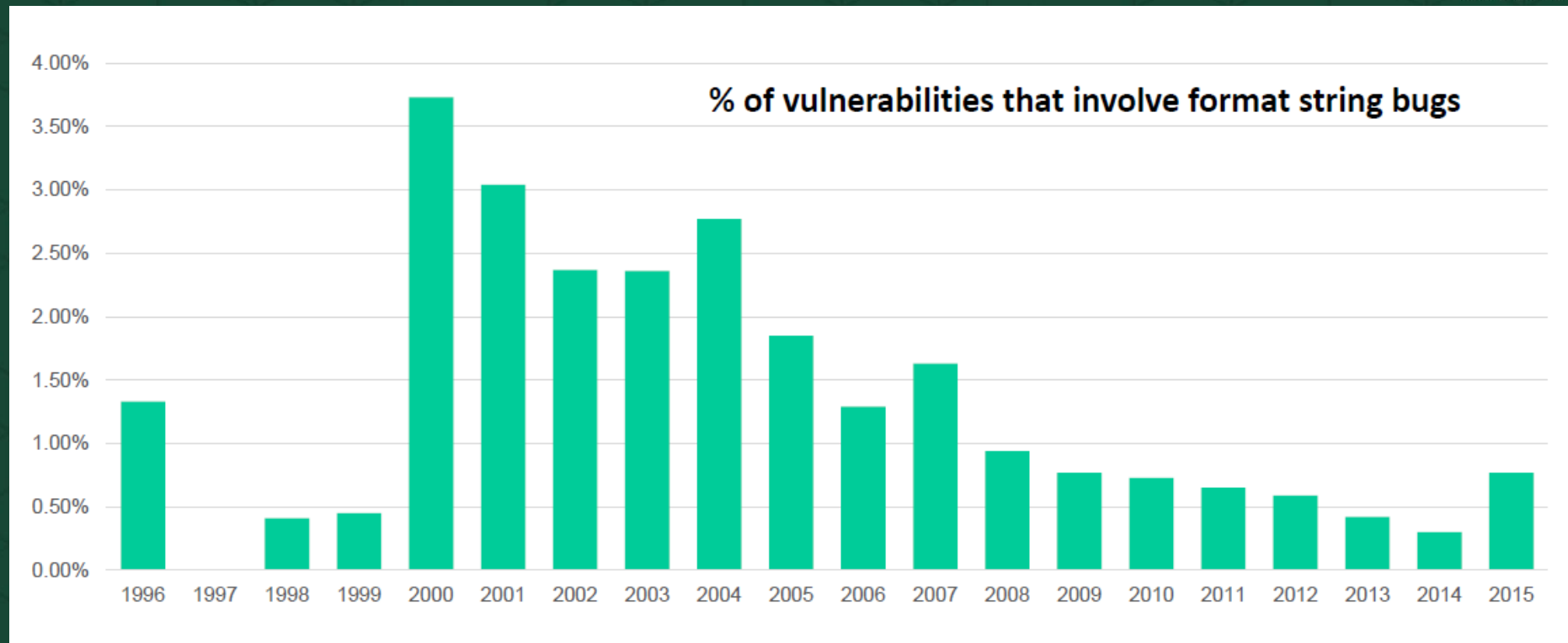
— ROLLA, MISSOURI —

```
/* 2.4.9: drivers/net/wan/farsync.c */  
...  
copy_from_user(&wrthdr, addr, sizeof wrthdr);  
if ( wrthdr.size + wrthdr.offset > FST_MEMSIZE )  
    return -ENXIO;  
copy_from_user(card>mem+wrthdr.offset,data,wrthdr.size)  
...
```

Format string bugs



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —



History

- First exploit discovered in June 2000
- Examples:
 - wu-ftpd2.* :remote root
 - Linux rpc.statd:remote root
 - IRIX telnetd:remote root
 - BSD chpass:local root



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Task

- Make a binary ***echo_info*** that prints the first argument a user types in the command line.
 - E.g., `echo_info hello` → a “hello” is printed

```
int main(int argc, char *argv[])
{
    if (argc > 1)
        printf(argv[1]);
}
```



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Vulnerable functions

- Any function using a format string
- Printing:
 - printf, fprintf, sprintf, ...
 - vprintf, vfprintf, vsprintf, ...
- Logging:
 - syslog, err, warn



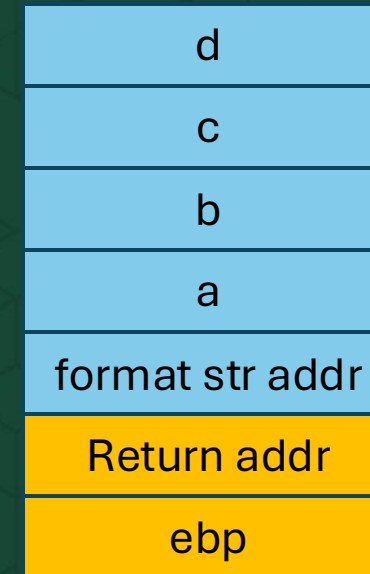
MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Format function execution

- When printf is called
 - It assumes that a, b, c, d are already pushed on the stack
- `printf("%d, %d, %d, %d\n", a, b, c, d)`

main stack
frame

printf stack
frame



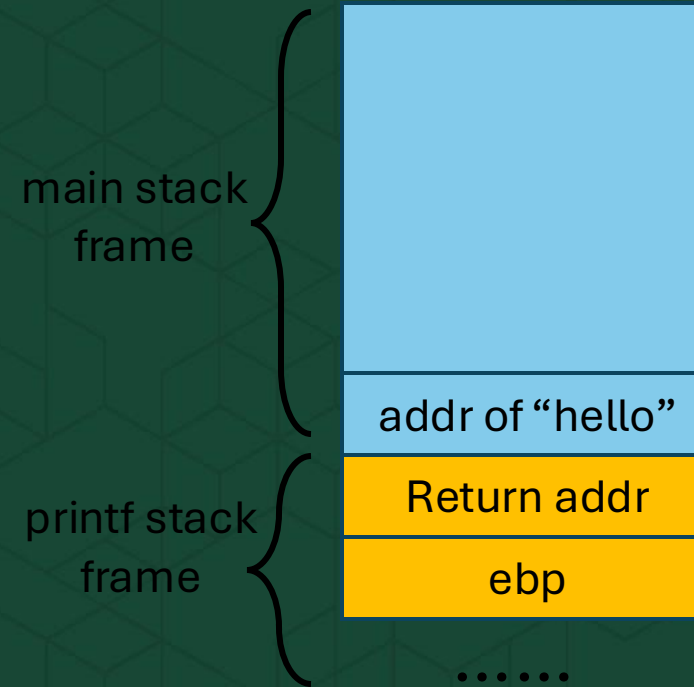
Format function parses the format string by reading a character at a time

- If not '%', the character is copied to the output
- If %, the character behind the '%' specifies the type of parameter that should be evaluated; **the parameter is located on the stack**

Return to our function

- `printf(argv[1]);`

The user enters “hello”



Return to our function

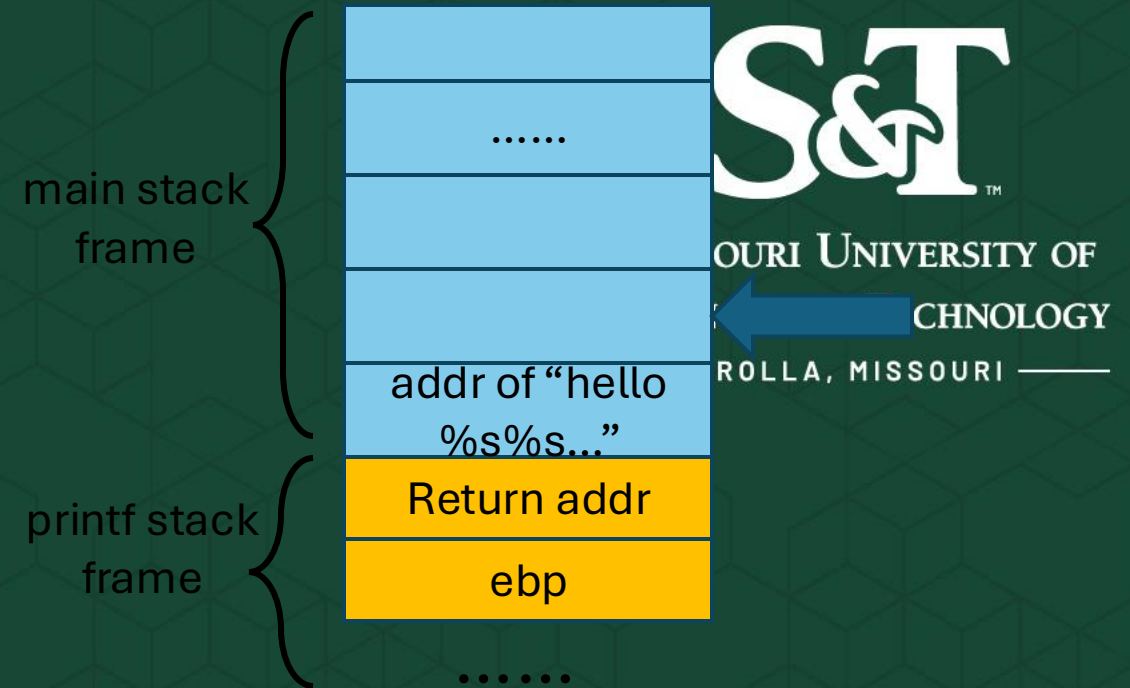
- `printf(argv[1]);`

What happens if the user enters

“hello `%s%s%s%s%s%s%s%s`”?

The `printf` function interprets “hello `%s%s%s%s%s%s%s%s`” as

a format string, outputs hello, and then checks the memory stack to find the corresponding character behind each %.



Return to our function

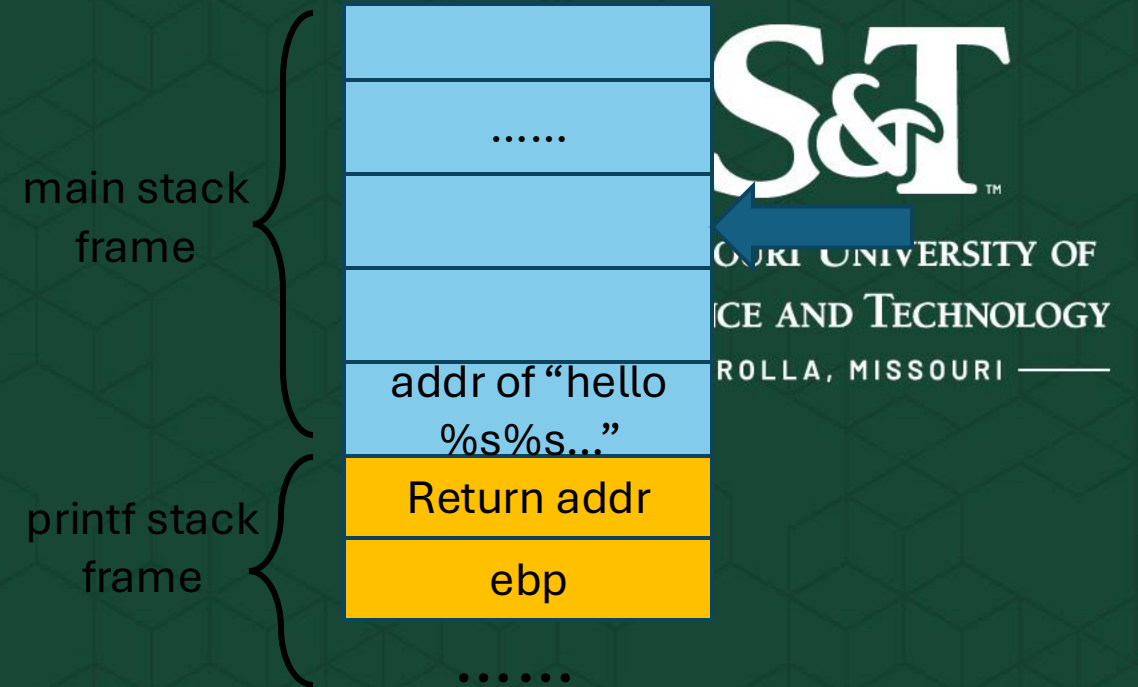
- `printf(argv[1]);`

What happens if the user enters

“hello %s%s%s%s%s%s%s%s”?

The printf function interprets “hello %s%s%s%s%s%s%s%s” as

a format string, outputs hello, and then checks the memory stack to find the corresponding character behind each %.



Return to our function

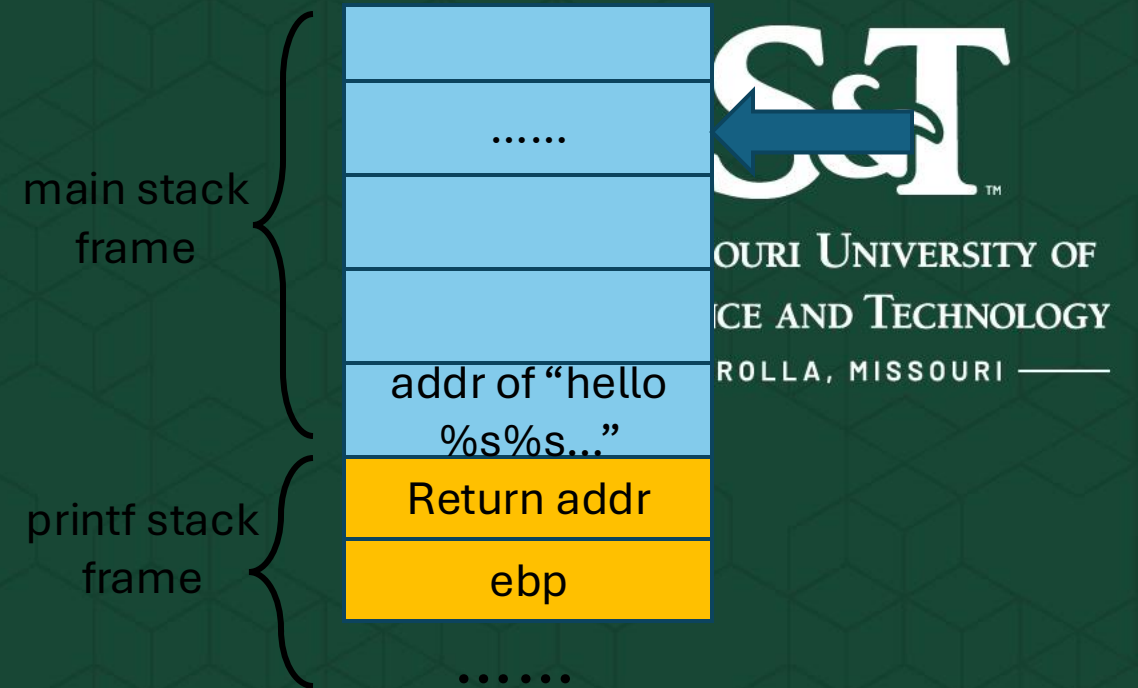
- `printf(argv[1]);`

What happens if the user enters

“hello `%s%s%s%s%s%s%s%s`”?

The `printf` function interprets “hello `%s%s%s%s%s%s%s%s`” as

a format string, outputs hello, and then outputs the content of the memory unit in the memory stack



Consequences

- In this example, the format string vulnerability can cause two consequences.
- First, if the printf function gets to an invalid memory address(e.g., outside of the memory boundary) and attempts to access it, the program will crash.
- Second, the values printed after the “Hello” text, are the values on the stack of the victim computer at the moment of running this example. This leaks information about the memory stack.



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

A right way to fix the bug

```
int main(int argc, char *argv[])
{
    if (argc > 1)
        printf(argv[1]);
}
```



```
int main(int argc, char *argv[])
{
    if (argc > 1)
        printf("%s", argv[1]);
}
```



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

More Info



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- The format string bug cause also cause the execution of arbitrary codes.
- For more information, please see the Exploiting Format String Vulnerabilities article from 2001.

<https://cs155.stanford.edu/papers/formatstring-1.2.pdf>

Summary: overflow exploits



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- Understanding C functions and the stack
 - Some familiarity with machine code
 - Calling convention
- **Attacker needs to know which CPU and OS are running on the target machine:**
 - Our examples are for x86 running Linux
 - Details vary slightly between CPUs and OSs:
 - Little endian vs. big endian
 - Stack frame structure (Linux vs. Windows)
 - Stack growth direction

Preventing hijacking attacks



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- **Fix bugs:**
 - Audit software
 - Automated tools: Coverity, Prefast/Prefix.
 - Rewrite software in a type safe language(Java, ML)
 - C does not protect language abstractions
 - Rewriting difficult for existing (legacy) code
- **Concede overflow, but prevent code execution**
- **Add runtime code (code instrumentation) to detect overflows exploits**
 - Halt process when overflow exploit detected
 - StackGuard, LibSafe, ...

} today



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

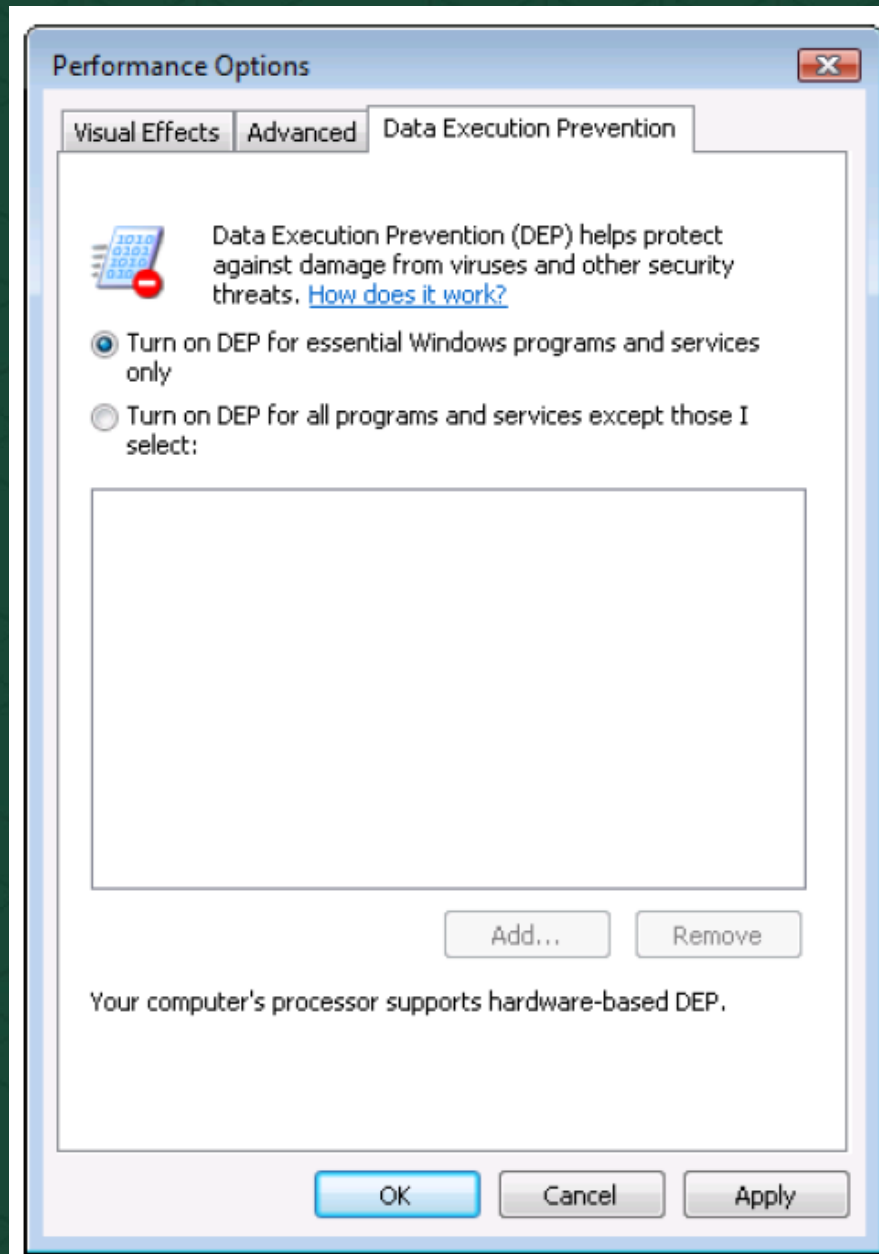
Marking memory as non-executable

- **Prevent overflow code execution by marking stack and heap as non-executable**

- Linux (via PaXproject); OpenBSD
- Windows since XP SP2 (DEP)

- **Limitations:**

- Some apps need executable heap (e.g., JITs)
- Does not defend against `return-to-libc' exploit

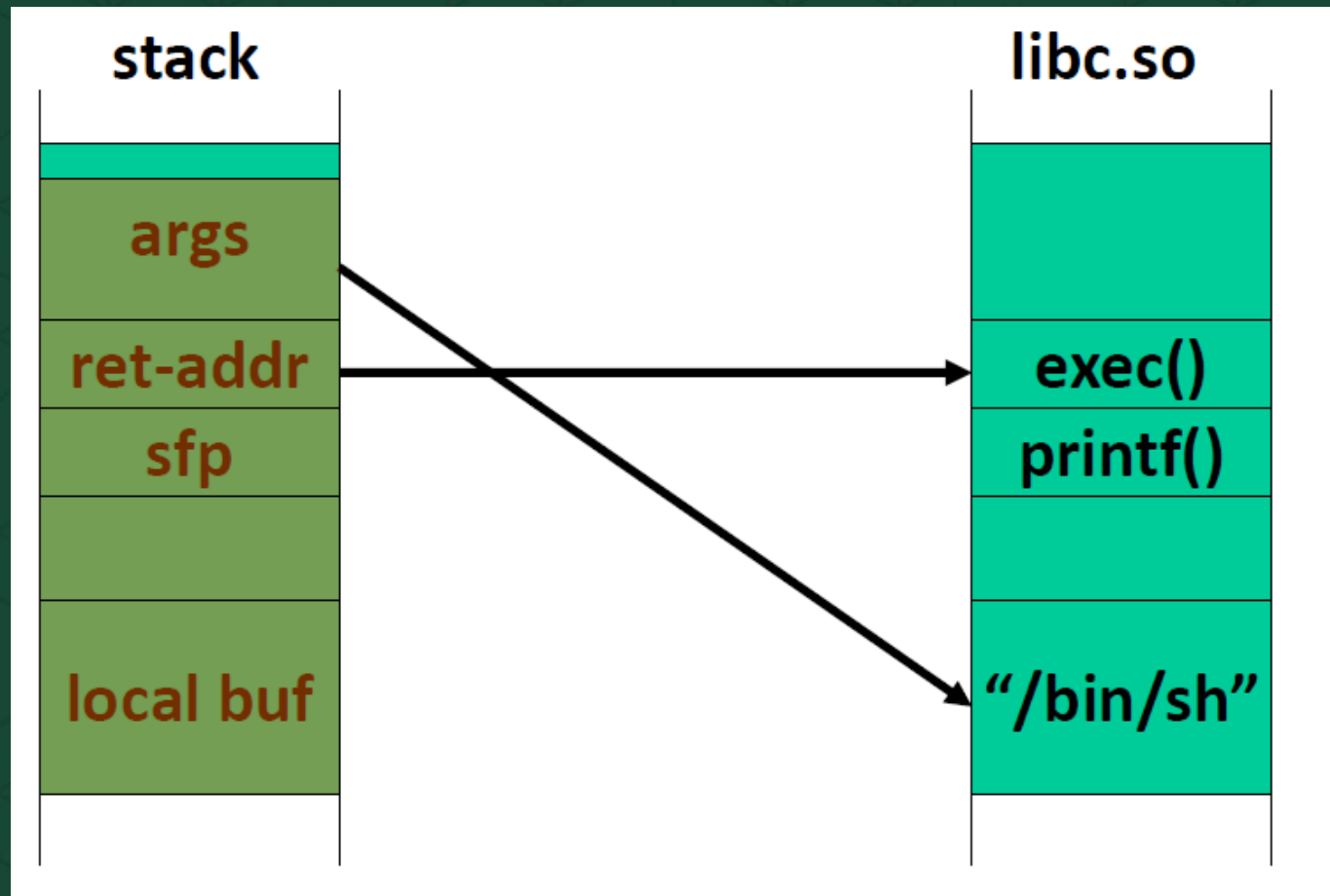


MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —



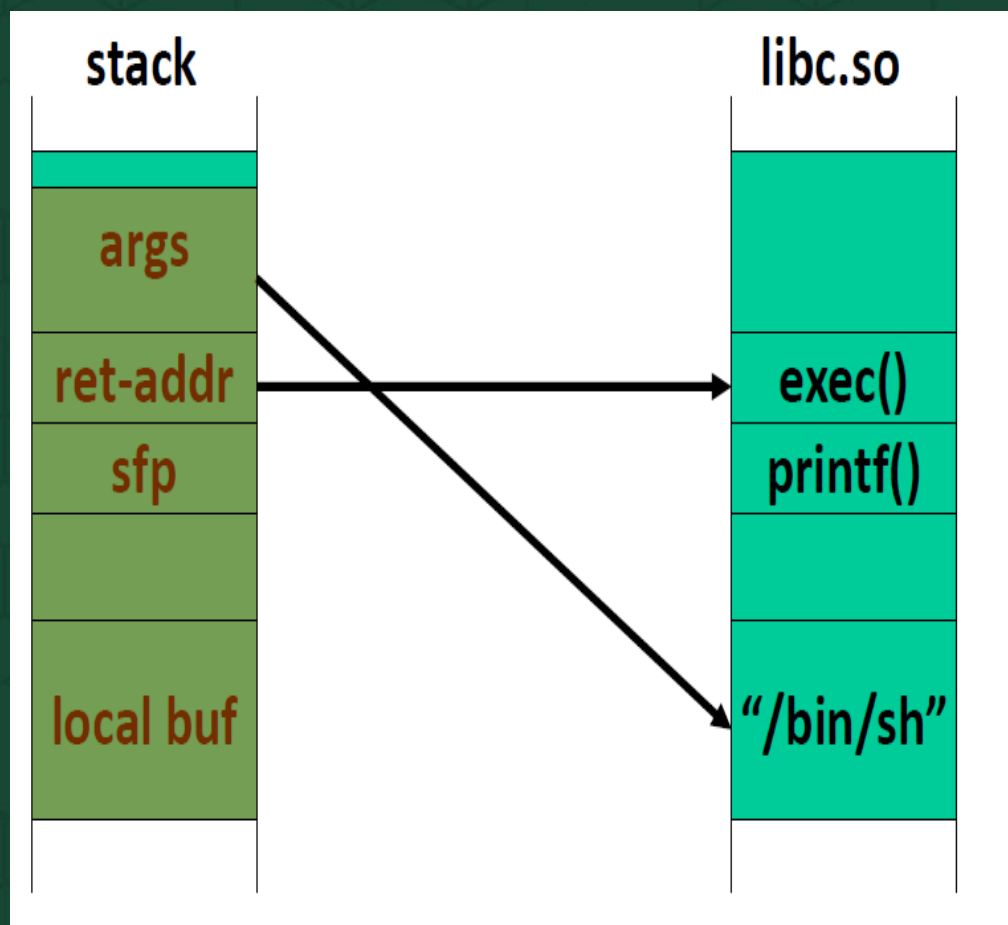
Return to libc

- Control hijacking without code injection



Randomization Technique

- **ASLR (Address Space Layout randomization)**



Map shared libraries to **random location in process memory**

- Attacker cannot jump directly to **exec()**
- Deployment:
 - Windows Vista: 8 bits of randomness for DLLs
 - Linux (via PaX): 16 bits of randomness for libraries
 - More effective on 64-bit architectures

ASLR example



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

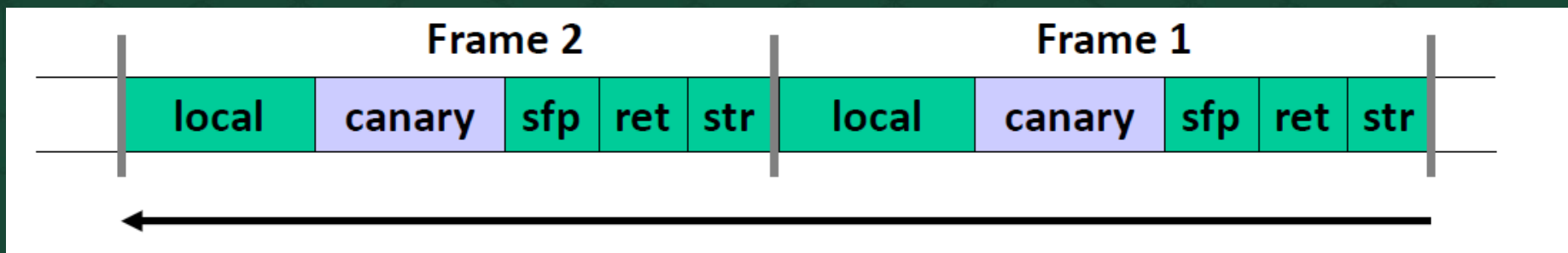
- **Booting Vista twice loads libraries into different locations:**

ntlanman.dll	0x6D7F0000	Microsoft® Lan Manager
ntmarta.dll	0x75370000	Windows NT MARTA provider
ntshrui.dll	0x6F2C0000	Shell extensions for sharing
ole32.dll	0x76160000	Microsoft OLE for Windows

ntlanman.dll	0x6DA90000	Microsoft® Lan Manager
ntmarta.dll	0x75660000	Windows NT MARTA provider
ntshrui.dll	0x6D9D0000	Shell extensions for sharing
ole32.dll	0x763C0000	Microsoft OLE for Windows

Run-time checking: StackGuard

- **Many run-time checking techniques ...**
- **Solution 1: StackGuard**
 - Run time tests for stack integrity
 - Embed “canaries” in stack frames and verify their integrity prior to function return



Canaries



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- ***Random canary:***
 - Choose random string at program startup
 - Insert canary string into every stack frame
 - Verify canary before returning from function
 - To corrupt random canary, attacker must learn current random string
- ***Terminator canary:***
 - Canary = 0, newline, linefeed, EOF
 - String functions will not copy beyond terminator
 - Attacker cannot use string functions to corrupt stack

StackGuard with Canaries

- StackGuard implemented as a GCC patch
- Program must be recompiled
 - Minimal performance effects: 8% for Apache
- When you test your own buffer overflow example
 - To compile a vulnerable binary, disable StackGuard in GCC with option `-fno-stack-protector`



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

StackGuard: Proof of Concept in C

- Goal: Illustrate the idea of using canaries

```
int myfun(char *str, int len) {  
    char buf[8];  
    memcpy(buf, str);  
    ...  
    do_something(buf);  
    ...  
}
```

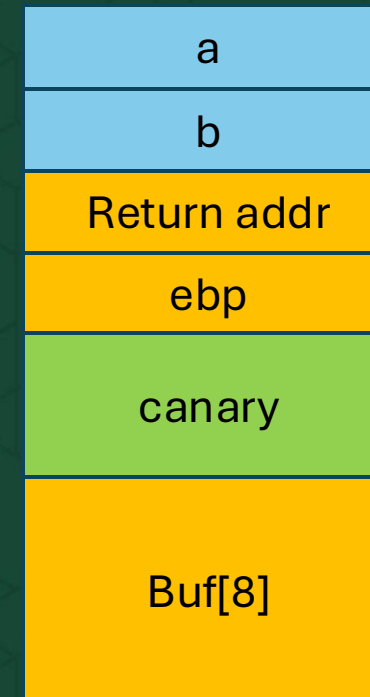


MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

What the compiler should do to add a canary?

- Goal: Illustrate the idea of using canaries

```
int myfun(char *str, int len) {
    PLACE_CANARY();
    char buf[8];
    memcpy(buf, str);
    ...
    do_something(buf);
    ...
    SAFE_RETURN();
}
```



StackGuard: Proof of Concept Implementation

- Using macros

```
int myfun(char *str, int len) {  
    PLACE_CANARY();  
    char buf[8];  
    memcpy(buf, str);  
    ...  
    do_something(buf);  
    ...  
    SAFE_RETURN();  
}
```

```
#define PLACE_CANARY(x) int __canary = __SECRET_VAR;  
#define SAFE_RETURN(x) if (__canary == __SECRET_VAR) \  
    return x; \  
else { \  
    fprintf(stderr, "Stack is overrun!\n"); \  
    exit(-1); }
```

StackShield

- At function prologue, copy return address **RET** and **ebp** to “safe” location (beginning of data segment)
- Before return, restore saved **RET** and **ebp**
- Implemented as assembler file processor (GCC)



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Type safety

- We have illustrated buffer overflow vulnerability in C.
 - Are other languages such as C++ and Java vulnerable?
- In any language that is compiled, the compiler usually does not do (and is not capable of doing) the checking required to identify potential buffer overflows and remedy the situation.
- In programming languages that are interpreted, sometimes the interpreter does provide such checking at runtime.



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Type safety

- Java and C# are compiled as well as interpreted.
 - Java and C# programs are compiled into a “bytecode” or “common language” that is interpreted at runtime
- A Java or CLR (Common Language Runtime) interpreter will enforce type safety and check for incorrect accesses, overflows of buffers, and code that runs past the bounds of an array.
- In Java or C#, you do not typically have to worry about buffer overflow vulnerabilities because the interpreter does such checking for you and will raise an exception if a buffer is overrun or overflowed.



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Summary: Control hijacking attacks



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- Attacker's goal:
 - Crash target machine
 - Take over target machine
 - Execute arbitrary attack code on target by hijacking application control flow
- Three typical examples
 - Buffer overflow attacks
 - Integer overflow attacks
 - Format string vulnerabilities
- Some protections: ASLR, StackGuard with Canaries, StackShield